



1D CNN Architecture for Variable-Length Sequential Data in RL

Overview: Recent research has shown that **1D convolutional networks** can model sequential data as effectively as (or even better than) recurrent networks in many domains ¹. This makes them a promising choice for reinforcement learning (RL) policies that must handle **variable-length sequences** of instructions. Below we describe a **proven Conv1D architecture** (inspired by the Temporal Convolutional Network, TCN) and how to adapt it to an actor-critic (A2C/PPO-style) model for your quantum circuit input. This architecture is *fully convolutional* along the sequence dimension (no RNNs), and is designed to handle arbitrary sequence lengths end-to-end ².

Temporal Convolutional Network (TCN) – Robust 1D CNN for Sequences

A **Temporal Convolutional Network (TCN)** is a deep 1D CNN architecture that has been empirically shown to outperform RNNs on various sequence modeling tasks ¹. It's well-suited to sequences of *variable length with repeating patterns* (like a series of quantum gate instructions). Key features of a TCN (or similar Conv1D sequence models) include:

- **Stacked 1D Convolutions with Dilation:** The network stacks multiple Conv1D layers that slide over the sequence (time) dimension. Using *dilated* convolutions allows the receptive field to grow exponentially with depth ³. For example, each layer can have a small kernel (e.g. 3) but with dilation factors doubling at each layer (1, 2, 4, 8, ...). This way, a relatively **deep network (e.g. 8-12 layers)** can capture long-range dependencies (hundreds of time-steps) without needing an impractically large kernel ⁴. In other words, dilation lets the model **cover very long sequences** while keeping kernels small and efficient.
- **Residual Blocks for Stability:** Rather than plain sequential conv layers, TCNs use **residual blocks** (skip connections) to ease training of deep networks ⁴. Each residual block typically contains **two Conv1D layers** (with dilation) plus non-linearities, and the block's input is added to its output (with a 1×1 conv on the input if needed to match dimensions) ⁵. This design lets layers **learn modifications to the identity mapping** instead of entire transformations, which greatly stabilizes training for deep CNNs ⁶ ⁷. In fact, experiments found that adding residual connections yields faster convergence and better final performance on sequence tasks ⁷. *Recommendation:* Wrap your Conv1d layers in residual blocks if you stack many of them, especially if you increase channel counts in deeper layers.
- **Normalization and Dropout:** Successful Conv1D architectures often include normalization and regularization in each block. The TCN reference model applied **weight normalization** to conv filters and **spatial dropout** (dropout entire channels) after conv layers ⁸. In practice, using **LayerNorm + dropout** (or BatchNorm) after conv layers is also effective to stabilize training – you already use

LayerNorm and PReLU, which is consistent with these best practices. These help deal with the high-dimensional input (e.g. 150 features per instruction) and prevent overfitting as the network grows deep.

- **Causal vs. Non-causal:** In some domains (like forecasting), conv layers are made *causal* (no future leakage). For your use-case (analyzing a circuit as a whole to output an action), **causality isn't required** – you can use “same” padding so the conv layers consider the full neighborhood around each instruction. The important part is that **input and output lengths remain the same** through the conv stack (achieved via padding) ⁹, so that every instruction's effects can propagate to the final representation.

Why this architecture fits: A TCN-style ConvNet naturally handles **arbitrary-length sequences** by sliding the filters over the instruction sequence ², just as you described (each “cell” of the kernel covers one instruction's features). It can learn repeating local patterns (via small kernel convolutions) and also long-term patterns (via stacked dilations). In fact, one major advantage of Conv1D over RNNs is the **flexible receptive field** – you can increase it by adding layers or larger dilation factors as needed ¹⁰. This means the network's “memory” of past instructions can be tuned to the task's needs more easily than an RNN's fixed state size. And unlike RNNs, a Conv1D processes the whole sequence in parallel, avoiding sequential bottlenecks and vanishing gradients through time ¹¹ ¹².

Proven performance: Conv1D architectures with these elements (dilations, residuals, gating or ReLU) have matched or surpassed recurrent nets in **audio generation, language modeling, and translation** tasks ¹. For example, DeepMind's WaveNet (audio) and Facebook's ConvS2S (translation) are fully convolutional sequence models that achieved state-of-art results using ~10-20 layers of 1D conv with residual/gated activations. The TCN formulation (Bai et al. 2018) distilled many of these best practices into a simple architecture that worked across diverse tasks ¹³ ¹⁴. All this gives confidence that a well-designed 1D CNN can **learn complex sequence patterns** (even something as unusual as quantum gate sequences) given sufficient depth and training.

Designing the Conv1D Network: Depth, Kernel Size, Channels

When building your 1D CNN policy network, consider the following design choices informed by the above architectures:

- **Kernel Size vs. Depth:** Instead of using an extremely large kernel (e.g. 130) in one layer, it's usually better to use a **small kernel (3-5)** and achieve a large receptive field by stacking layers (with dilation). Research shows that **small kernels focus on local patterns** (which is often important for “language-like” sequences), while stacking/dilating layers captures global context ¹⁵. Larger kernels can help very long-memory tasks converge a bit faster, but they didn't always improve final accuracy in language modeling ¹⁵. A reasonable approach is to start with kernel size 3 or 5 for all conv layers. With dilation doubling each layer, the receptive field grows exponentially – for example, 6 layers with kernel=3 gives a field of 127 time-steps, covering most circuits in your case. This way each layer learns **n-gram patterns of instructions**, and deeper layers combine them into longer motifs. (If you truly expect patterns spanning >100 instructions, you can increase depth or kernel slightly. But avoid one giant filter covering the whole sequence, as it may over-smooth or ignore the sequential structure.)

- **Number of Layers:** Use enough layers (with dilation) such that the network's receptive field $\geq$ the maximum sequence length you need to consider. Bai et al. note that **up to ~12 layers** might be needed if the model should potentially look across an entire long sequence ⁴. In practice, you might start with ~6–8 layers and adjust. Each residual block effectively counts as **one layer** in terms of receptive field growth (since both convs in the block typically use the same dilation before it increases in the next block). Ensure dilation resets or increases properly per layer. For example: layers 1–6 with dilation 1,2,4,8,16,32 (then you have receptive field $\sim 2^6$). If the sequence can be extremely long, you could either add more layers or use a second pass of dilation (some implementations cycle dilation after a certain point).
- **Channels (Filters) per Layer:** It's common to start with a moderate number of filters and **increase the channel count in deeper layers** (to capture higher-level features). For example, one successful design used 32 filters in the first conv layer, 64 in the second ¹⁶, and you could continue to 128, etc., as depth increases. This is similar to how image CNNs increase channels after each block. More channels = more feature maps to represent different instruction patterns. That said, increasing channels will increase model size, so find a balance. You might try a progression like $64 \rightarrow 64 \rightarrow 128 \rightarrow 128$ (depending on your compute budget). The **residual 1×1 conv** in each block will ensure the skip connection matches the new width ⁵ when you increase channels.
- **Activation Functions:** ReLU is standard (TCN used ReLU ⁸), but you are using **PReLU**, which is fine (it can help with negative values and maybe learns a slight bias in activation). The exact choice (ReLU, LeakyReLU, PReLU) is not critical as long as it's applied after each conv. Just ensure it's **within the residual block** (i.e., Conv $\rightarrow$ Activation $\rightarrow$ Norm $\rightarrow$ Conv $\rightarrow$ Activation $\rightarrow$ Norm, then add skip). Gated activations (e.g. GLU or gated tanh as in WaveNet/GLU) are another option. Some NLP conv models used **GLUs** which multiply a tanh and sigmoid branch for each conv output ¹⁷. Gating can improve modeling of certain data (TCN authors found GLU helped on word language modeling but not on other tasks) ¹⁸. If you're feeling experimental, you could try replacing ReLU with a gated unit in the conv blocks. However, **starting with a simple ReLU/PReLU is reasonable** – the **TCN without gating performed robustly across many tasks** ¹⁸.
- **Normalization & Regularization:** Continue using **Layer Normalization** after each conv (or each block) as you have – it helps with training stability, especially given the very different scales (gate indicators vs. continuous parameters) in your 150 features. Dropout is also important for RL to prevent overfitting to early trajectories. TCN used **Spatial Dropout** (drop entire channels) ⁵, which is effective for conv nets on sequences. In PyTorch, you can use `nn.Dropout1d` on the conv outputs to drop whole feature maps. This forces the network to not rely on any single feature channel too much. A dropout rate around 0.1–0.3 is a typical starting point. *Tip:* Place dropout **within the residual block** (e.g. after the first or second conv before adding the residual back). This was done in the original TCN design ⁸.
- **Pooling/Downsampling:** Many sequence CNNs (like TCN) do **not** downsample the time dimension at all – they keep output length equal to input length by padding ("same" convolution) ⁹. This is useful if you need a prediction at every time-step. In your case, you ultimately need a single output (policy & value) for the whole sequence, not at every step. There are two ways to handle this:
- **Global Pooling:** After the final conv layer, apply a **Global Average Pooling (1D)** across the sequence length, converting an output of shape $(seq_len, channels)$ into a single $1 \times channels$ vector. This yields a

fixed-length representation regardless of sequence length ¹⁹ ²⁰. Global Max Pooling is another option (taking the strongest feature activation over time). Global pooling is simple and makes the network truly length-agnostic – it’s often used in sequence classification CNNs to aggregate features ¹⁹.

- **Flatten with Masking/Padding:** Alternatively, you could pad all circuits to a max length and flatten the conv output, then use a fully-connected layer. However, this is less elegant and can introduce a dependence on the max length. Given you prefer an end-to-end conv solution, **Global Average Pooling is recommended** to summarize the sequence features ¹⁹. (It effectively acts like an adaptive stride that reduces the sequence dimension to 1.)

Both approaches end up with a **fixed-size feature vector** that represents the entire circuit. The rest of the network (policy/value heads) can then be standard fully-connected layers.

Integration into an Actor-Critic (A2C/PPO) Agent

To use this Conv1D architecture in an RL agent, you will treat it as a **feature extractor** that feeds into policy and value outputs:

- **Shared Conv Trunk:** In on-policy algorithms like A2C/PPO, it’s common to **share the feature extractor** between the policy and value networks ²¹. That is, you apply the Conv1D stack (with all the layers discussed above) to the input sequence *once*, get the final pooled feature vector, and then have **two output heads** on top of that: one for the action probability distribution $\pi(a|s)$, one for the value $V(s)$. Sharing the conv backbone saves computation and tends to work well unless the value function needs a very different representation. (Off-policy methods like SAC sometimes use separate encoders ²¹, but for simplicity you can start with a shared encoder.)
- **Policy Head:** Take the pooled feature vector and pass it through a fully-connected (dense) layer to produce a logit for each action. For example, a single `Linear(feats_dim, N_actions)` can output unnormalized scores for each of the N possible actions, which then go through a softmax to yield the policy distribution π . If you want a bit more capacity, you could insert one hidden layer (e.g. `Linear to 128 → ReLU → Linear to N_actions`), but often a direct projection is sufficient when the conv already did heavy lifting.
- **Value Head:** Similarly, take the same feature vector and feed it to a linear layer (or small MLP) to output a single scalar value V . For instance `Linear(feats_dim, 1)` gives the state-value. In an Actor-Critic, this value head shares the conv features but has its own weights for the linear output.
- **End-to-End Training:** The entire network (conv + heads) can be trained end-to-end with the RL algorithm’s loss functions (policy gradient loss + value loss, etc.). The conv layers will learn to extract those features from the instruction sequence that are relevant for predicting good actions and state value. Gradient backpropagation works through the conv blocks just like any neural net. Notably, because we use **global pooling**, the conv will receive signal about *the whole sequence’s outcome*. (If using “same” conv without pooling, you might choose the *last time-step output* to connect to the value/policy, but with circuits of varying length it’s safer to pool or average over all time-steps.)

Performance considerations: A Conv1D feature extractor can be heavier than a simple MLP, so monitor the inference speed if your sequences are long. However, convolution can be parallelized across the

sequence, so with a reasonable GPU this architecture should still run efficiently ¹¹. Also, by sharing the conv trunk between actor & critic, you roughly halve the computations compared to separate networks. Memory-wise, 1D conv layers are quite lean compared to sequence length – they don't need to unroll steps like RNNs, so memory usage scales with network depth rather than sequence length ²² (aside from the need to hold the whole sequence in the forward pass, which you're already doing).

Summary of the proposed architecture:

- **Input:** Quantum circuit as a sequence of instructions, each encoded as a 150-dim vector (features = gate type one-hot ± 1 , 3 parameters, 130 qubit role flags). So input tensor shape is `(batch, seq_len, 150)` if using batch-first, or `(batch, 150, seq_len)` for channels-first convention. (Many frameworks use batch $\times$ channels $\times$ length for Conv1D.)
- **Conv1D Stack:** e.g. 6–8 residual blocks, each with two Conv1D layers. Use kernel size 3–5, stride 1, **padding="same"**. Dilation grows each block (1,2,4,8,...). Start with 64 channels, increase to 128 in deeper layers. Each conv is followed by PReLU (or ReLU) and LayerNorm; each block has a skip connection. Apply dropout in blocks (e.g. 10–20% rate). This yields an output of shape `(batch, seq_len, C_final)` (or `(batch, C_final, seq_len)` in channels-first).
- **Global Pooling:** Average (or max) pool over the sequence dimension to get a `(batch, C_final)` vector. This ensures the network output doesn't depend on sequence length (works for any seq_len).¹⁹
- **Policy Head:** Dense layer from the pooled vector to N_{action} outputs, then softmax to get π .
- **Value Head:** Dense layer from the pooled vector to 1 output (linear).

This design gives you a **fully convolutional pipeline** from input sequence to output, with no manual feature engineering. It leverages **known-effective CNN patterns** (dilated conv + residual + normalization) to handle sequential patterns. You can confidently initialize with this architecture since it has been used in many settings (from text to time-series) with good results ¹³ ¹⁴. Adjust the depth or width as needed based on performance (e.g. if the agent struggles to use very long-term information, you might add more layers or slightly enlarge kernels). But overall, this TCN-based design should provide a **strong starting point** for an RL policy network dealing with variable-length quantum circuit inputs.

Sources:

- Bai et al., “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling” – introduced the TCN architecture and showed Conv1D outperforms RNNs on many sequence tasks ¹ ¹¹. Describes using dilations, residual blocks, etc., for long sequences ⁴ ⁸. Residual connections improved training stability and accuracy ⁷.
- MATLAB Documentation – example of a 1D CNN for sequence classification (uses two Conv1D+ReLU+LayerNorm layers with small kernel=5, followed by global average pooling and dense output) ¹⁶ ¹⁹. Demonstrates a simple but effective Conv1D architecture for variable-length data.
- Stable Baselines3 – RL library noting that for policy networks, CNN feature extractors can be shared between actor and critic in on-policy methods like A2C/PPO ²¹ (recommending a single shared conv backbone with separate output layers for efficiency). This aligns with the proposed architecture for integrating the Conv1D into your RL agent.

1 2 3 4 5 6 7 8 10 11 12 15 17 18 22 [1803.01271] An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

<https://ar5iv.labs.arxiv.org/html/1803.01271>

9 13 14 Temporal Convolutional Networks and Forecasting - Unit8

<https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/>

16 19 20 Sequence Classification Using 1-D Convolutions - MATLAB & Simulink

<https://www.mathworks.com/help/deeplearning/ug/sequence-classification-using-1-d-convolutions.html>

21 Policy Networks — Stable Baselines3 2.7.1a3 documentation

https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Shaojie Bai¹ J. Zico Kolter² Vladlen Koltun³

Abstract

For most deep learning practitioners, sequence modeling is synonymous with recurrent networks. Yet recent results indicate that convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation. Given a new sequence modeling task or dataset, which architecture should one use? We conduct a systematic evaluation of generic convolutional and recurrent architectures for sequence modeling. The models are evaluated across a broad range of standard tasks that are commonly used to benchmark recurrent networks. Our results indicate that a simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory. We conclude that the common association between sequence modeling and recurrent networks should be reconsidered, and convolutional networks should be regarded as a natural starting point for sequence modeling tasks. To assist related work, we have made code available at <http://github.com/locuslab/TCN>.

1. Introduction

Deep learning practitioners commonly regard recurrent architectures as the default starting point for sequence modeling tasks. The sequence modeling chapter in the canonical textbook on deep learning is titled “Sequence Modeling: Recurrent and Recursive Nets” (Goodfellow et al., 2016), capturing the common association of sequence modeling and recurrent architectures. A well-regarded recent online course on “Sequence Models” focuses exclusively on recurrent architectures (Ng, 2018).

¹Machine Learning Department, Carnegie Mellon University, Pittsburgh, PA, USA ²Computer Science Department, Carnegie Mellon University, Pittsburgh, PA, USA ³Intel Labs, Santa Clara, CA, USA. Correspondence to: Shaojie Bai <shaojie@cs.cmu.edu>, J. Zico Kolter <zkolter@cs.cmu.edu>, Vladlen Koltun <vkoltun@gmail.edu>.

On the other hand, recent research indicates that certain convolutional architectures can reach state-of-the-art accuracy in audio synthesis, word-level language modeling, and machine translation (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b). This raises the question of whether these successes of convolutional sequence modeling are confined to specific application domains or whether a broader reconsideration of the association between sequence processing and recurrent networks is in order.

We address this question by conducting a systematic empirical evaluation of convolutional and recurrent architectures on a broad range of sequence modeling tasks. We specifically target a comprehensive set of tasks that have been repeatedly used to compare the effectiveness of different recurrent network architectures. These tasks include polyphonic music modeling, word- and character-level language modeling, as well as synthetic stress tests that had been deliberately designed and frequently used to benchmark RNNs. Our evaluation is thus set up to compare convolutional and recurrent approaches to sequence modeling on the recurrent networks’ “home turf”.

To represent convolutional networks, we describe a generic temporal convolutional network (TCN) architecture that is applied across all tasks. This architecture is informed by recent research, but is deliberately kept simple, combining some of the best practices of modern convolutional architectures. It is compared to canonical recurrent architectures such as LSTMs and GRUs.

The results suggest that TCNs convincingly outperform baseline recurrent architectures across a broad range of sequence modeling tasks. This is particularly notable because the tasks include diverse benchmarks that have commonly been used to evaluate recurrent network designs (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Zhang et al., 2016). This indicates that the recent successes of convolutional architectures in applications such as audio processing are not confined to these domains.

To further understand these results, we analyze more deeply the memory retention characteristics of recurrent networks. We show that despite the theoretical ability of recurrent architectures to capture infinitely long history, TCNs exhibit

substantially longer memory, and are thus more suitable for domains where a long history is required.

To our knowledge, the presented study is the most extensive systematic comparison of convolutional and recurrent architectures on sequence modeling tasks. The results suggest that the common association between sequence modeling and recurrent networks should be reconsidered. The TCN architecture appears not only more accurate than canonical recurrent networks such as LSTMs and GRUs, but also simpler and clearer. It may therefore be a more appropriate starting point in the application of deep networks to sequences.

2. Background

Convolutional networks (LeCun et al., 1989) have been applied to sequences for decades (Sejnowski & Rosenberg, 1987; Hinton, 1989). They were used prominently for speech recognition in the 80s and 90s (Waibel et al., 1989; Bottou et al., 1990). ConvNets were subsequently applied to NLP tasks such as part-of-speech tagging and semantic role labelling (Collobert & Weston, 2008; Collobert et al., 2011; dos Santos & Zadrozny, 2014). More recently, convolutional networks were applied to sentence classification (Kalchbrenner et al., 2014; Kim, 2014) and document classification (Zhang et al., 2015; Conneau et al., 2017; Johnson & Zhang, 2015; 2017). Particularly inspiring for our work are the recent applications of convolutional architectures to machine translation (Kalchbrenner et al., 2016; Gehring et al., 2017a;b), audio synthesis (van den Oord et al., 2016), and language modeling (Dauphin et al., 2017).

Recurrent networks are dedicated sequence models that maintain a vector of hidden activations that are propagated through time (Elman, 1990; Werbos, 1990; Graves, 2012). This family of architectures has gained tremendous popularity due to prominent applications to language modeling (Sutskever et al., 2011; Graves, 2013; Hermans & Schrauwen, 2013) and machine translation (Sutskever et al., 2014; Bahdanau et al., 2015). The intuitive appeal of recurrent modeling is that the hidden state can act as a representation of everything that has been seen so far in the sequence. Basic RNN architectures are notoriously difficult to train (Bengio et al., 1994; Pascanu et al., 2013) and more elaborate architectures are commonly used instead, such as the LSTM (Hochreiter & Schmidhuber, 1997) and the GRU (Cho et al., 2014). Many other architectural innovations and training techniques for recurrent networks have been introduced and continue to be actively explored (El Hihi & Bengio, 1995; Schuster & Paliwal, 1997; Gers et al., 2002; Koutnik et al., 2014; Le et al., 2015; Ba et al., 2016; Wu et al., 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018).

Multiple empirical studies have been conducted to evaluate the effectiveness of different recurrent architectures. These studies have been motivated in part by the many degrees of freedom in the design of such architectures. Chung et al. (2014) compared different types of recurrent units (LSTM vs. GRU) on the task of polyphonic music modeling. Pascanu et al. (2014) explored different ways to construct deep RNNs and evaluated the performance of different architectures on polyphonic music modeling, character-level language modeling, and word-level language modeling. Jozefowicz et al. (2015) searched through more than ten thousand different RNN architectures and evaluated their performance on various tasks. They concluded that if there were “architectures much better than the LSTM”, then they were “not trivial to find”. Greff et al. (2017) benchmarked the performance of eight LSTM variants on speech recognition, handwriting recognition, and polyphonic music modeling. They also found that “none of the variants can improve upon the standard LSTM architecture significantly”. Zhang et al. (2016) systematically analyzed the connecting architectures of RNNs and evaluated different architectures on character-level language modeling and on synthetic stress tests. Melis et al. (2018) benchmarked LSTM-based architectures on word-level and character-level language modeling, and concluded that “LSTMs outperform the more recent models”.

Other recent works have aimed to combine aspects of RNN and CNN architectures. This includes the Convolutional LSTM (Shi et al., 2015), which replaces the fully-connected layers in an LSTM with convolutional layers to allow for additional structure in the recurrent layers; the Quasi-RNN model (Bradbury et al., 2017) that interleaves convolutional layers with simple recurrent layers; and the dilated RNN (Chang et al., 2017), which adds dilations to recurrent architectures. While these combinations show promise in combining the desirable aspects of both types of architectures, our study here focuses on a comparison of generic convolutional and recurrent architectures.

While there have been multiple thorough evaluations of RNN architectures on representative sequence modeling tasks, we are not aware of a similarly thorough comparison of convolutional and recurrent approaches to sequence modeling. (Yin et al. (2017) have reported a comparison of convolutional and recurrent networks for sentence-level and document-level classification tasks. In contrast, sequence modeling calls for architectures that can synthesize whole sequences, element by element.) Such comparison is particularly intriguing in light of the aforementioned recent success of convolutional architectures in this domain. Our work aims to compare generic convolutional and recurrent architectures on typical sequence modeling tasks that are commonly used to benchmark RNN variants themselves (Hermans & Schrauwen, 2013; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016).

3. Temporal Convolutional Networks

We begin by describing a generic architecture for convolutional sequence prediction. Our aim is to distill the best practices in convolutional network design into a simple architecture that can serve as a convenient but powerful starting point. We refer to the presented architecture as a temporal convolutional network (TCN), emphasizing that we adopt this term not as a label for a truly new architecture, but as a simple descriptive term for a family of architectures. (Note that the term has been used before (Lea et al., 2017).) The distinguishing characteristics of TCNs are: 1) the convolutions in the architecture are causal, meaning that there is no information “leakage” from future to past; 2) the architecture can take a sequence of any length and map it to an output sequence of the same length, just as with an RNN. Beyond this, we emphasize how to build very long effective history sizes (i.e., the ability for the networks to look very far into the past to make a prediction) using a combination of very deep networks (augmented with residual layers) and dilated convolutions.

Our architecture is informed by recent convolutional architectures for sequential data (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b), but is distinct from all of them and was designed from first principles to combine simplicity, autoregressive prediction, and very long memory. For example, the TCN is much simpler than WaveNet (van den Oord et al., 2016) (no skip connections across layers, conditioning, context stacking, or gated activations).

Compared to the language modeling architecture of Dauphin et al. (2017), TCNs do not use gating mechanisms and have much longer memory.

3.1. Sequence Modeling

Before defining the network structure, we highlight the nature of the sequence modeling task. Suppose that we are given an input sequence $x_0, \dots, x_T$, and wish to predict some corresponding outputs $y_0, \dots, y_T$ at each time. The key constraint is that to predict the output y_t for some time t , we are constrained to only use those inputs that have been previously observed: $x_0, \dots, x_t$. Formally, a sequence modeling network is any function $f : \mathcal{X}^{T+1} \rightarrow \mathcal{Y}^{T+1}$ that produces the mapping

$$\hat{y}_0, \dots, \hat{y}_T = f(x_0, \dots, x_T) \quad (1)$$

if it satisfies the causal constraint that y_t depends only on $x_0, \dots, x_t$ and not on any “future” inputs $x_{t+1}, \dots, x_T$. The goal of learning in the sequence modeling setting is to find a network f that minimizes some expected loss between the actual outputs and the predictions, $L(y_0, \dots, y_T, f(x_0, \dots, x_T))$, where the sequences and outputs are drawn according to some distribution.

This formalism encompasses many settings such as autoregressive prediction (where we try to predict some signal given its past) by setting the target output to be simply the input shifted by one time step. It does not, however, directly capture domains such as machine translation, or sequence-to-sequence prediction in general, since in these cases the entire input sequence (including “future” states) can be used to predict each output (though the techniques can naturally be extended to work in such settings).

3.2. Causal Convolutions

As mentioned above, the TCN is based upon two principles: the fact that the network produces an output of the same length as the input, and the fact that there can be no leakage from the future into the past. To accomplish the first point, the TCN uses a 1D fully-convolutional network (FCN) architecture (Long et al., 2015), where each hidden layer is the same length as the input layer, and zero padding of length (kernel size $- 1$) is added to keep subsequent layers the same length as previous ones. To achieve the second point, the TCN uses *causal convolutions*, convolutions where an output at time t is convolved only with elements from time t and earlier in the previous layer.

To put it simply: TCN = 1D FCN + causal convolutions.

Note that this is essentially the same architecture as the time delay neural network proposed nearly 30 years ago by Waibel et al. (1989), with the sole tweak of zero padding to ensure equal sizes of all layers.

A major disadvantage of this basic design is that in order to achieve a long effective history size, we need an extremely deep network or very large filters, neither of which were particularly feasible when the methods were first introduced. Thus, in the following sections, we describe how techniques from modern convolutional architectures can be integrated into a TCN to allow for both very deep networks and very long effective history.

3.3. Dilated Convolutions

A simple causal convolution is only able to look back at a history with size linear in the depth of the network. This makes it challenging to apply the aforementioned causal convolution on sequence tasks, especially those requiring longer history. Our solution here, following the work of van den Oord et al. (2016), is to employ dilated convolutions that enable an exponentially large receptive field (Yu & Koltun, 2016). More formally, for a 1-D sequence input $\mathbf{x} \in \mathbb{R}^n$ and a filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation F on element s of the sequence is defined as

$$F(s) = (\mathbf{x} *_{\text{d}} f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i} \quad (2)$$

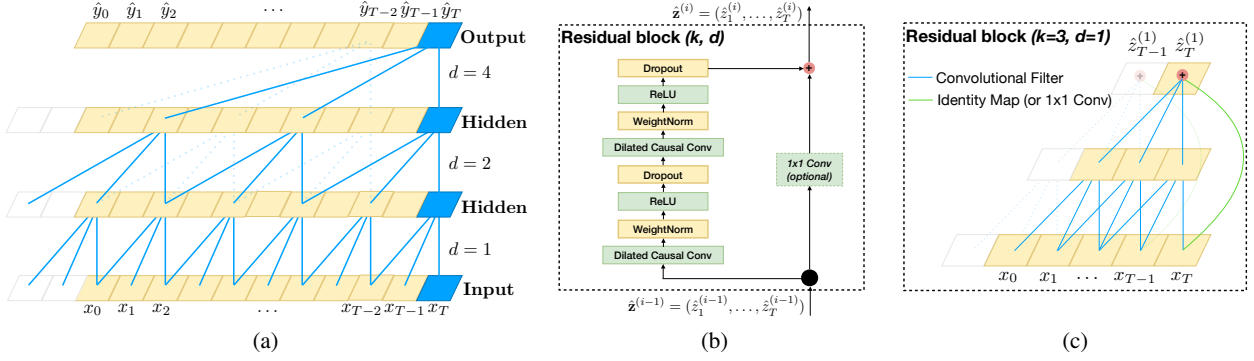


Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. A 1x1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

where d is the dilation factor, k is the filter size, and $s - d \cdot i$ accounts for the direction of the past. Dilation is thus equivalent to introducing a fixed step between every two adjacent filter taps. When $d = 1$, a dilated convolution reduces to a regular convolution. Using larger dilation enables an output at the top level to represent a wider range of inputs, thus effectively expanding the receptive field of a ConvNet.

This gives us two ways to increase the receptive field of the TCN: choosing larger filter sizes k and increasing the dilation factor d , where the effective history of one such layer is $(k - 1)d$. As is common when using dilated convolutions, we increase d exponentially with the depth of the network (i.e., $d = O(2^i)$ at level i of the network). This ensures that there is some filter that hits each input within the effective history, while also allowing for an extremely large effective history using deep networks. We provide an illustration in Figure 1(a).

3.4. Residual Connections

A residual block (He et al., 2016) contains a branch leading out to a series of transformations $\mathcal{F}$, whose outputs are added to the input $\mathbf{x}$ of the block:

$$o = \text{Activation}(\mathbf{x} + \mathcal{F}(\mathbf{x})) \quad (3)$$

This effectively allows layers to learn modifications to the identity mapping rather than the entire transformation, which has repeatedly been shown to benefit very deep networks.

Since a TCN’s receptive field depends on the network depth n as well as filter size k and dilation factor d , stabilization of deeper and larger TCNs becomes important. For example, in a case where the prediction could depend on a history of size 2^{12} and a high-dimensional input sequence, a network of up to 12 layers could be needed. Each layer, more specifically, consists of multiple filters for feature extraction. In our design of the generic TCN model, we therefore employ a generic residual module in place of a convolutional layer.

The residual block for our baseline TCN is shown in Figure 1(b). Within a residual block, the TCN has two layers of dilated causal convolution and non-linearity, for which we used the rectified linear unit (ReLU) (Nair & Hinton, 2010). For normalization, we applied weight normalization (Salimans & Kingma, 2016) to the convolutional filters. In addition, a spatial dropout (Srivastava et al., 2014) was added after each dilated convolution for regularization: at each training step, a whole channel is zeroed out.

However, whereas in standard ResNet the input is added directly to the output of the residual function, in TCN (and ConvNets in general) the input and output could have different widths. To account for discrepant input-output widths, we use an additional 1x1 convolution to ensure that element-wise addition $\oplus$ receives tensors of the same shape (see Figure 1(b,c)).

3.5. Discussion

We conclude this section by listing several advantages and disadvantages of using TCNs for sequence modeling.

- **Parallelism.** Unlike in RNNs where the predictions for later timesteps must wait for their predecessors to complete, convolutions can be done in parallel since the same filter is used in each layer. Therefore, in both training and evaluation, a long input sequence can be processed as a whole in TCN, instead of sequentially as in RNN.
- **Flexible receptive field size.** A TCN can change its receptive field size in multiple ways. For instance, stacking more dilated (causal) convolutional layers, using larger dilation factors, or increasing the filter size are all viable options (with possibly different interpretations). TCNs thus afford better control of the model’s memory size, and are easy to adapt to different domains.
- **Stable gradients.** Unlike recurrent architectures, TCN has a backpropagation path different from the temporal direction of the sequence. TCN thus avoids the problem

of exploding/vanishing gradients, which is a major issue for RNNs (and which led to the development of LSTM, GRU, HF-RNN (Martens & Sutskever, 2011), etc.).

- **Low memory requirement for training.** Especially in the case of a long input sequence, LSTMs and GRUs can easily use up a lot of memory to store the partial results for their multiple cell gates. However, in a TCN the filters are shared across a layer, with the backpropagation path depending only on network depth. Therefore in practice, we found gated RNNs likely to use up to a multiplicative factor more memory than TCNs.
- **Variable length inputs.** Just like RNNs, which model inputs with variable lengths in a recurrent way, TCNs can also take in inputs of arbitrary lengths by sliding the 1D convolutional kernels. This means that TCNs can be adopted as drop-in replacements for RNNs for sequential data of arbitrary length.

There are also two notable disadvantages to using TCNs.

- **Data storage during evaluation.** In evaluation/testing, RNNs only need to maintain a hidden state and take in a current input x_t in order to generate a prediction. In other words, a “summary” of the entire history is provided by the fixed-length set of vectors h_t , and the actual observed sequence can be discarded. In contrast, TCNs need to take in the raw sequence up to the effective history length, thus possibly requiring more memory during evaluation.
- **Potential parameter change for a transfer of domain.** Different domains can have different requirements on the amount of history the model needs in order to predict. Therefore, when transferring a model from a domain where only little memory is needed (i.e., small k and d) to a domain where much longer memory is required (i.e., much larger k and d), TCN may perform poorly for not having a sufficiently large receptive field.

4. Sequence Modeling Tasks

We evaluate TCNs and RNNs on tasks that have been commonly used to benchmark the performance of different RNN sequence modeling architectures (Hermans & Schrauwen, 2013; Chung et al., 2014; Pascanu et al., 2014; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016). The intention is to conduct the evaluation on the “home turf” of RNN sequence models. We use a comprehensive set of synthetic stress tests along with real-world datasets from multiple domains.

The adding problem. In this task, each input consists of a length- n sequence of depth 2, with all values randomly chosen in $[0, 1]$, and the second dimension being all zeros except for two elements that are marked by 1. The objective is to sum the two random values whose second dimensions

are marked by 1. Simply predicting the sum to be 1 should give an MSE of about 0.1767. First introduced by Hochreiter & Schmidhuber (1997), the adding problem has been used repeatedly as a stress test for sequence models (Martens & Sutskever, 2011; Pascanu et al., 2013; Le et al., 2015; Arjovsky et al., 2016; Zhang et al., 2016).

Sequential MNIST and P-MNIST. Sequential MNIST is frequently used to test a recurrent network’s ability to retain information from the distant past (Le et al., 2015; Zhang et al., 2016; Wisdom et al., 2016; Cooijmans et al., 2016; Krueger et al., 2017; Jing et al., 2017). In this task, MNIST images (LeCun et al., 1998) are presented to the model as a 784×1 sequence for digit classification. In the more challenging P-MNIST setting, the order of the sequence is permuted at random (Le et al., 2015; Arjovsky et al., 2016; Wisdom et al., 2016; Krueger et al., 2017).

Copy memory. In this task, each input sequence has length $T + 20$. The first 10 values are chosen randomly among the digits $1, \dots, 8$, with the rest being all zeros, except for the last 11 entries that are filled with the digit ‘9’ (the first ‘9’ is a delimiter). The goal is to generate an output of the same length that is zero everywhere except the last 10 values after the delimiter, where the model is expected to repeat the 10 values it encountered at the start of the input. This task was used in prior works such as Zhang et al. (2016); Arjovsky et al. (2016); Wisdom et al. (2016); Jing et al. (2017).

JSB Chorales and Nottingham. JSB Chorales (Allan & Williams, 2005) is a polyphonic music dataset consisting of the entire corpus of 382 four-part harmonized chorales by J. S. Bach. Each input is a sequence of elements. Each element is an 88-bit binary code that corresponds to the 88 keys on a piano, with 1 indicating a key that is pressed at a given time. Nottingham is a polyphonic music dataset based on a collection of 1,200 British and American folk tunes, and is much larger than JSB Chorales. JSB Chorales and Nottingham have been used in numerous empirical investigations of recurrent sequence modeling (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Greff et al., 2017). The performance on both tasks is measured in terms of negative log-likelihood (NLL).

PennTreebank. We used the PennTreebank (PTB) (Marcus et al., 1993) for both character-level and word-level language modeling. When used as a character-level language corpus, PTB contains 5,059K characters for training, 396K for validation, and 446K for testing, with an alphabet size of 50. When used as a word-level language corpus, PTB contains 888K words for training, 70K for validation, and 79K for testing, with a vocabulary size of 10K. This is a highly studied but relatively small language modeling dataset (Miyamoto & Cho, 2016; Krueger et al., 2017; Merity et al., 2017).

Wikitext-103. Wikitext-103 (Merity et al., 2016) is almost

Table 1. Evaluation of TCNs and recurrent architectures on synthetic stress tests, polyphonic music modeling, character-level language modeling, and word-level language modeling. The generic TCN architecture outperforms canonical recurrent networks across a comprehensive suite of tasks and datasets. Current state-of-the-art results are listed in the supplement. ^h means that higher is better. ^ℓ means that lower is better.

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

110 times as large as PTB, featuring a vocabulary size of about 268K. The dataset contains 28K Wikipedia articles (about 103 million words) for training, 60 articles (about 218K words) for validation, and 60 articles (246K words) for testing. This is a more representative and realistic dataset than PTB, with a much larger vocabulary that includes many rare words, and has been used in Merity et al. (2016); Grave et al. (2017); Dauphin et al. (2017).

LAMBADA. Introduced by Paperno et al. (2016), LAMBADA is a dataset comprising 10K passages extracted from novels, with an average of 4.6 sentences as context, and 1 target sentence the last word of which is to be predicted. This dataset was built so that a person can easily guess the missing word when given the context sentences, but not when given only the target sentence without the context sentences. Most of the existing models fail on LAMBADA (Paperno et al., 2016; Grave et al., 2017). In general, better results on LAMBADA indicate that a model is better at capturing information from longer and broader context. The training data for LAMBADA is the full text of 2,662 novels with more than 200M words. The vocabulary size is about 93K.

text8. We also used the text8 dataset for character-level language modeling (Mikolov et al., 2012). text8 is about 20 times larger than PTB, with about 100M characters from Wikipedia (90M for training, 5M for validation, and 5M for testing). The corpus contains 27 unique alphabets.

5. Experiments

We compare the generic TCN architecture described in Section 3 to canonical recurrent architectures, namely LSTM, GRU, and vanilla RNN, with standard regularizations. All

experiments reported in this section used exactly the same TCN architecture, just varying the depth of the network n and occasionally the kernel size k so that the receptive field covers enough context for predictions. We use an exponential dilation $d = 2^i$ for layer i in the network, and the Adam optimizer (Kingma & Ba, 2015) with learning rate 0.002 for TCN, unless otherwise noted. We also empirically find that gradient clipping helped convergence, and we pick the maximum norm for clipping from $[0.3, 1]$. When training recurrent models, we use grid search to find a good set of hyperparameters (in particular, optimizer, recurrent drop $p \in [0.05, 0.5]$, learning rate, gradient clipping, and initial forget-gate bias), while keeping the network around the same size as TCN. No other architectural elaborations, such as gating mechanisms or skip connections, were added to either TCNs or RNNs. Additional details and controlled experiments are provided in the supplementary material.

5.1. Synopsis of Results

A synopsis of the results is shown in Table 1. Note that on several of these tasks, the generic, canonical recurrent architectures we study (e.g., LSTM, GRU) are not the state-of-the-art. (See the supplement for more details.) With this caveat, the results strongly suggest that the generic TCN architecture *with minimal tuning* outperforms canonical recurrent architectures across a broad variety of sequence modeling tasks that are commonly used to benchmark the performance of recurrent architectures themselves. We now analyze these results in more detail.

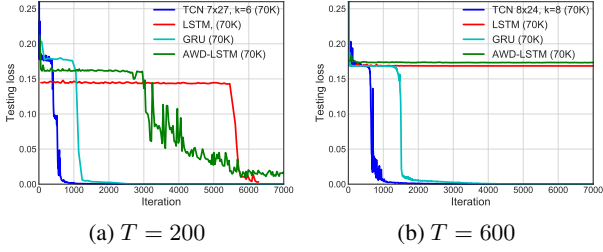


Figure 2. Results on the adding problem for different sequence lengths T . TCNs outperform recurrent architectures.

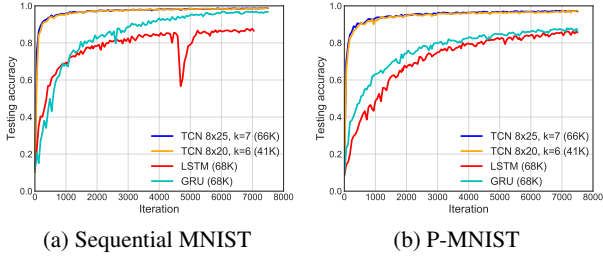


Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

5.2. Synthetic Stress Tests

The adding problem. Convergence results for the adding problem, for problem sizes $T = 200$ and 600 , are shown in Figure 2. All models were chosen to have roughly 70K parameters. TCNs quickly converged to a virtually perfect solution (i.e., MSE near 0). GRUs also performed quite well, albeit slower to converge than TCNs. LSTMs and vanilla RNNs performed significantly worse.

Sequential MNIST and P-MNIST. Convergence results on sequential and permuted MNIST, run over 10 epochs, are shown in Figure 3. All models were configured to have roughly 70K parameters. For both problems, TCNs substantially outperform the recurrent architectures, both in terms of convergence and in final accuracy on the task. For P-MNIST, TCNs outperform state-of-the-art results (95.9%) based on recurrent networks with Zoneout and Recurrent BatchNorm (Cooijmans et al., 2016; Krueger et al., 2017).

Copy memory. Convergence results on the copy memory task are shown in Figure 4. TCNs quickly converge to correct answers, while LSTMs and GRUs simply converge to the same loss as predicting all zeros. In this case we also compare to the recently-proposed EURNN (Jing et al., 2017), which was highlighted to perform well on this task. While both TCN and EURNN perform well for sequence length $T = 500$, the TCN has a clear advantage for $T = 1000$ and longer (in terms of both loss and rate of convergence).

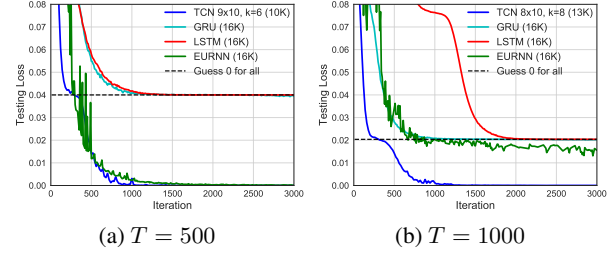


Figure 4. Result on the copy memory task for different sequence lengths T . TCNs outperform recurrent architectures.

5.3. Polyphonic Music and Language Modeling

We now discuss the results on polyphonic music modeling, character-level language modeling, and word-level language modeling. These domains are dominated by recurrent architectures, with many specialized designs developed for these tasks (Zhang et al., 2016; Ha et al., 2017; Krueger et al., 2017; Grave et al., 2017; Greff et al., 2017; Merity et al., 2017). We mention some of these specialized architectures when useful, but our primary goal is to compare the generic TCN model to similarly generic recurrent architectures, before domain-specific tuning. The results are summarized in Table 1.

Polyphonic music. On Nottingham and JSB Chorales, the TCN with virtually no tuning outperforms the recurrent models by a considerable margin, and even outperforms some enhanced recurrent architectures for this task such as HF-RNN (Boulanger-Lewandowski et al., 2012) and Diagonal RNN (Subakan & Smaragdis, 2017). Note however that other models such as the Deep Belief Net LSTM perform better still (Vohra et al., 2015); we believe this is likely due to the fact that the datasets are relatively small, and thus the right regularization method or generative modeling procedure can improve performance significantly. This is largely orthogonal to the RNN/TCN distinction, as a similar variant of TCN may well be possible.

Word-level language modeling. Language modeling remains one of the primary applications of recurrent networks and many recent works have focused on optimizing LSTMs for this task (Krueger et al., 2017; Merity et al., 2017). Our implementation follows standard practice that ties the weights of encoder and decoder layers for both TCN and RNNs (Press & Wolf, 2016), which significantly reduces the number of parameters in the model. For training, we use SGD and anneal the learning rate by a factor of 0.5 for both TCN and RNNs when validation accuracy plateaus.

On the smaller PTB corpus, an optimized LSTM architecture (with recurrent and embedding dropout, etc.) outperforms the TCN, while the TCN outperforms both GRU and vanilla RNN. However, on the much larger Wikitext-103 corpus and the LAMBADA dataset (Paperno et al., 2016), without any hyperparameter search, the TCN outperforms

the LSTM results of Grave et al. (2017), achieving much lower perplexities.

Character-level language modeling. On character-level language modeling (PTB and text8, accuracy measured in bits per character), the generic TCN outperforms regularized LSTMs and GRUs as well as methods such as Normalized LSTMs (Krueger & Memisevic, 2015). (Specialized architectures exist that outperform all of these, see the supplement.)

5.4. Memory Size of TCN and RNNs

One of the theoretical advantages of recurrent architectures is their unlimited memory: the theoretical ability to retain information through sequences of unlimited length. We now examine specifically how long the different architectures can retain information in practice. We focus on 1) the copy memory task, which is a stress test designed to evaluate long-term, distant information propagation in recurrent networks, and 2) the LAMBADA task, which tests both local and non-local textual understanding.

The copy memory task is perfectly set up to examine a model’s ability to retain information for different lengths of time. The requisite retention time can be controlled by varying the sequence length T . In contrast to Section 5.2, we now focus on the accuracy on the last 10 elements of the output sequence (which are the nontrivial elements that must be recalled). We used models of size 10K for both TCN and RNNs.

The results of this focused study are shown in Figure 5. TCNs consistently converge to 100% accuracy for all sequence lengths, whereas LSTMs and GRUs of the same size quickly degenerate to random guessing as the sequence length T grows. The accuracy of the LSTM falls below 20% for $T < 50$, while the GRU falls below 20% for $T < 200$. These results indicate that TCNs are able to maintain a much longer effective history than their recurrent counterparts.

This observation is backed up on real data by experiments on the large-scale LAMBADA dataset, which is specifically designed to test a model’s ability to utilize broad context (Paperno et al., 2016). As shown in Table 1, TCN outperforms LSTMs and vanilla RNNs by a significant margin in perplexity on LAMBADA, with a substantially smaller network and virtually no tuning. (State-of-the-art results on this dataset are even better, but only with the help of additional memory mechanisms (Grave et al., 2017).)

6. Conclusion

We have presented an empirical evaluation of generic convolutional and recurrent architectures across a comprehensive suite of sequence modeling tasks. To this end, we have described a simple temporal convolutional network (TCN)

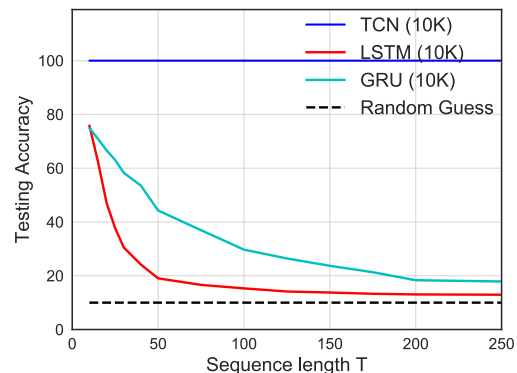


Figure 5. Accuracy on the copy memory task for sequences of different lengths T . While TCN exhibits 100% accuracy for all sequence lengths, the LSTM and GRU degenerate to random guessing as T grows.

that combines best practices such as dilations and residual connections with the causal convolutions needed for autoregressive prediction. The experimental results indicate that TCN models substantially outperform generic recurrent architectures such as LSTMs and GRUs. We further studied long-range information propagation in convolutional and recurrent networks, and showed that the “infinite memory” advantage of RNNs is largely absent in practice. TCNs exhibit longer memory than recurrent architectures with the same capacity.

Numerous advanced schemes for regularizing and optimizing LSTMs have been proposed (Press & Wolf, 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018). These schemes have significantly advanced the accuracy achieved by LSTM-based architectures on some datasets. The TCN has not yet benefited from this concerted community-wide investment into architectural and algorithmic elaborations. We see such investment as desirable and expect it to yield advances in TCN performance that are commensurate with the advances seen in recent years in LSTM performance. We will release the code for our project to encourage this exploration.

The preeminence enjoyed by recurrent networks in sequence modeling may be largely a vestige of history. Until recently, before the introduction of architectural elements such as dilated convolutions and residual connections, convolutional architectures were indeed weaker. Our results indicate that with these elements, a simple convolutional architecture is more effective across diverse sequence modeling tasks than recurrent architectures such as LSTMs. Due to the comparable clarity and simplicity of TCNs, we conclude that convolutional networks should be regarded as a natural starting point and a powerful toolkit for sequence modeling.

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Supplementary Material

A. Hyperparameters Settings

A.1. Hyperparameters for TCN

Table 2 lists the hyperparameters we used when applying the generic TCN model on various tasks and datasets. The most important factor for picking parameters is to make sure that the TCN has a sufficiently large receptive field by choosing k and d that can cover the amount of context needed for the task.

As discussed in Section 5, the number of hidden units was chosen so that the model size is approximately at the same level as the recurrent models with which we are comparing. In Table 2, a gradient clip of N/A means no gradient clipping was applied. In larger tasks (e.g., language modeling), we empirically found that gradient clipping (we randomly picked a threshold from $[0.3, 1]$) helps with regularizing TCN and accelerating convergence.

All weights were initialized from a Gaussian distribution $\mathcal{N}(0, 0.01)$. In general, we found TCN to be relatively insensitive to hyperparameter changes, as long as the effective history (i.e., receptive field) size is sufficient.

A.2. Hyperparameters for LSTM/GRU

Table 3 reports hyperparameter settings that were used for the LSTM. These values are picked from hyperparameter search for LSTMs that have up to 3 layers, and the optimizers are chosen from {SGD, Adam, RMSprop, Adagrad}. For certain larger datasets, we adopted the settings used in prior work (e.g., [Grave et al. \(2017\)](#) on Wikitext-103). GRU hyperparameters were chosen in a similar fashion, but typically with more hidden units than in LSTM to keep the total network size approximately the same (since a GRU cell is more compact).

B. State-of-the-Art Results

As previously noted, the generic TCN and LSTM/GRU models we used can be outperformed by more specialized architectures on some tasks. State-of-the-art results are summarized in Table 4. The same TCN architecture is used across all tasks. Note that the size of the state-of-the-art model may be different from the size of the TCN.

C. Effect of Filter Size and Residual Block

In this section we briefly study the effects of different components of a TCN layer. Overall, we believe dilation is required for modeling long-term dependencies, and so we mainly focus on two other factors here: the filter size k used by each layer, and the effect of residual blocks.

We perform a series of controlled experiments, with the results of the ablative analysis shown in Figure 6. As before, we kept the model size and depth exactly the same for different models, so that the dilation factor is strictly controlled. The experiments were conducted on three different tasks: copy memory, permuted MNIST (P-MNIST), and Penn Treebank word-level language modeling. These experiments confirm that both factors (filter size and residual connections) contribute to sequence modeling performance.

Filter size k . In both the copy memory and the P-MNIST tasks, we observed faster convergence and better accuracy for larger filter sizes. In particular, looking at Figure 6a, a TCN with filter size ≤ 3 only converges to the same level as random guessing. In contrast, on word-level language modeling, a smaller kernel with filter size of $k = 3$ works best. We believe this is because a smaller kernel (along with fixed dilation) tends to focus more on the local context, which is especially important for PTB language modeling (in fact, the very success of n -gram models suggests that only a relatively short memory is needed for modeling language).

Residual block. In all three scenarios that we compared here, we observed that the residual function stabilized training and brought faster convergence with better final results. Especially in language modeling, we found that residual connections contribute substantially to performance (See Figure 6f).

D. Gating Mechanisms

One component that had been used in prior work on convolutional architectures for language modeling is the gated activation ([van den Oord et al., 2016](#); [Dauphin et al., 2017](#)). We have chosen not to use gating in the generic TCN model. We now examine this choice more closely.

Table 2. TCN parameter settings for experiments in Section 5.

TCN SETTINGS							
Dataset/Task	Subtask	k	n	Hidden	Dropout	Grad Clip	Note
The Adding Problem	$T = 200$	6	7	27	0.0	N/A	
	$T = 400$	7	7	27			
	$T = 600$	8	8	24			
Seq. MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Permuted MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Copy Memory Task	$T = 500$	6	9	10	0.05	1.0	RMSprop 5e-4
	$T = 1000$	8	8	10			
	$T = 2000$	8	9	10			
Music JSB Chorales	-	3	2	150	0.5	0.4	
Music Nottingham	-	6	4	150	0.2	0.4	
Word-level LM	PTB	3	4	600	0.5	0.4	Embed. size 600
	Wiki-103	3	5	1000	0.4		Embed. size 400
	LAMBADA	4	5	500			Embed. size 500
Char-level LM	PTB	3	3	450	0.1	0.15	Embed. size 100
	text8	2	5	520			

Dauphin et al. (2017) compared the effects of gated linear units (GLU) and gated tanh units (GTU), and adopted GLU in their non-dilated gated ConvNet. Following the same choice, we now compare TCNs using ReLU and TCNs with gating (GLU), represented by an elementwise product between two convolutional layers, with one of them also passing through a sigmoid function $\sigma(x)$. Note that the gates architecture uses approximately twice as many convolutional layers as the ReLU-TCN.

The results are shown in Table 5, where we kept the number of model parameters at about the same size. The GLU does further improve TCN accuracy on certain language modeling datasets like PTB, which agrees with prior work. However, we do not observe comparable benefits on other tasks, such as polyphonic music modeling or synthetic stress tests that require longer information retention. On the copy memory task with $T = 1000$, we found that TCN with gating converged to a worse result than TCN with ReLU (though still better than recurrent models).

Table 3. LSTM parameter settings for experiments in Section 5.

LSTM SETTINGS (KEY PARAMETERS)							
Dataset/Task	Subtask	n	Hidden	Dropout	Grad Clip	Bias	Note
The Adding Problem	$T = 200$	2	77	0.0	50	5.0	SGD 1e-3
	$T = 400$	2	77		50	10.0	Adam 2e-3
	$T = 600$	1	130		5	1.0	-
Seq. MNIST	-	1	130	0.0	1	1.0	RMSprop 1e-3
Permuted MNIST	-	1	130	0.0	1	10.0	RMSprop 1e-3
Copy Memory Task	$T = 500$	1	50	0.05	0.25	-	RMSprop/Adam
	$T = 1000$	1	50		1		
	$T = 2000$	3	28		1		
Music JSB Chorales	-	2	200	0.2	1	10.0	SGD/Adam
Music Nottingham	-	3	280	0.1	0.5	-	Adam 4e-3
		1	500		1	-	
Word-level LM	PTB	3	700	0.4	0.3	1.0	SGD 30, Emb. 700, etc.
	Wiki-103	-	-	-	-	-	Grave et al. (2017)
	LAMBADA	-	-	-	-	-	Grave et al. (2017)
Char-level LM	PTB	2	600	0.1	0.5	-	Emb. size 120
	text8	1	1024	0.15	0.5	-	Adam 1e-2

Table 4. State-of-the-art (SoTA) results for tasks in Section 5.

TCN VS. SOTA RESULTS					
Task	TCN Result	Size	SoTA	Size	Model
Seq. MNIST (acc.)	99.0	21K	99.0	21K	Dilated GRU (Chang et al., 2017)
P-MNIST (acc.)	97.2	42K	95.9	42K	Zoneout (Krueger et al., 2017)
Adding Prob. 600 (loss)	5.8e-5	70K	5.3e-5	70K	Regularized GRU
Copy Memory 1000 (loss)	3.5e-5	70K	0.011	70K	EURNN (Jing et al., 2017)
JSB Chorales (loss)	8.10	300K	3.47	-	DBN+LSTM (Vohra et al., 2015)
Nottingham (loss)	3.07	1M	1.32	-	DBN+LSTM (Vohra et al., 2015)
Word PTB (ppl)	88.68	13M	47.7	22M	AWD-LSTM-MoS + Dynamic Eval. (Yang et al., 2018)
Word Wiki-103 (ppl)	45.19	148M	40.4	>300M	Neural Cache Model (Large) (Grave et al., 2017)
Word LAMBADA (ppl)	1279	56M	138	>100M	Neural Cache Model (Large) (Grave et al., 2017)
Char PTB (bpc)	1.31	3M	1.22	14M	2-LayerNorm HyperLSTM (Ha et al., 2017)
Char text8 (bpc)	1.45	4.6M	1.29	>12M	HM-LSTM (Chung et al., 2016)

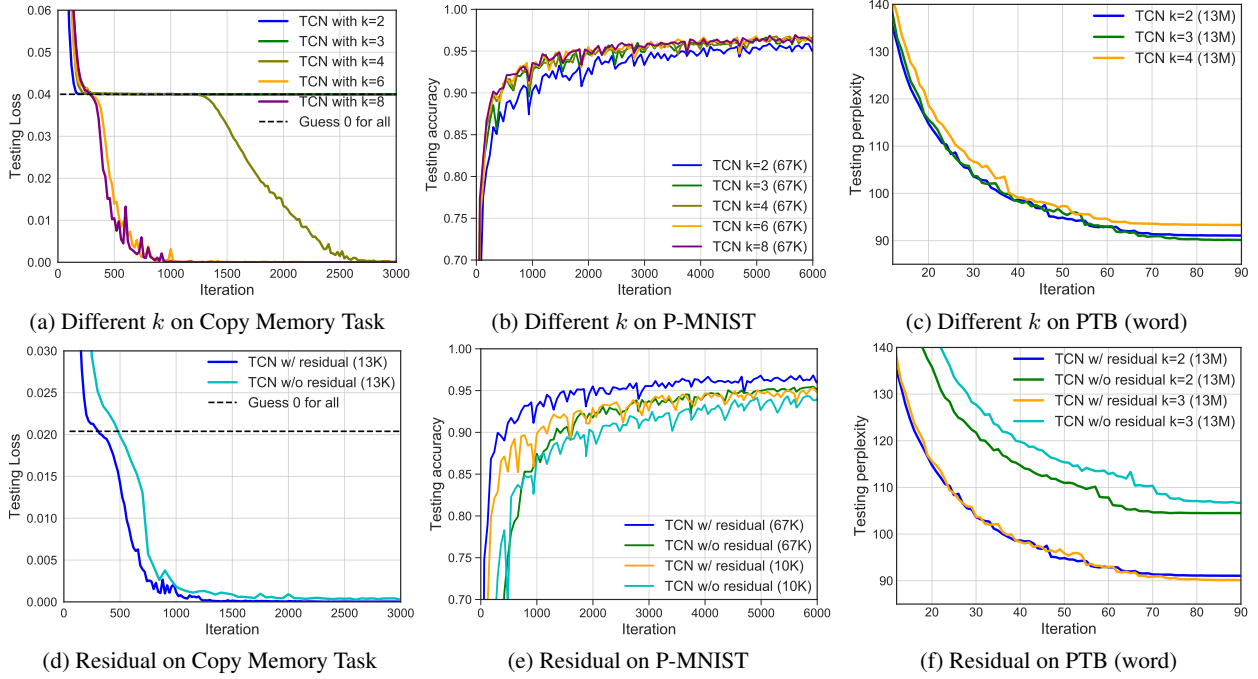


Figure 6. Controlled experiments that study the effect of different components of the TCN model.

Table 5. An evaluation of gating in TCN. A plain TCN is compared to a TCN that uses gated activations.

Task	TCN	TCN + Gating
Sequential MNIST (acc.)	99.0	99.0
Permuted MNIST (acc.)	97.2	96.9
Adding Problem $T = 600$ (loss)	5.8e-5	5.6e-5
Copy Memory $T = 1000$ (loss)	3.5e-5	0.00508
JSB Chorales (loss)	8.10	8.13
Nottingham (loss)	3.07	3.12
Word-level PTB (ppl)	88.68	87.94
Char-level PTB (bpc)	1.31	1.306
Char text8 (bpc)	1.45	1.485

Temporal Convolutional Networks and Forecasting



Share This Article



📅 Jul 6, 2021

🕒 16 min read

Written By

Francesco Lässig

How a convolutional network with some simple adaptations can become a powerful tool for sequence modeling and forecasting.

Although commonly associated with image classification tasks, convolutional neural networks (CNNs) have proven to be valuable tools for sequence modeling and forecasting, given the right modifications. In this article we explore in detail the basic building blocks that a Temporal Convolutional Network (TCN) consists of, and how they all fit together to create a powerful forecasting model. Using our open-source *Darts* TCN implementation, we show that accurate predictions can be achieved on a real-world dataset with only a few lines of code.

The following description of a Temporal Convolutional Network is based on the following <https://arxiv.org/pdf/1803.01271.pdf>. References to this paper will be denoted by (*).

We use cookies to ensure that we give you the best experience on our website. [Accept all](#) [Reject](#)

protected by reCAPTCHA
[Privacy](#) · [Terms](#)

Motivation

Up until recently, the topic of sequence modeling in the context of deep learning has been largely associated with recurrent neural network architectures such as LSTM and GRU. S. Bai et al. (*) suggest that this way of thinking is antiquated, and that convolutional networks should be taken into consideration as one of the primary candidates when modeling sequential data. They were able to show that convolutional networks can achieve better performance than RNNs in many tasks while avoiding common drawbacks of recurrent models, such as the exploding/vanishing gradient problem or lacking memory retention. Furthermore, using a convolutional network instead of a recurrent one can lead to performance improvements as it allows for parallel computation of outputs. The architecture they propose is called Temporal Convolutional Network (TCN) and will be explained in the following sections. To facilitate understanding the TCN architecture in conjunction with its *Darts* implementation, this article will use the same model parameter names as seen in the library wherever possible (indicated in **bold**).

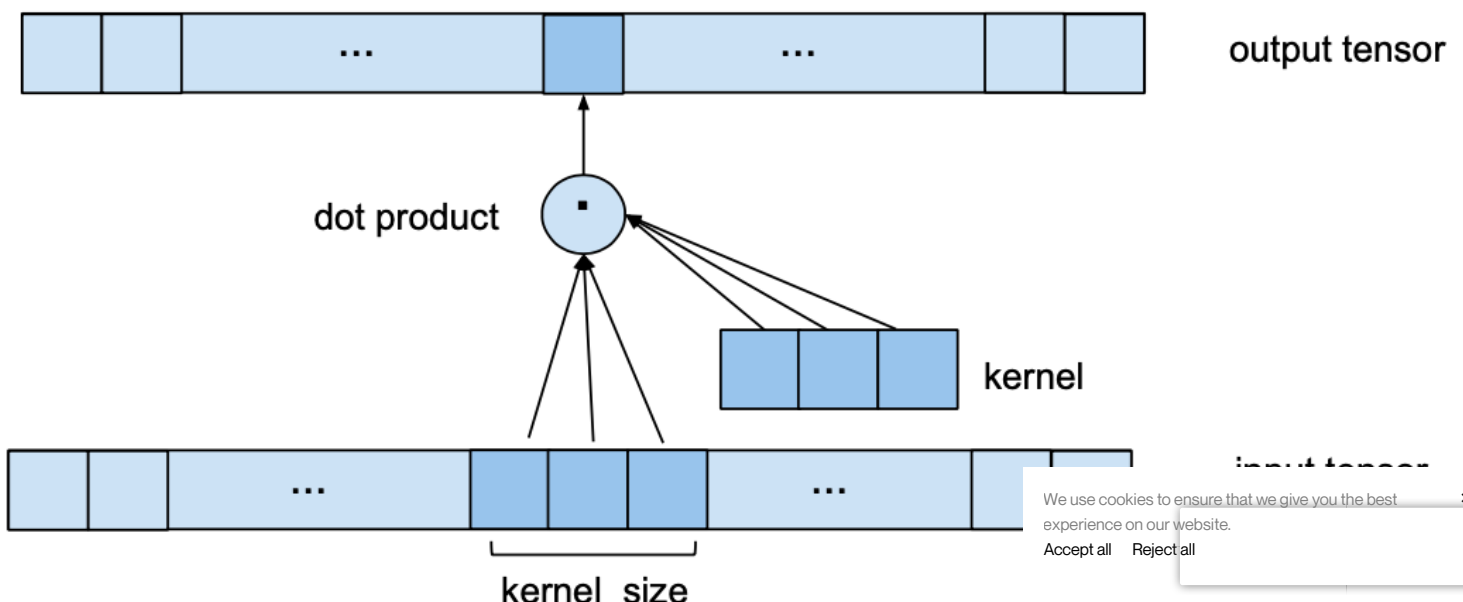
Overview

A TCN, short for Temporal Convolutional Network, consists of dilated, causal 1D convolutional layers with the same input and output lengths. The following sections go into detail about what these terms actually mean.

1D Convolutional Network

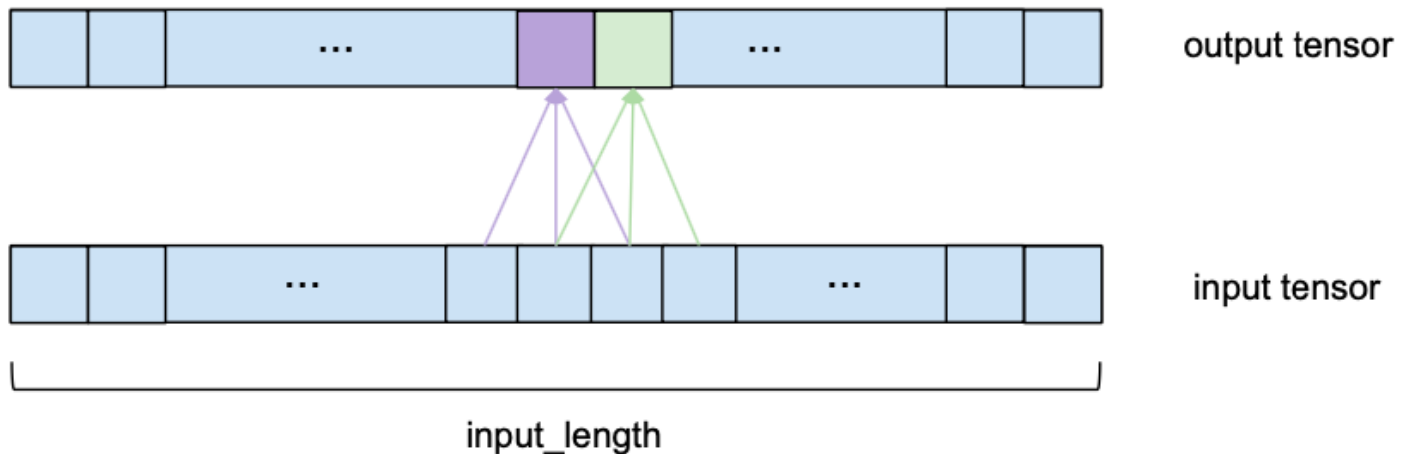
A 1D convolutional network takes as input a 3-dimensional tensor and also outputs a 3-dimensional tensor. The input tensor of our TCN implementation has the shape (batch_size, input_length, input_size) and the output tensor has the shape (batch_size, input_length, output_size). Since every layer in a TCN has the same input and output length, only the third dimension of the input and output tensors differs. In the univariate case, input_size and output_size will both be equal to one. In the more general multivariate case, input_size and output_size might differ since we might not want to forecast every component of the input sequence.

One single 1D convolutional layer receives an input tensor of shape (batch_size, input_length, nr_input_channels) and outputs a tensor of shape (batch_size, input_length, nr_output_channels). To see how one single layer converts its input into the output, let's look at one element of the batch (the same process takes place for every element in the batch). Let's start with the simplest case where nr_input_channels and nr_output_channels both equal 1. In this case we are looking at 1D input and output tensors. The following image shows how one element of the output tensor is computed.



input_length

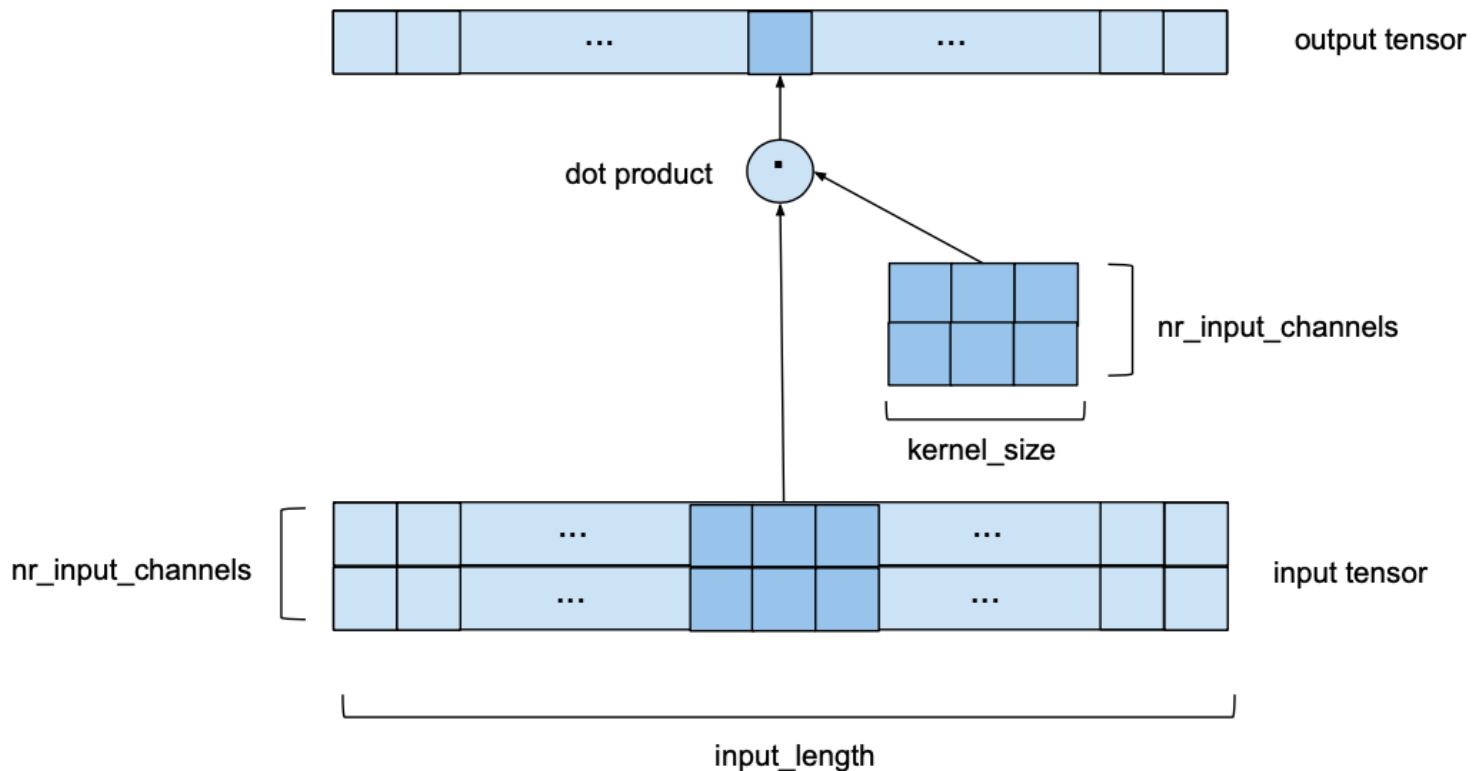
We can see that to compute one element of the output, we look at a series of consecutive elements of length `kernel_size` of the input. In the above example we chose a `kernel_size` of 3. To obtain the output, we take the dot product of the subsequence of the input and a kernel vector of learned weights of the same length. To get the next element of the output, the same procedure is applied, but the `kernel_size`-sized window of the input sequence is shifted to the right by one element (for the purposes of this forecasting model, the stride is always set to 1). Please note that the same set of kernel weights will be used to compute every output of one convolutional layer. The following image shows two consecutive output elements and their respective input subsequences.



To make the visualization simpler, the dot product with the kernel vector is not shown anymore, but takes place for every output element with the same kernel weights.

To make sure that the output sequence has the same length as the input sequence, some zero-padding is applied. This means that additional zero-valued entries are added to either the beginning or the end of the input tensor to ensure that the output has the desired length. How this is done exactly will be explained in later sections.

Now let's look at the case where we have multiple input channels, i.e. `nr_input_channels` is larger than 1. In this case, the process described above is repeated for every single input channel, but each time with a different kernel. This leads to `nr_input_channels` intermediate output vectors and a number of kernel weights of `kernel_size * nr_input_channels`. Then, all the intermediate output vectors are summed up to obtain the final output vector. In some sense this is equivalent to a 2D convolution with an input tensor of shape `(input_size, nr_input_channels)` and a kernel of shape `(kernel_size, nr_input_channels)` as shown in the following image. It's still 1D in the sense that the window only moves along a single axis, but we do have a 2D convolution at every step in the sense that we are using a 2-dimensional kernel matrix.



For this example we chose $nr_input_channels$ to be equal to 2. Now, instead of a kernel vector sliding over a 1-dimensional input sequence we have a $nr_input_channels$ by $kernel_size$ kernel matrix sliding along a $nr_input_channels$ wide series of length $input_length$.

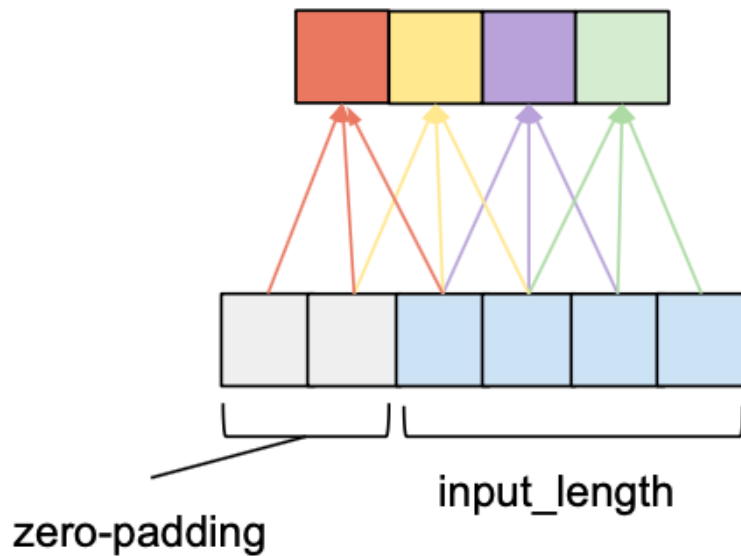
If both $nr_input_channels$ and $nr_output_channels$ are larger than 1, the above process is simply repeated for every output channel with a different kernel matrix. The output vectors are then stacked on top of each other resulting in an output tensor of shape $(input_length, nr_output_channels)$. The number of kernel weights in this case is equal to $kernel_size * nr_input_channels * nr_output_channels$.

The two variables $nr_input_channels$ and $nr_output_channels$ depend on the position of the layer within the network. The first layer will have $nr_input_channels = input_size$, and the last layer will have $nr_output_channels = output_size$. All other layers will use the intermediate channel number given by $num_filters$.

Causal Convolution

For a convolutional layer to be causal, for every i in $\{0, \dots, input_length - 1\}$ the i 'th element of the output sequence may only depend on the elements of the input sequence with indices $\{0, \dots, i\}$. In other words, an element in the output sequence can only depend on elements that come before it in the input sequence. As mentioned before, to ensure an output tensor has the same length as the input tensor, we need to apply zero padding. If we only apply zero-padding on the left side of the input tensor, then causal convolution will be ensured. To understand this, consider the rightmost output element. Given that there is no padding on the right side of the input sequence, the last element it depends on is the last element of the input. Now consider the second to last output element of the output sequence. Its kernel window is shifted to the left by one compared to the last output element, that means its rightmost dependency in the input sequence is the second to last element of the input sequence. It follows by induction that for every element in the output sequence, its latest dependency in the input sequence has the same index as itself. The following figure shows an exam

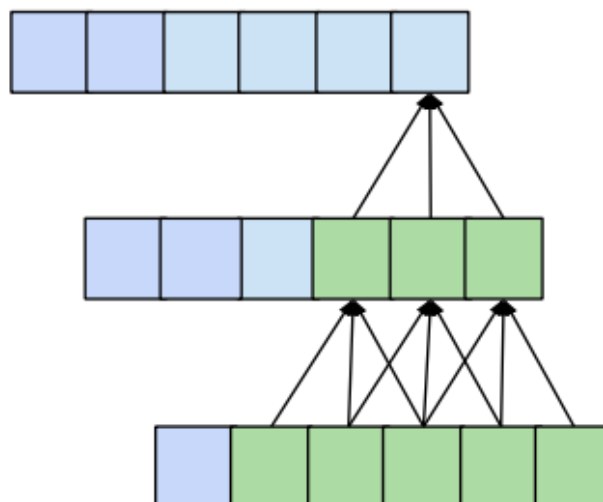
input_length of 4 and a kernel_size of 3.



We can see that with a left zero-padding of 2 entries we can achieve the same output length while obeying the causality rule. In fact, without dilation, the number of zero-padding entries required for maintaining the input length is always equal to $\text{kernel_size} - 1$.

Dilation

One desirable quality of a forecasting model is that the value of a specific entry in the output depends on all previous entries in the input, i.e. all entries that have an index smaller or equal to itself. This is achieved when the receptive field, meaning the set of entries of the original input that affect a specific entry of the output, has size input_length . We also call this 'full history coverage'. As we have seen before, one conventional convolutional layer makes an entry in the output dependent on the kernel_size entries of the input that have an index smaller or equal to itself. For instance, if we have a kernel_size of 3, the 5th element in the output will be dependent on elements 3, 4 and 5 of the input. This reach is expanded when we stack multiple layers on top of each other. In the following figure we can see that by stacking two layers with kernel_size 3 we get a receptive field size of 5.



More generally, a 1D convolutional network with n layers and a kernel_size k has a receptive field r of size

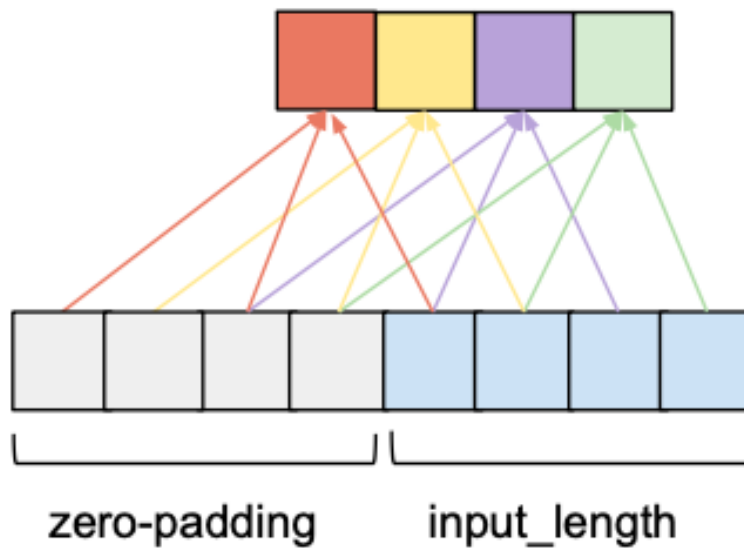
$$r = 1 + n * (k - 1)$$

To know how many layers are needed for full coverage, we can set the receptive field size to input_length l and solve for the number of layers n (we need to round up in case of non-integer values):

$$n = \lceil (l - 1) / (k - 1) \rceil$$

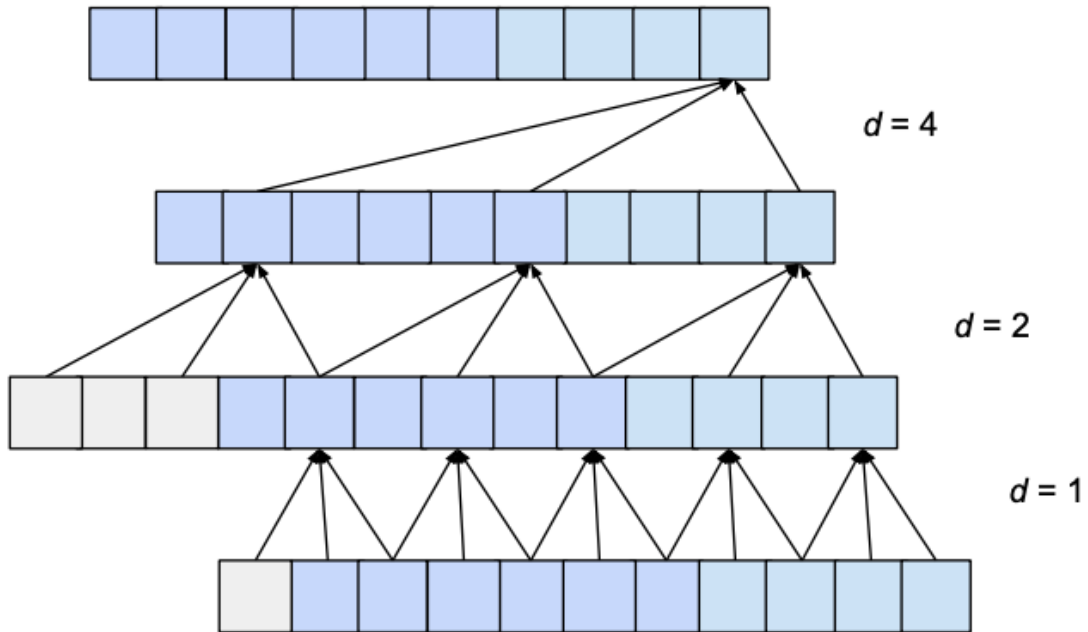
This means that, given a fixed kernel_size, the number of layers required for full history coverage is linear in the length of the input tensor, which will result in networks that become very deep very fast, leading to models with a very large number of parameters that take longer to train. Furthermore, a high number of layers has been shown to lead to degradation problems related to the gradient of the loss function. One way to increase the receptive field size while still keeping the number of layers relatively small is to introduce dilation to the convolutional network.

Dilation in the context of a convolutional layer refers to the distance between the elements of the input sequence that are used to compute one entry of the output sequence. So a conventional convolutional layer could be seen as a 1-dilated layer, since the input elements for 1 output value are adjacent. The following figure shows an example of a 2-dilated layer with an input_length of 4 and a kernel_size of 3.



Compared to the 1-dilated case, this layer has a receptive field that spreads across a length of 5 rather than 3. More generally, a d -dilated layer with a kernel size k has a receptive field spreading across a length of $1+d*(k-1)$. If d is fixed, this will still require a number linear in the length of the input tensor to achieve full receptive field coverage (we just decreased the constant).

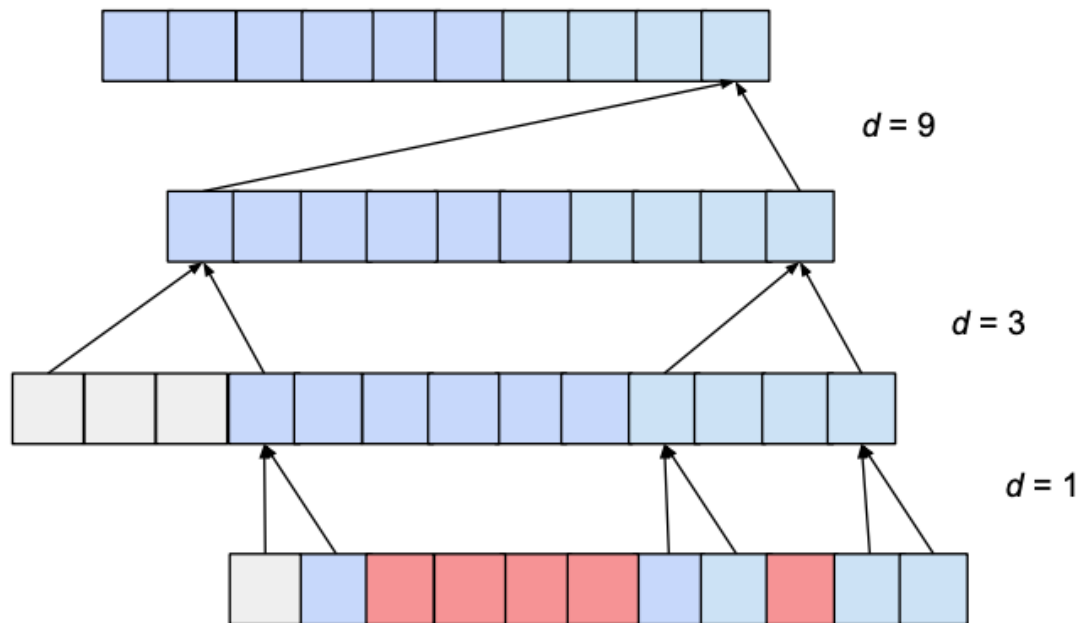
This problem can be addressed by increasing the value of d exponentially as we move up through the layers. For this, we choose a constant dilation_base integer b which will let us compute the dilation d of a specific layer as a function of the number of layers below it, i , as $d = b^{**i}$. The following figure shows a network with an input_length of 10, a kernel_size of 3 and a dilation_base of 2 which results in 3 dilated convolutional layers for full coverage.



Here we only show the influence of inputs that affect the last value of the output. Likewise, only zero-padding entries necessary for the last output value are shown. Clearly, the last output value is dependent on the whole input coverage. Actually, given the hyperparameters, an input_length of up to 15 could be used while maintaining full receptive field coverage. Generally speaking, every additional layer adds a value of $d*(k-1)$ to the current receptive field width, where d is computed as $d=b^{**i}$, with i representing the number of layers below our new layer. Consequently, the width of the receptive field w of a TCN with exponential dilation of base b , kernel size k and number of layers n is given by

$$w = 1 + \sum_{i=0}^{n-1} (k-1) \cdot b^i = 1 + (k-1) \cdot \frac{b^n - 1}{b - 1}$$

However, depending on the values b and k , this receptive field can have 'holes'. Consider the following network with a dilation_base of 3 and a kernel size of 2:



The receptive field does cover a range that is larger than the input size (namely 15). However, the receptive field has holes in it; that is there are entries in the input sequence that the output value is not dependent on (shown above in red). To solve this problem, we need to either increase the kernel size to 3, or decrease the dilation base to 2. Generally speaking, for a receptive field with no holes, the kernel size k has to be at least as big as the dilation base b .

Considering these observations, we can compute how many layers our network needs for full history coverage. Given a kernel size k , dilation base b , where $k \geq b$, and input length l , the following inequality must hold for full history coverage:

$$1 + (k - 1) \cdot \frac{b^n - 1}{b - 1} \geq l$$

We can solve for n and get the minimum number of required layers as

$$n = \left\lceil \log_b \left(\frac{(l - 1) \cdot (b - 1)}{(k - 1)} + 1 \right) \right\rceil$$

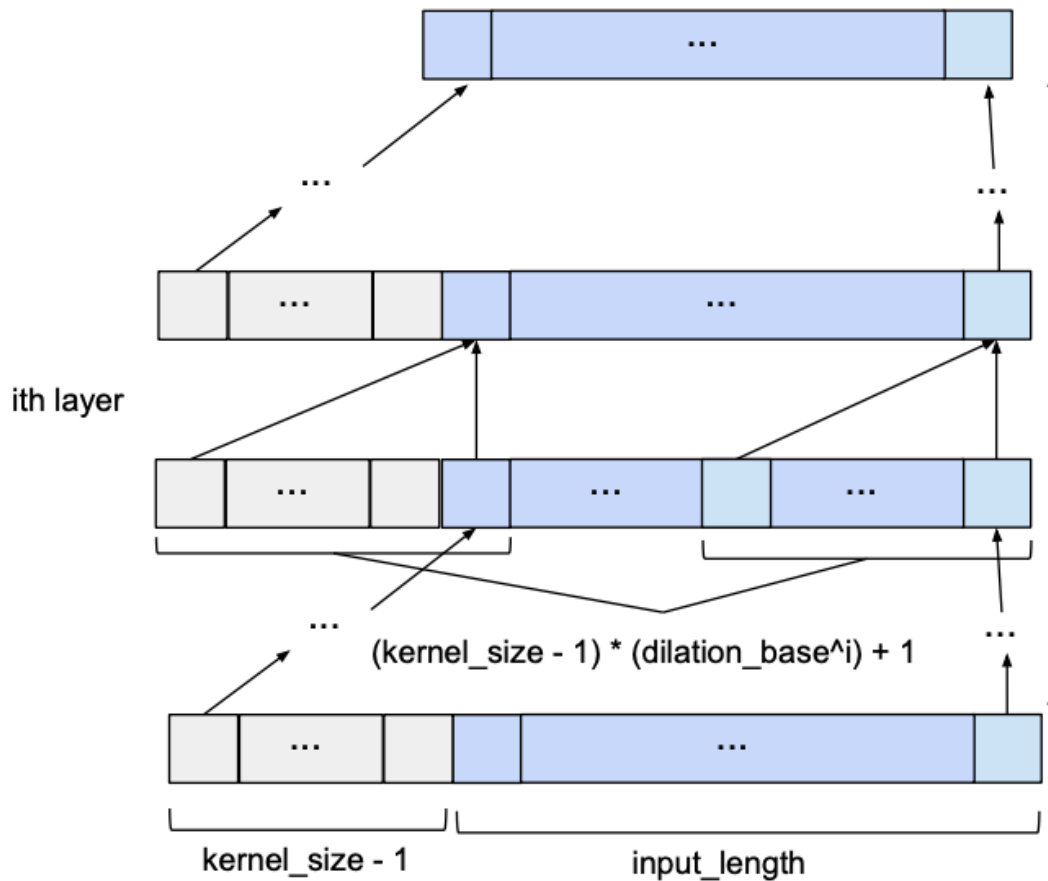
We can see that the number of layers is now logarithmic rather than linear in the length of the input. This constitutes a significant improvement, which can be achieved without sacrificing receptive field coverage.

Now the only thing left to specify is the number of zero-padding entries required at each layer. Given a dilation base b , a kernel size k and a number of i layers below our current layer, then the number of zero-padding entries p needed for the current layer are computed as follows:

$$p = b^i \cdot (k - 1)$$

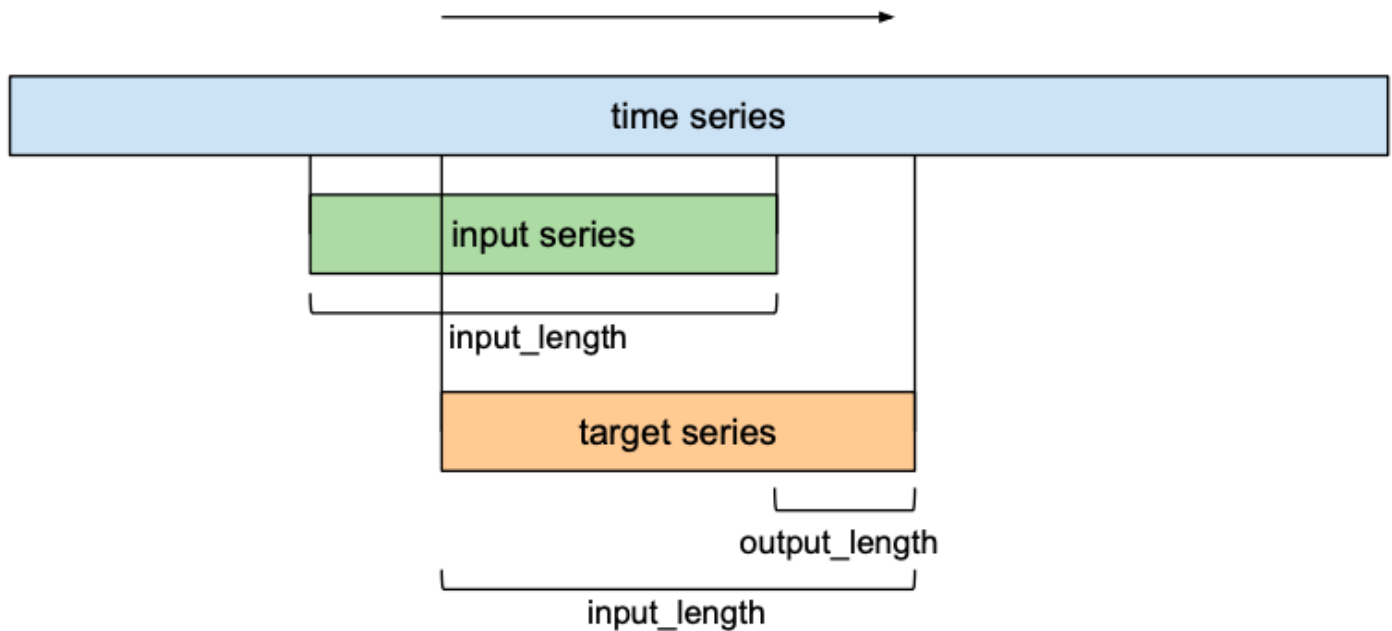
Basic TCN Overview

Given `input_length`, `kernel_size`, `dilation_base` and the minimum number of layers required for full history coverage, the basic TCN network would look something like this:



Forecasting

So far we have only talked about the 'input sequence' and the 'output sequence' without getting into how they relate to each other. In the context of forecasting, we want to predict the next entries of a time series into the future. To train our TCN network to do forecasting, the training set will consist of (input sequence, target sequence)-pairs of equally-sized subsequences of the given time series. A target series will be a series that is shifted forward in relation to its respective input series by a certain number `output_length`. This means that a target series of length `input_length` contains the last `(input_length - output_length)` elements of its respective input sequence as first elements, and the `output_length` elements that come after the last entry of the input series as its final elements. In the context of forecasting, this means that the maximum forecasting horizon that can be predicted with such a model is equal to `output_length`. Using a sliding window approach, many overlapping pairs of input and target sequences can be created out of one time series.



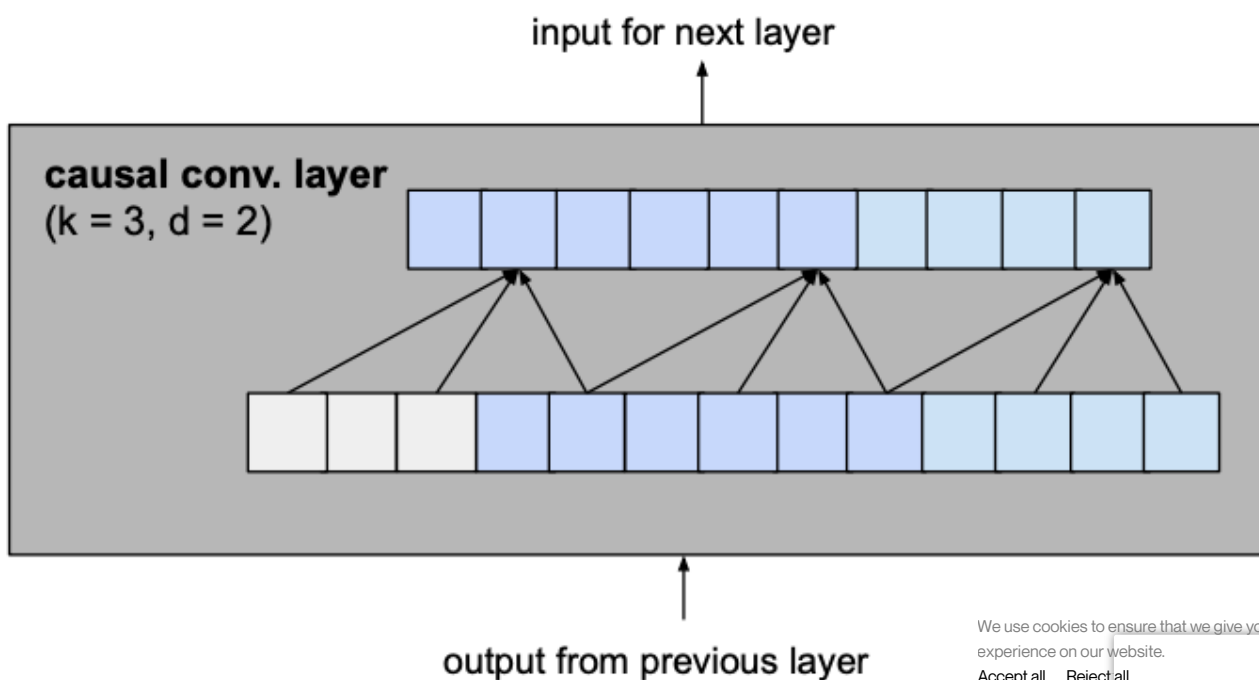
Improvements to the Model

S. Bai et al. (*) suggest a few additions to the basic TCN architecture for improved performance which will be discussed in this section, namely residual connections, regularization and activation functions.

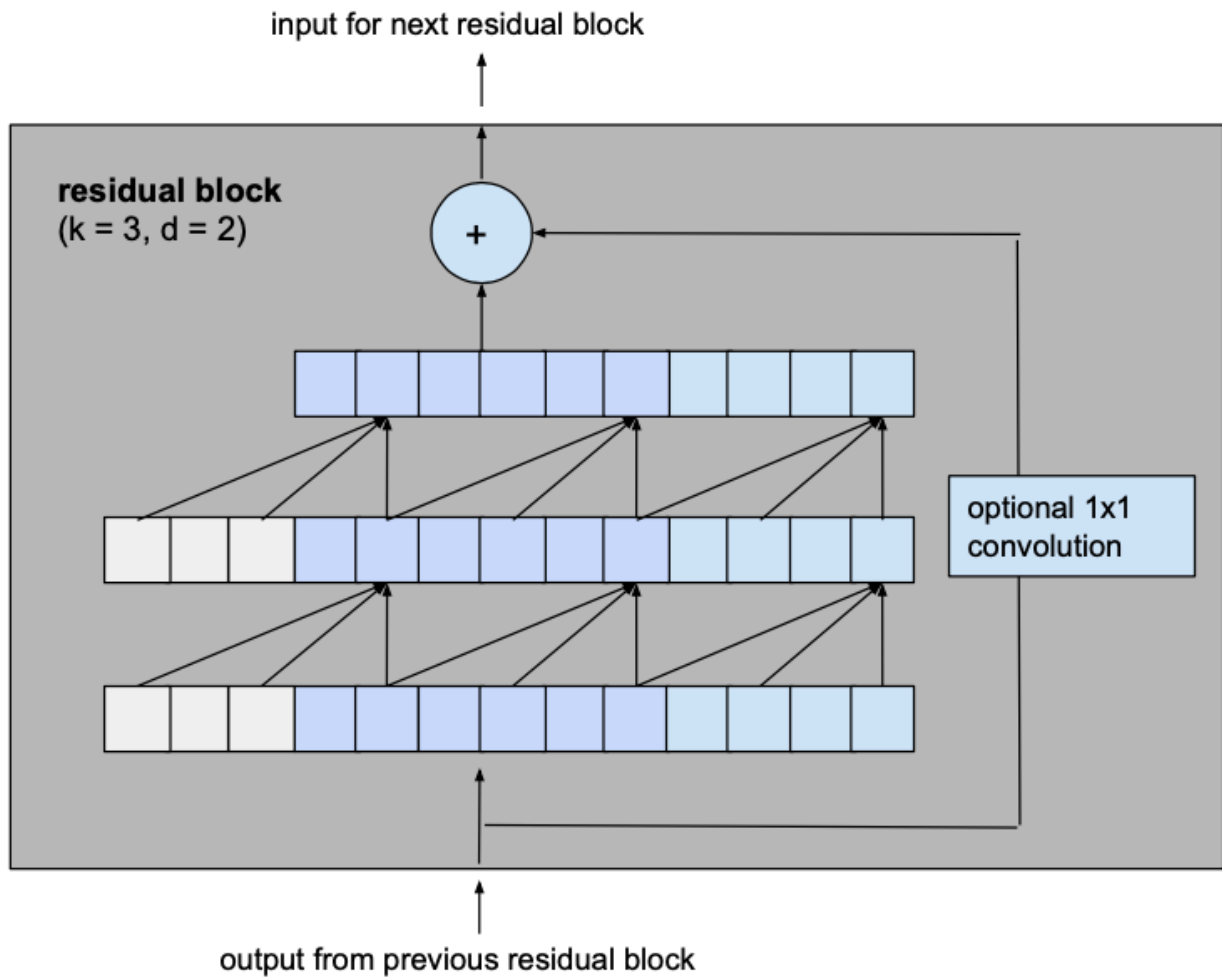
Residual Blocks

The biggest modification we make to the previously introduced basic model is to change the fundamental building block of the model from a simple 1D causal convolutional layer to a residual block which consists of 2 layers with the same dilation factor and a residual connection.

Let's consider a layer with a dilation factor d of 2 and kernel size k of 3 from the basic model to see how this translates into a residual block of the improved model.



becomes



The output of the two convolutional layers will be added to the input of the residual block to produce the input for the next block. For all inner blocks of the network, i.e. all but the first and the last one, the input and output channel width is the same, namely `num_filters`. Since the first convolutional layer of the first residual block and the second convolutional layer of the last residual block may have different input and output channel widths, the width of the residual tensor might have to be adjusted, which is done using a 1×1 convolution.

This change affects the calculus for the minimum number of required layers for full coverage. Now we have to think about how many residual blocks are necessary to achieve a full receptive field coverage. Adding a residual block to a TCN adds twice as much receptive field width than when adding a basic causal layer, since it includes 2 such layers. So the total size of the receptive field r of a TCN with dilation base b , kernel size k with $k \geq b$ and number of residual blocks n can be computed as

$$r = 1 + \sum_{i=0}^{n-1} 2 \cdot (k-1) \cdot b^i = 1 + 2 \cdot (k-1) \cdot \frac{b^n - 1}{b - 1}$$

which leads to a minimum number of residual blocks n for full history coverage of input

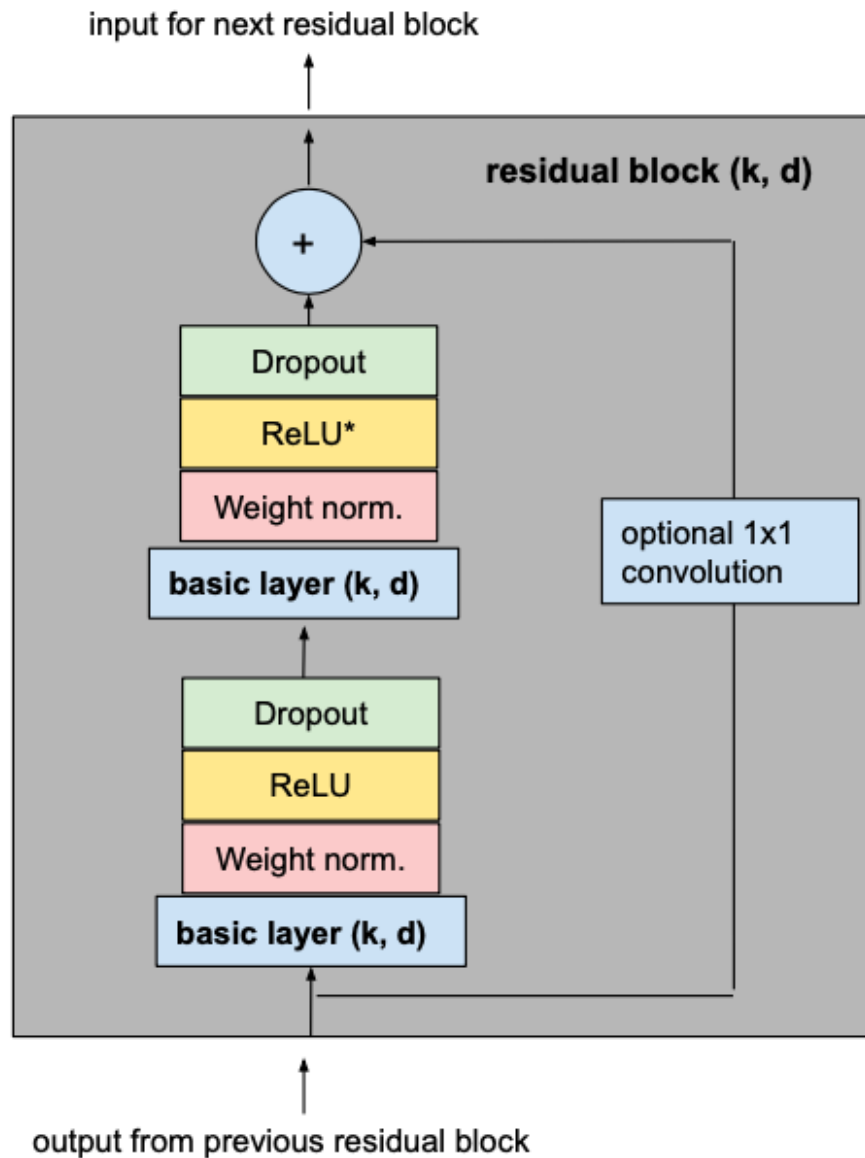
$$n = \left\lceil \log_b \left(\frac{(l-1) \cdot (b-1)}{(k-1) \cdot 2} + 1 \right) \right\rceil$$

Activation, Normalization, Regularization

To make our TCN more than just an overly complex linear regression model, activation functions need to be added on top of the convolutional layers to introduce non-linearities. ReLU activations are added to the residual blocks after both convolutional layers.

To normalize the input of hidden layers (which counteracts the exploding gradient problem among other things), weight normalization is applied to every convolutional layer.

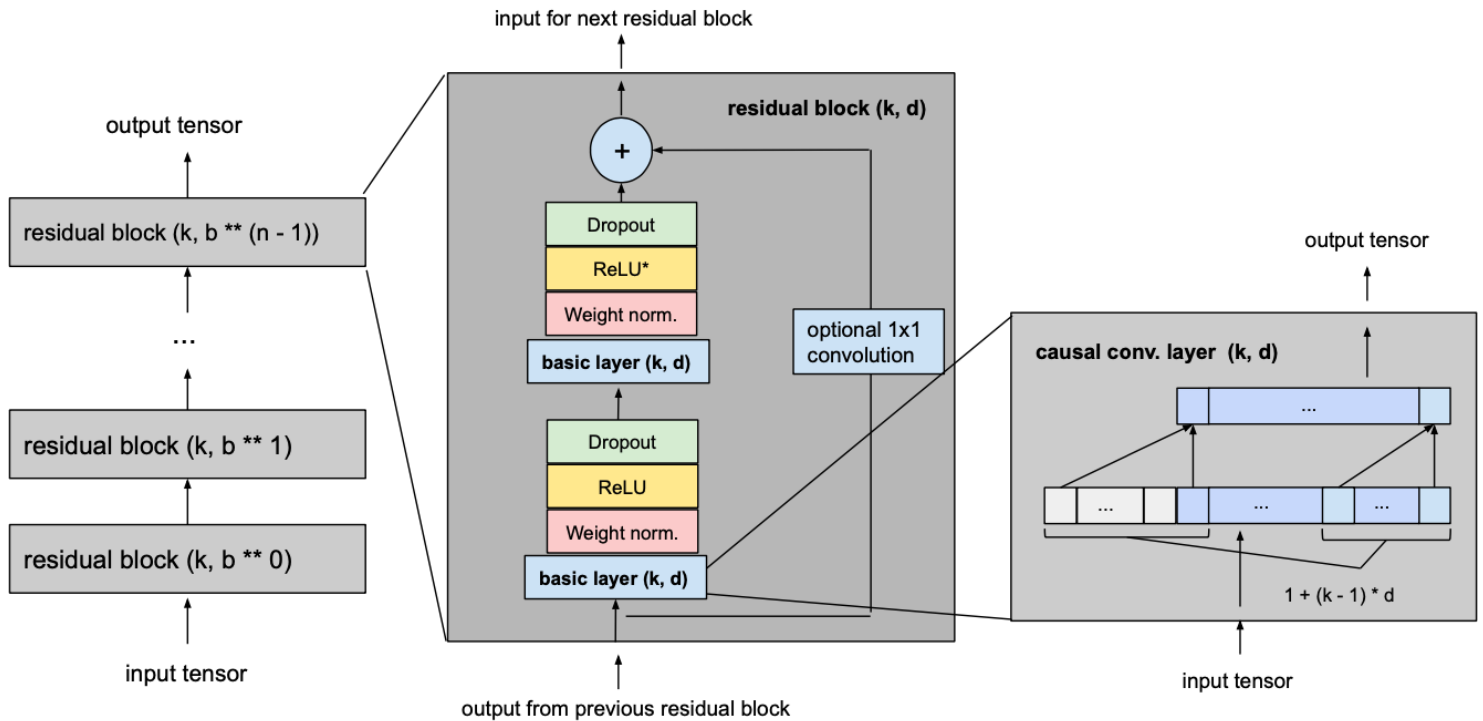
In order to prevent overfitting, regularization is introduced via dropout after every convolutional layer in every residual block. The following figure shows the final residual block.



The asterisk in the second ReLU unit indicates that it is present in every layer but the last. We want our final output to be able to take on negative values as well (this differs from the standard ReLU outlined in the paper).

Final Model

The following picture shows our final TCN model with l equal to input_length, k equal to kernel_size, b equal to dilation_base, $k \geq b$ and with a minimum number of residual blocks for full history coverage n , where n can be computed from the other values as explained above.



Example

Let's take a look at an example of how we can use the TCN architecture to forecast a time series using the *Darts* library.

First, we need a time series to train and evaluate our model on. For this, we use a [Kaggle dataset](#) containing hourly energy production data from Spain. More specifically, we choose to predict the 'run-of-river hydroelectricity' production. Furthermore, to make the problem less computation-intensive, we average the energy production over each day to get a daily time series.

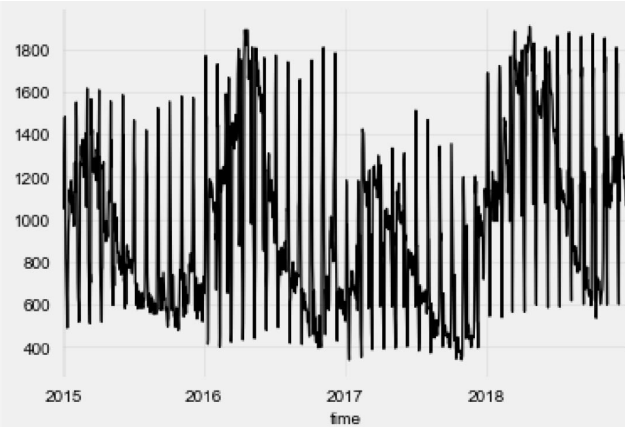
```
from darts import TimeSeries
from darts.dataprocessing.transformers import
MissingValuesFiller
import pandas as pd

df = pd.read_csv('energy_dataset.csv', delimiter=",")
df['time'] = pd.to_datetime(df['time'], utc=True)
df['time'] = df['time'].dt.tz_localize(None)

df_day_avg =
df.groupby(df['time'].astype(str).str.split(" ").str[0]).mean().reset_index()

value_filler = MissingValuesFiller()
series =
value_filler.transform(TimeSeries.from_dataframe(df_day_avg, 'time', ['generation hydro run-of-river']))

series.plot()
```



We can see that, in addition to a yearly seasonality, there are regular 'spikes' in energy production that come in monthly intervals. Since the TCN model supports multiple input channels, we can add additional time series components to the current time series that encode the current day of the month. This can help our TCN model to converge faster.

```
series = series.add_datetime_attribute('day', one_hot=True)
```

Now we split our data into train and validation components and perform standardization.

```
from darts.dataprocessing.transformers import Scaler

train, val =
series.split_after(pd.Timestamp('20170901'))

scaler = Scaler()
train_transformed = scaler.fit_transform(train)
val_transformed = scaler.transform(val)
series_transformed = scaler.transform(series)
```

Now it's time to create and train our TCN model. Note that all bold variable names that have appeared in the above description of the architecture can be used as arguments for the constructor of the *Darts* TCN implementation. The `output_length` parameter is set to 7 since we want to perform weekly forecasts. When training the model, we specify as `target_series` only the first component of our training series, since we do not want to predict the helper time series we added earlier. We tried out a few different hyperparameter combinations, but most of the values were chosen quite arbitrarily.

```
from darts.models import TCNModel

model = TCNModel(
    input_size=train.width,
    n_epochs=20,
    input_length=365,
    output_length=7,
    dropout=0,
    dilation_base=2,
    weight_norm=True,
    kernel_size=7,
    num_filters=4,
    random_state=0
)

model.fit(
    training_series=train_transformed,
    target_series=train_transformed['0'],
    val_training_series=val_transformed,
    val_target_series=val_transformed['0'],
    verbose=True
)
```

To evaluate our model, we want to test its performance using a 7-day forecasting horizon

different points in time in the validation set. For this, we use the historic backtesting functionality from *Darts*. Note that the model is provided new input data for every prediction, but it is never retrained. To save time, we set the stride to 5.

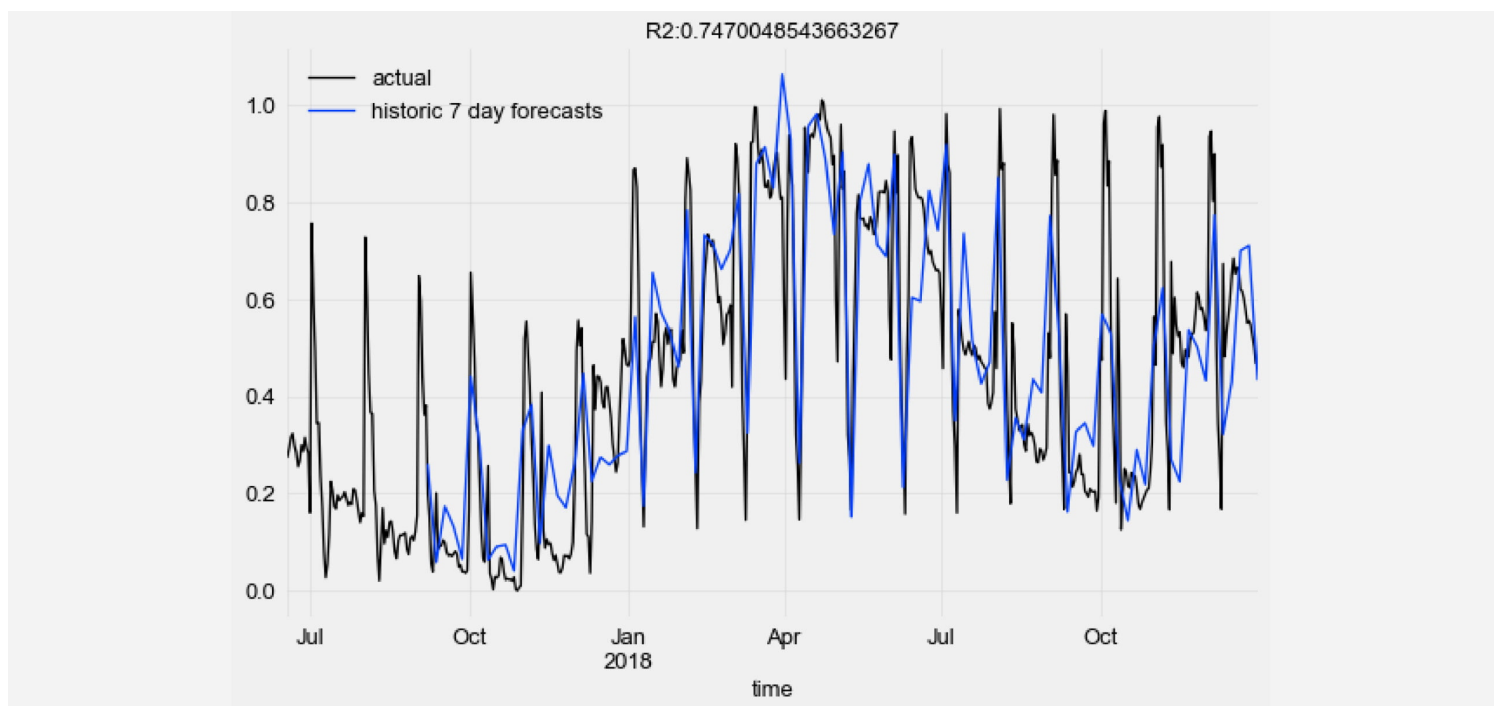
```
pred_series = model.backtest(
    series_transformed,
    target_series=series_transformed['0'],
    start=pd.Timestamp('20170901'),
    forecast_horizon=7,
    stride=5,
    retrain=False,
    verbose=True,
    use_full_output_length=True
)
```

Let's visualize the historic forecast predictions of our TCN model against the ground truth data points and compute the R2 score.

```
from darts.metrics import r2_score
import matplotlib.pyplot as plt

series_transformed[900:]['0'].plot(label='actual')
pred_series.plot(label='historic 7 day forecasts')
r2_score_value = r2_score(series_transformed['0'], pred_series)

plt.title('R2: ' + str(r2_score_value))
plt.legend()
```



For more details and additional examples, please check out our [TCN examples notebook](#) on GitHub.

Conclusion

Deep learning in sequence modeling is, for the most part, still widely associated with recurrent network architectures. [But research has shown](#) that these types of models can be outperformed by a TCN, both in terms of predictive performance and efficiency. In this article we explored how this promising model can be understood in terms of simple building blocks.

We use cookies to ensure that we give you the best experience on our website.
Accept all Reject all

Temporal Convolutional Networks: A Unified Approach to Action Segmentation

Colin Lea René Vidal Austin Reiter Gregory D. Hager
Johns Hopkins University
{clea1@, rvidal@cis., areiter@cs., hager@cs.}jhu.edu

Abstract

The dominant paradigm for video-based action segmentation is composed of two steps: first, for each frame, compute low-level features using Dense Trajectories or a Convolutional Neural Network that encode spatiotemporal information locally, and second, input these features into a classifier that captures high-level temporal relationships, such as a Recurrent Neural Network (RNN). While often effective, this decoupling requires specifying two separate models, each with their own complexities, and prevents capturing more nuanced long-range spatiotemporal relationships. We propose a unified approach, as demonstrated by our Temporal Convolutional Network (TCN), that hierarchically captures relationships at low-, intermediate-, and high-level time-scales. Our model achieves superior or competitive performance using video or sensor data on three public action segmentation datasets and can be trained in a fraction of the time it takes to train an RNN.

1. Introduction

Action segmentation is crucial for numerous applications ranging from collaborative robotics to modeling activities of daily living. Given a video, the goal is to simultaneously segment every action in time and classify each constituent segment. While recent work has shown strong improvements on this task, models tend to decouple low-level feature representations from high-level temporal models. Within video analysis, these low-level features may be computed by pooling handcrafted features (e.g. Improved Dense Trajectories (IDT) [21]) or concatenating learned features (e.g. Spatiotemporal Convolutional Neural Networks (ST-CNN) [8, 12]) over a short period of time. High-level temporal classifiers capture a local history of these low-level features. In a Conditional Random Field (CRF), the action prediction at one time step is often a function of the prediction at the previous time step, and in a Recurrent Neural Network (RNN), the predictions are a function

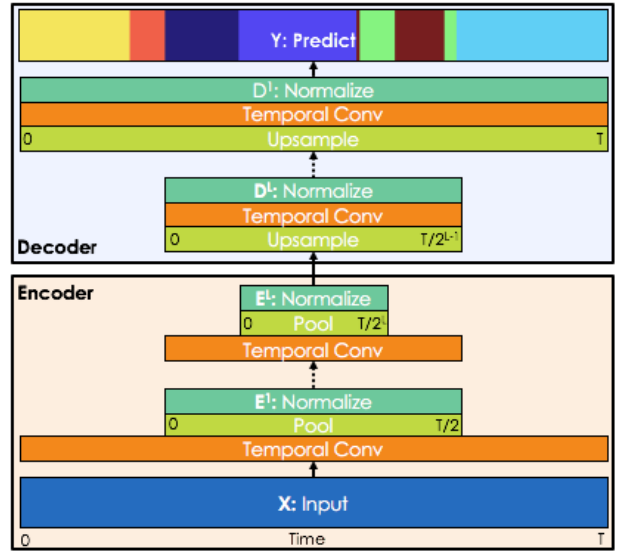


Figure 1. Our temporal encoder-decoder network hierarchically models actions from video or other time-series data.

of a set of latent states at each time step, where the latent states are connected across time. This two-step paradigm has been around for decades (e.g., [6]) and typically goes unquestioned. However, we posit that valuable information is lost between steps.

In this work, we introduce a unified approach to action segmentation that uses a single set of computational mechanisms – 1D convolutions, pooling, and channel-wise normalization – to hierarchically capture low-, intermediate-, and high-level temporal information. For each layer, 1D convolutions capture how features at lower levels change over time, pooling enables efficient computation of long-range temporal patterns, and normalization improves robustness towards various environmental conditions. In contrast with RNN-based models, which compute a set of latent activations that are updated sequentially per-frame, we compute a set of latent activations that are updated hierarchically per-layer. As a byproduct, our model takes much

less time to train. Our model can be viewed as a generalization of the recent ST-CNN [8] and is more similar to recent models for semantic segmentation than it is to models for video-analysis. We show this approach is broadly applicable to video and other types of robot sensors.

Prior Work: Due to space limitations, here we will only briefly describe models for time-series and semantic segmentation. See [8] for related work on action segmentation or [20] for a broader overview on action recognition.

RNNs and CRFs are popular high-level temporal classifiers. RNN variations, including Long Short Term Memory (LSTM) and Gated Recurrent Units (GRU), model hidden temporal states via internal gating mechanisms. However, they are hard to introspect and difficult to correctly train [13]. It has been shown that in practice LSTM only keeps a memory of about 4 seconds on some video-based action segmentation datasets [15]. CRFs typically model pairwise transitions between the labels or latent states (e.g., [8]), which are easy to interpret, but over-simplify the temporal dynamics of complex actions. Both of these models suffer from the same fundamental issue: intermediate activations are typically a function of the low-level features at the current time step and the state at the previous time step. Our temporal convolutional filters are a function of raw data across a much longer period of time.

Until recently, the dominant paradigm for semantic segmentation was similar to that of action segmentation. Approaches typically combined low-level texture features (e.g., TextonBoost) with high-level spatial models (e.g., grid-based CRFs) that model the relationships between different regions of an image [7]. This is similar to action segmentation where low-level spatiotemporal features are used in tandem with high-level temporal models. Recently, with the introduction of Fully Convolutional Networks (FCNs), the dominant semantic segmentation paradigm has started to change. Long *et al.* [11] introduced the first FCN, which leverages typical classification CNNs like AlexNet, to compute per-pixel object labels. This is done by intelligently upsampling the intermediate activations in each region of an image. Our model is more similar to the recent encoder-decoder network by Badrinarayanan *et al.* [1]. Their encoder step uses the first half of a VGG-like network to capture patterns in different regions of an image and their decoder step takes the activations from the encoder, which are of a reduced image resolution, and uses convolutional filters to upsample back to the original image size. In subsequent sections we describe our temporal variation in detail.

2. Temporal Convolutional Networks (TCN)

The input to our Temporal Convolutional Network can be a sensor signal (e.g. accelerometers) or latent encoding of a spatial CNN applied to each frame. Let $X_t \in \mathbb{R}^{F_0}$ be the input feature vector of length F_0 for time step t for $0 <$

$t \leq T$. Note that the time T may vary for each sequence, and we denote the number of time steps in each layer as T_l . The true action label for each frame is given by $y_t \in \{1, \dots, C\}$, where C is the number of classes.

Our encoder-decoder framework, as depicted in Figure 1, is composed of temporal convolutions, 1D pooling/upsampling, and channel-wise normalization layers.

For each of the L convolutional layers in the encoder, we apply a set of 1D filters that capture how the input signals evolve over the course of an action. The filters for each layer are parameterized by tensor $W^{(l)} \in \mathbb{R}^{F_l \times d \times F_{l-1}}$ and biases $b^{(l)} \in \mathbb{R}^{F_l}$, where $l \in \{1, \dots, L\}$ is the layer index and d is the filter duration. For the l -th layer of the encoder, the i -th component of the (unnormalized) activation $\hat{E}_t^{(l)} \in \mathbb{R}^{F_l}$ is a function of the incoming (normalized) activation matrix $E^{(l-1)} \in \mathbb{R}^{F_{l-1} \times T_{l-1}}$ from the previous layer

$$\hat{E}_{i,t}^{(l)} = f(b_i^{(l)} + \sum_{t'=1}^d \langle W_{i,t',\cdot}^{(l)}, E_{\cdot, t+d-t'}^{(l-1)} \rangle) \quad (1)$$

for each time t where $f(\cdot)$ is a Leaky Rectified Linear Unit. The normalization process is described below.

Max pooling is applied with width 2 across time (in 1D) such that $T_l = \frac{1}{2}T_{l-1}$.¹ Pooling enables us to efficiently compute activations over a long period of time.

We apply channel-wise normalization after each pooling step in the encoder. This has been effective in recent CNN methods including Trajectory-Pooled Deep-Convolutional Descriptors (TDD) [10]. We normalize the pooled activation vector $\hat{E}_t^{(l)}$ by the highest response at that time step, $m = \max_i \hat{E}_{i,t}^{(l)}$, with some small $\epsilon = 1\text{E-}5$ such that

$$E_t^{(l)} = \frac{1}{m + \epsilon} \hat{E}_t^{(l)}. \quad (2)$$

Our decoder is similar to the encoder, except that upsampling is used instead of pooling, and the order of the operations is now upsample, convolve, then normalize. Upsampling is performed by simply repeating each entry twice.

The probability that frame t corresponds to one of the C action classes is predicted by vector $\hat{Y}_t \in [0, 1]^C$ using weight matrix $U \in \mathbb{R}^{C \times F_0}$ and bias $c \in \mathbb{R}^C$

$$\hat{Y}_t = \text{softmax}(UD_t^{(1)} + c). \quad (3)$$

We explored many other mechanisms, such as adding skip connections between layers, using different patterns of convolutional layers, and other normalization schemes. These helped at times and hurt in others. The aforementioned solution was superior in aggregate.

¹In theory, this implies T must be divisible by 2^L . In practice, we pad each sequence to be of an appropriate length, given the pooling operations, such that the input length of the whole sequence, T , and the length of the output predictions are the same.

Implementation details: Each of the $L = 3$ layers has $F_l = \{32, 64, 96\}$ filters. Filter duration, d , is set as the mean segment duration for the shortest class from the training set. For example, $d = 10$ seconds for 50 Salads. Parameters of our model were learned using the cross entropy loss with Stochastic Gradient Descent and ADAM step updates. All models were implemented using Keras and TensorFlow.

For each frame in our video experiments, the input, X_t , is the first fully connected layer computed in a spatial CNN trained solely on each dataset. We trained the model of [8], except instead of using Motion History Images (MHI) as input to the CNN, we concatenate the following for image I_t at frame t : $[I_t, I_{t-d} - I_t, I_{t+d} - I_t, I_{t-2d} - I_t, I_{t+2d} - I_t]$ for $d = 0.5$ seconds. In our experiments, these difference images – which are a simple type of attention mechanism – tend to perform better than MHI or optical flow across these datasets. Furthermore, for each time step, we perform channel-wise normalization before feeding it into the TCN. This helps with large environmental fluctuations, such as changes in lighting.

3. Evaluation

We evaluate on three public datasets that contain action segmentation labels, video, and in two cases sensor data.

University of Dundee 50 Salads [18] contains 50 sequences of users making a salad. Each video is 5-10 minutes in duration and contains around 30 action instances such as *cutting a tomato* or *peeling a cucumber*. This dataset includes video and synchronized accelerometers attached to ten objects in the scene, such as the *bowl*, *knife*, and *plate*. We performed cross validation with 5 splits on the “eval” action granularity which includes 10 action classes. Our sensor results used the features from [9] which are the absolute values of accelerometer values. Previous results (e.g., [9, 14]) were evaluated using different setups. For example, [9] smoothed out short interstitial background segments. We reran all results to be consistent with [14]. We also included an LSTM baseline for comparison which uses 64 hidden states.

JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS) [5] was introduced to improve quantitative evaluation of robotic surgery training tasks. We used Leave One User Out cross validation on the suturing activity, which consists of 39 sequences performed by 8 users about 5 times each. The dataset includes video and synchronized robot kinematics (position, velocity, and gripper angle) for each robot end effector as well as corresponding action labels with 10 action classes. Sequences are a few minutes long and typically contain around 20 action instances.

Georgia Tech Egocentric Activities (GTEA) [4] contains 28 videos of 7 kitchen activities including making a sandwich and making coffee. For each of the four subjects, there is one instance of each activity. The camera is

mounted on the head of the user and is pointing at the area in front them. On average there are about 30 actions per video and videos are around a minute long. We used the 11 action classes defined in [3] and evaluated using leave one user out. We show results for user 2 to be consistent with [3] and [16].

Metrics: We evaluated using accuracy, which is simply the percent of correctly labeled frames, and segmental edit distance [9], which measures the correctness of the predicted temporal ordering of actions. This edit score is computed by applying the Levenshtein distance to the segmented predictions (e.g. $AAABBA \rightarrow ABA$). This is normalized to be in the range 0 to 100 such that higher is better.

4. Experiments and Discussion

Table 1 includes results for all datasets and corresponding sensing modalities. We include results from the spatial CNN which is input into the TCN, the Spatiotemporal CNN of Lea *et al.* [8] applied to the spatial features, and our TCN.

One of the most interesting findings is that some layers of convolutional filters appear to learn temporal shifts. There are certain actions in each dataset which are not easy to distinguish given the sensor data. By visualizing the activations for each layer, we found our model surmounts this issue by learning temporal offsets from activations in the previous layer. In addition, we find that despite the fact that we do not use a traditional temporal model, such as an RNN or CRF, our predictions do not suffer as heavily from issues like over-segmentation. This is highlighted by the large increase in edit score on most experiments.

Richard *et al.* [14] evaluated their model on the mid-level action granularity of 50 Salads which has 17 action classes. Their model achieved 54.2% accuracy, 44.8% edit, 0.379 mAP IoU overlap with a threshold of 0.1, and 0.229 mAP with a threshold of 0.5.² Our model achieves 59.7% accuracy, 47.3% edit, 0.579 mAP at 0.1, and 0.378 mAP at 0.5.

On GTEA, Singh *et al.* [16] reported 64.4% accuracy by performing cross validation on users 1 through 3. We achieve 62.5% using this setup. We found performance of our model has high variance between different trials on GTEA – even with the same hyper parameters – thus, the difference in accuracy is not likely to be statistically significant. Our approach could be used in tandem with features from Singh *et al.* to achieve superior performance.

Our model can be trained much faster than an RNN-LSTM. Using an Nvidia Titan X, it takes on the order of a minute to train a TCN for each split, whereas it takes on the order of an hour to train an RNN-LSTM. The speedup comes from the fact that we compute one set of convolutions for each layer, whereas RNN-LSTM effectively computes one set of convolutions for each time step.

²We computed our metrics using the predictions given by the authors.

50 Salads (“eval” setup)		
Sensor-based	Edit	Acc
[9] LC-SC-CRF	50.2	77.8
LSTM	54.5	73.3
TCN	65.6	82.0
Video-based	Edit	Acc
[8] VGG	7.6	38.3
[8] IDT	16.8	54.3
[8] Seg-ST-CNN	62.0	72.0
Spatial CNN	28.4	68.6
ST-CNN	55.5	74.2
TCN	61.1	74.4

GTEA		
Video-based	Edit	Acc
[3] Hand-crafted	-	47.7
[16] EgoNet	-	57.6
[16] TDD	-	59.5
[16] EgoNet+TDD	-	68.5
Spatial CNN	36.6	56.1
ST-CNN	53.4	64.5
TCN	58.8	66.1

JIGSAWS		
Sensor-based	Edit	Acc
[2] LSTM	75.3	80.5
[9] LC-SC-CRF	76.8	83.4
[2] Bidir LSTM	81.1	83.3
[17] SD-SDL	83.3	78.6
TCN	85.8	79.6
Vision-based	Edit	Acc
[19] MsM-CRF	-	71.7
[8] IDT	8.5	53.9
[8] VGG	24.3	45.9
[8] Seg-ST-CNN	66.6	74.7
Spatial CNN	37.7	74.0
ST-CNN	68.0	77.7
TCN	83.1	81.4

Table 1. Results on 50 Salads, Georgia Tech Egocentric Activities, and JHU-ISI Gesture and Skill Assessment Working Set. Notes: (1) Results using VGG and Improved Dense Trajectories (IDT) were intentionally computed without a temporal component for ablative analysis, hence their low edit scores. (2) We re-computed [9] using the author’s public code to be consistent with the setup of [14].

Conclusion: We introduced a model for action segmentation that learns a hierarchy of intermediate feature representations, which contrasts with the traditional low- versus high-level paradigm. This model achieves competitive or superior performance on several datasets and can be trained much more quickly than other models. A future version of this manuscript will include more comparisons and insights on the TCN.

References

- [1] V. Badrinarayanan, A. Handa, and R. Cipolla. Segnet: A deep convolutional encoder-decoder architecture for robust semantic pixel-wise labelling. *arXiv preprint arXiv:1505.07293*, 2015. 2
- [2] R. DiPietro, C. Lea, A. Malpani, N. Ahmidi, S. S. Vedula, G. I. Lee, M. R. Lee, and G. D. Hager. Recognizing surgical activities with recurrent neural networks. In *MICCAI*, 2016. 4
- [3] A. Fathi, A. Farhadi, and J. M. Rehg. Understanding egocentric activities. In *ICCV*, 2011. 3, 4
- [4] A. Fathi, R. Xiaofeng, and J. M. Rehg. Learning to recognize objects in egocentric activities. In *CVPR*, 2011. 3
- [5] Y. Gao, S. S. Vedula, C. E. Reiley, N. Ahmidi, B. Varadara-jan, H. C. Lin, L. Tao, L. Zappella, B. Béjar, D. D. Yuh, et al. JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS): A surgical activity dataset for human motion modeling. In *MICCAI Workshop: M2CAI*, 2014. 3
- [6] F. G. Hofmann, P. Heyer, and G. Hommel. Velocity profile based recognition of dynamic gestures with discrete hidden markov models. In *International Workshop on Gesture and Sign Language in Human-Computer Interaction*, 1998. 1
- [7] P. Krähenbühl and V. Koltun. Efficient inference in fully connected CRFs with gaussian edge potentials. In *NIPS*, 2011. 2
- [8] C. Lea, A. Reiter, R. Vidal, and G. D. Hager. Segmental spatio-temporal CNNs for fine-grained action segmentation. *ECCV*, 2016. 1, 2, 3, 4
- [9] C. Lea, R. Vidal, and G. D. Hager. Learning convolutional action primitives for fine-grained action recognition. In *ICRA*, 2016. 3, 4
- [10] Y. Q. Limin Wang and X. Tang. Action recognition with trajectory-pooled deep-convolutional descriptors. In *CVPR*, 2015. 2
- [11] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. *CVPR*, 2015. 2
- [12] J. Y. Ng, M. J. Hausknecht, S. Vijayanarasimhan, O. Vinyals, R. Monga, and G. Toderici. Beyond short snippets: Deep networks for video classification. In *CVPR*, 2015. 1
- [13] R. Pascanu, T. Mikolov, and Y. Bengio. On the difficulty of training recurrent neural networks. In *ICML*, 2013. 2
- [14] A. Richard and J. Gall. Temporal action detection using a statistical language model. In *CVPR*, 2016. 3, 4
- [15] B. Singh, T. K. Marks, M. Jones, O. Tuzel, and M. Shao. A multi-stream bi-directional recurrent neural network for fine-grained action detection. In *CVPR*, 2016. 2
- [16] S. Singh, C. Arora, and C. V. Jawahar. First person action recognition using deep learned descriptors. In *CVPR*, June 2016. 3, 4
- [17] S. Stefati, N. Cowan, and R. Vidal. Learning shared, discriminative dictionaries for surgical gesture segmentation and classification. In *MICCAI Workshop: M2CAI*, 2015. 4
- [18] S. Stein and S. J. McKenna. Combining embedded accelerometers with computer vision for recognizing food preparation activities. In *UbiComp*, 2013. 3
- [19] L. Tao, L. Zappella, G. D. Hager, and R. Vidal. Surgical gesture segmentation and recognition. In *MICCAI*, 2013. 4
- [20] M. Vrigkas, C. Nikou, and I. Kakadiaris. A review of human activity recognition methods. *Frontiers in Robotics and AI*, 2015. 2
- [21] H. Wang and C. Schmid. Action recognition with improved trajectories. In *ICCV*, 2013. 1

Convolutional Sequence to Sequence Learning

Jonas Gehring
 Michael Auli
 David Grangier
 Denis Yarats
 Yann N. Dauphin
 Facebook AI Research

Abstract

The prevalent approach to sequence to sequence learning maps an input sequence to a variable length output sequence via recurrent neural networks. We introduce an architecture based entirely on convolutional neural networks.¹ Compared to recurrent models, computations over all elements can be fully parallelized during training to better exploit the GPU hardware and optimization is easier since the number of non-linearities is fixed and independent of the input length. Our use of gated linear units eases gradient propagation and we equip each decoder layer with a separate attention module. We outperform the accuracy of the deep LSTM setup of Wu et al. (2016) on both WMT’14 English-German and WMT’14 English-French translation at an order of magnitude faster speed, both on GPU and CPU.

1. Introduction

Sequence to sequence learning has been successful in many tasks such as machine translation, speech recognition (Sutskever et al., 2014; Chorowski et al., 2015) and text summarization (Rush et al., 2015; Nallapati et al., 2016; Shen et al., 2016) amongst others. The dominant approach to date encodes the input sequence with a series of bi-directional recurrent neural networks (RNN) and generates a variable length output with another set of decoder RNNs, both of which interface via a soft-attention mechanism (Bahdanau et al., 2014; Luong et al., 2015). In machine translation, this architecture has been demonstrated to outperform traditional phrase-based models by large margins (Sennrich et al., 2016b; Zhou et al., 2016; Wu et al., 2016; §2).

¹The source code and models are available at <https://github.com/facebookresearch/fairseq>.

Convolutional neural networks are less common for sequence modeling, despite several advantages (Waibel et al., 1989; LeCun & Bengio, 1995). Compared to recurrent layers, convolutions create representations for fixed size contexts, however, the effective context size of the network can easily be made larger by stacking several layers on top of each other. This allows to precisely control the maximum length of dependencies to be modeled. Convolutional networks do not depend on the computations of the previous time step and therefore allow parallelization over every element in a sequence. This contrasts with RNNs which maintain a hidden state of the entire past that prevents parallel computation within a sequence.

Multi-layer convolutional neural networks create hierarchical representations over the input sequence in which nearby input elements interact at lower layers while distant elements interact at higher layers. Hierarchical structure provides a shorter path to capture long-range dependencies compared to the chain structure modeled by recurrent networks, e.g. we can obtain a feature representation capturing relationships within a window of n words by applying only $\mathcal{O}(\frac{n}{k})$ convolutional operations for kernels of width k , compared to a linear number $\mathcal{O}(n)$ for recurrent neural networks. Inputs to a convolutional network are fed through a constant number of kernels and non-linearities, whereas recurrent networks apply up to n operations and non-linearities to the first word and only a single set of operations to the last word. Fixing the number of non-linearities applied to the inputs also eases learning.

Recent work has applied convolutional neural networks to sequence modeling such as Bradbury et al. (2016) who introduce recurrent pooling between a succession of convolutional layers or Kalchbrenner et al. (2016) who tackle neural translation without attention. However, none of these approaches has been demonstrated improvements over state of the art results on large benchmark datasets. Gated convolutions have been previously explored for machine translation by Meng et al. (2015) but their evaluation was restricted to a small dataset and the model was used in tandem with a traditional count-based model. Architec-

tures which are partially convolutional have shown strong performance on larger tasks but their decoder is still recurrent (Gehring et al., 2016).

In this paper we propose an architecture for sequence to sequence modeling that is entirely convolutional. Our model is equipped with gated linear units (Dauphin et al., 2016) and residual connections (He et al., 2015a). We also use attention in every decoder layer and demonstrate that each attention layer only adds a negligible amount of overhead. The combination of these choices enables us to tackle large scale problems (§3).

We evaluate our approach on several large datasets for machine translation as well as summarization and compare to the current best architectures reported in the literature. On WMT’16 English-Romanian translation we achieve a new state of the art, outperforming the previous best result by 1.9 BLEU. On WMT’14 English-German we outperform the strong LSTM setup of Wu et al. (2016) by 0.5 BLEU and on WMT’14 English-French we outperform the likelihood trained system of Wu et al. (2016) by 1.6 BLEU. Furthermore, our model can translate unseen sentences at an order of magnitude faster speed than Wu et al. (2016) on GPU and CPU hardware (§4, §5).

2. Recurrent Sequence to Sequence Learning

Sequence to sequence modeling has been synonymous with recurrent neural network based encoder-decoder architectures (Sutskever et al., 2014; Bahdanau et al., 2014). The encoder RNN processes an input sequence $\mathbf{x} = (x_1, \dots, x_m)$ of m elements and returns state representations $\mathbf{z} = (z_1, \dots, z_m)$. The decoder RNN takes $\mathbf{z}$ and generates the output sequence $\mathbf{y} = (y_1, \dots, y_n)$ left to right, one element at a time. To generate output y_{i+1} , the decoder computes a new hidden state h_{i+1} based on the previous state h_i , an embedding g_i of the previous target language word y_i , as well as a conditional input c_i derived from the encoder output $\mathbf{z}$. Based on this generic formulation, various encoder-decoder architectures have been proposed, which differ mainly in the conditional input and the type of RNN.

Models without attention consider only the final encoder state z_m by setting $c_i = z_m$ for all i (Cho et al., 2014), or simply initialize the first decoder state with z_m (Sutskever et al., 2014), in which case c_i is not used. Architectures with attention (Bahdanau et al., 2014; Luong et al., 2015) compute c_i as a weighted sum of $(z_1, \dots, z_m)$ at each time step. The weights of the sum are referred to as attention scores and allow the network to focus on different parts of the input sequence as it generates the output sequences. Attention scores are computed by essentially comparing each encoder state z_j to a combination of the previous decoder

state h_i and the last prediction y_i ; the result is normalized to be a distribution over input elements.

Popular choices for recurrent networks in encoder-decoder models are long short term memory networks (LSTM; Hochreiter & Schmidhuber, 1997) and gated recurrent units (GRU; Cho et al., 2014). Both extend Elman RNNs (Elman, 1990) with a gating mechanism that allows the memorization of information from previous time steps in order to model long-term dependencies. Most recent approaches also rely on bi-directional encoders to build representations of both past and future contexts (Bahdanau et al., 2014; Zhou et al., 2016; Wu et al., 2016). Models with many layers often rely on shortcut or residual connections (He et al., 2015a; Zhou et al., 2016; Wu et al., 2016).

3. A Convolutional Architecture

Next we introduce a fully convolutional architecture for sequence to sequence modeling. Instead of relying on RNNs to compute intermediate encoder states $\mathbf{z}$ and decoder states $\mathbf{h}$ we use convolutional neural networks (CNN).

3.1. Position Embeddings

First, we embed input elements $\mathbf{x} = (x_1, \dots, x_m)$ in distributional space as $\mathbf{w} = (w_1, \dots, w_m)$, where $w_j \in \mathbb{R}^f$ is a column in an embedding matrix $\mathcal{D} \in \mathbb{R}^{V \times f}$. We also equip our model with a sense of order by embedding the absolute position of input elements $\mathbf{p} = (p_1, \dots, p_m)$ where $p_j \in \mathbb{R}^f$. Both are combined to obtain input element representations $\mathbf{e} = (w_1 + p_1, \dots, w_m + p_m)$. We proceed similarly for output elements that were already generated by the decoder network to yield output element representations that are being fed back into the decoder network $\mathbf{g} = (g_1, \dots, g_n)$. Position embeddings are useful in our architecture since they give our model a sense of which portion of the sequence in the input or output it is currently dealing with (§5.4).

3.2. Convolutional Block Structure

Both encoder and decoder networks share a simple block structure that computes intermediate states based on a fixed number of input elements. We denote the output of the l -th block as $\mathbf{h}^l = (h_1^l, \dots, h_n^l)$ for the decoder network, and $\mathbf{z}^l = (z_1^l, \dots, z_m^l)$ for the encoder network; we refer to blocks and layers interchangeably. Each block contains a one dimensional convolution followed by a non-linearity. For a decoder network with a single block and kernel width k , each resulting state h_i^1 contains information over k input elements. Stacking several blocks on top of each other increases the number of input elements represented in a state. For instance, stacking 6 blocks with $k = 5$ results in an input field of 25 elements, i.e. each output depends on 25

inputs. Non-linearities allow the networks to exploit the full input field, or to focus on fewer elements if needed.

Each convolution kernel is parameterized as $W \in \mathbb{R}^{2d \times kd}$, $b_w \in \mathbb{R}^{2d}$ and takes as input $X \in \mathbb{R}^{k \times d}$ which is a concatenation of k input elements embedded in d dimensions and maps them to a single output element $Y \in \mathbb{R}^{2d}$ that has twice the dimensionality of the input elements; subsequent layers operate over the k output elements of the previous layer. We choose gated linear units (GLU; Dauphin et al., 2016) as non-linearity which implement a simple gating mechanism over the output of the convolution $Y = [A \ B] \in \mathbb{R}^{2d}$:

$$v([A \ B]) = A \otimes \sigma(B)$$

where $A, B \in \mathbb{R}^d$ are the inputs to the non-linearity, $\otimes$ is the point-wise multiplication and the output $v([A \ B]) \in \mathbb{R}^d$ is half the size of Y . The gates $\sigma(B)$ control which inputs A of the current context are relevant. A similar non-linearity has been introduced in Oord et al. (2016b) who apply $\tanh$ to A but Dauphin et al. (2016) shows that GLUs perform better in the context of language modelling.

To enable deep convolutional networks, we add residual connections from the input of each convolution to the output of the block (He et al., 2015a).

$$h_i^l = v(W^l[h_{i-k/2}^{l-1}, \dots, h_{i+k/2}^{l-1}] + b_w^l) + h_i^{l-1}$$

For encoder networks we ensure that the output of the convolutional layers matches the input length by padding the input at each layer. However, for decoder networks we have to take care that no future information is available to the decoder (Oord et al., 2016a). Specifically, we pad the input by $k - 1$ elements on both the left and right side by zero vectors, and then remove k elements from the end of the convolution output.

We also add linear mappings to project between the embedding size f and the convolution outputs that are of size $2d$. We apply such a transform to w when feeding embeddings to the encoder network, to the encoder output z_j^u , to the final layer of the decoder just before the softmax $\mathbf{h}^L$, and to all decoder layers $\mathbf{h}^l$ before computing attention scores (1).

Finally, we compute a distribution over the T possible next target elements y_{i+1} by transforming the top decoder output h_i^L via a linear layer with weights W_o and bias b_o :

$$p(y_{i+1}|y_1, \dots, y_i, \mathbf{x}) = \text{softmax}(W_o h_i^L + b_o) \in \mathbb{R}^T$$

3.3. Multi-step Attention

We introduce a separate attention mechanism for each decoder layer. To compute the attention, we combine the current decoder state h_i^l with an embedding of the previous

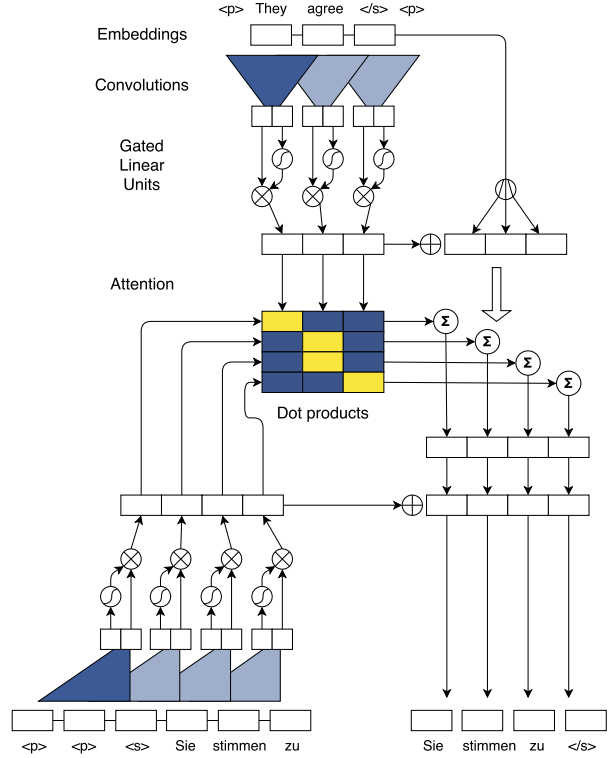


Figure 1. Illustration of batching during training. The English source sentence is encoded (top) and we compute all attention values for the four German target words (center) simultaneously. Our attentions are just dot products between decoder context representations (bottom left) and encoder representations. We add the conditional inputs computed by the attention (center right) to the decoder states which then predict the target words (bottom right). The sigmoid and multiplicative boxes illustrate Gated Linear Units.

target element g_i :

$$d_i^l = W_d^l h_i^l + b_d^l + g_i \quad (1)$$

For decoder layer l the attention a_{ij}^l of state i and source element j is computed as a dot-product between the decoder state summary d_i^l and each output z_j^u of the last encoder block u :

$$a_{ij}^l = \frac{\exp(d_i^l \cdot z_j^u)}{\sum_{t=1}^m \exp(d_i^l \cdot z_t^u)}$$

The conditional input c_i^l to the current decoder layer is a weighted sum of the encoder outputs as well as the input element embeddings e_j (Figure 1, center right):

$$c_i^l = \sum_{j=1}^m a_{ij}^l (z_j^u + e_j) \quad (2)$$

This is slightly different to recurrent approaches which compute both the attention and the weighted sum over z_j^u

only. We found adding e_j to be beneficial and it resembles key-value memory networks where the keys are the z_j^u and the values are the $z_j^u + e_j$ (Miller et al., 2016). Encoder outputs z_j^u represent potentially large input contexts and e_j provides point information about a specific input element that is useful when making a prediction. Once c_i^l has been computed, it is simply added to the output of the corresponding decoder layer h_i^l .

This can be seen as attention with multiple ‘hops’ (Sukhbaatar et al., 2015) compared to single step attention (Bahdanau et al., 2014; Luong et al., 2015; Zhou et al., 2016; Wu et al., 2016). In particular, the attention of the first layer determines a useful source context which is then fed to the second layer that takes this information into account when computing attention etc. The decoder also has immediate access to the attention history of the $k - 1$ previous time steps because the conditional inputs $c_{i-k}^{l-1}, \dots, c_i^{l-1}$ are part of $h_{i-k}^{l-1}, \dots, h_i^{l-1}$ which are input to h_i^l . This makes it easier for the model to take into account which previous inputs have been attended to already compared to recurrent nets where this information is in the recurrent state and needs to survive several non-linearities. Overall, our attention mechanism considers which words we previously attended to (Yang et al., 2016) and performs multiple attention ‘hops’ per time step. In Appendix §C, we plot attention scores for a deep decoder and show that at different layers, different portions of the source are attended to.

Our convolutional architecture also allows to batch the attention computation across all elements of a sequence compared to RNNs (Figure 1, middle). We batch the computations of each decoder layer individually.

3.4. Normalization Strategy

We stabilize learning through careful weight initialization (§3.5) and by scaling parts of the network to ensure that the variance throughout the network does not change dramatically. In particular, we scale the output of residual blocks as well as the attention to preserve the variance of activations. We multiply the sum of the input and output of a residual block by $\sqrt{0.5}$ to halve the variance of the sum. This assumes that both summands have the same variance which is not always true but effective in practice.

The conditional input c_i^l generated by the attention is a weighted sum of m vectors (2) and we counteract a change in variance through scaling by $m\sqrt{1/m}$; we multiply by m to scale up the inputs to their original size, assuming the attention scores are uniformly distributed. This is generally not the case but we found it to work well in practice.

For convolutional decoders with multiple attention, we scale the gradients for the encoder layers by the number

of attention mechanisms we use; we exclude source word embeddings. We found this to stabilize learning since the encoder received too much gradient otherwise.

3.5. Initialization

Normalizing activations when adding the output of different layers, e.g. residual connections, requires careful weight initialization. The motivation for our initialization is the same as for the normalization: maintain the variance of activations throughout the forward and backward passes. All embeddings are initialized from a normal distribution with mean 0 and standard deviation 0.1. For layers whose output is not directly fed to a gated linear unit, we initialize weights from $\mathcal{N}(0, \sqrt{1/n_l})$ where n_l is the number of input connections to each neuron. This ensures that the variance of a normally distributed input is retained.

For layers which are followed by a GLU activation, we propose a weight initialization scheme by adapting the derivations in (He et al., 2015b; Glorot & Bengio, 2010; Appendix A). If the GLU inputs are distributed with mean 0 and have sufficiently small variance, then we can approximate the output variance with $1/4$ of the input variance (Appendix A.1). Hence, we initialize the weights so that the input to the GLU activations have 4 times the variance of the layer input. This is achieved by drawing their initial values from $\mathcal{N}(0, \sqrt{4/n_l})$. Biases are uniformly set to zero when the network is constructed.

We apply dropout to the input of some layers so that inputs are retained with a probability of p . This can be seen as multiplication with a Bernoulli random variable taking value $1/p$ with probability p and 0 otherwise (Srivastava et al., 2014). The application of dropout will then cause the variance to be scaled by $1/p$. We aim to restore the incoming variance by initializing the respective layers with larger weights. Specifically, we use $\mathcal{N}(0, \sqrt{4p/n_l})$ for layers whose output is subject to a GLU and $\mathcal{N}(0, \sqrt{p/n_l})$ otherwise (Appendix A.3).

4. Experimental Setup

4.1. Datasets

We consider three major WMT translation tasks as well as a text summarization task.

WMT’16 English-Romanian. We use the same data and pre-processing as Sennrich et al. (2016b) but remove sentences with more than 175 words. This results in 2.8M sentence pairs for training and we evaluate on newstest2016.²

²We followed the pre-processing of <https://github.com/rsennrich/wmt16-scripts/blob/80e21e5/sample/preprocess.sh> and added the back-translated data from http://data.statmt.org/rsennrich/wmt16_

We experiment with word-based models using a source vocabulary of 200K types and a target vocabulary of 80K types. We also consider a joint source and target byte-pair encoding (BPE) with 40K types (Sennrich et al., 2016a;b).

WMT’14 English-German. We use the same setup as Luong et al. (2015) which comprises 4.5M sentence pairs for training and we test on newstest2014.³ As vocabulary we use 40K sub-word types based on BPE.

WMT’14 English-French. We use the full training set of 36M sentence pairs, and remove sentences longer than 175 words as well as pairs with a source/target length ratio exceeding 1.5. This results in 35.5M sentence-pairs for training. Results are reported on newstest2014. We use a source and target vocabulary with 40K BPE types.

In all setups a small subset of the training data serves as validation set (about 0.5-1% for each dataset) for early stopping and learning rate annealing.

Abstractive summarization. We train on the Gigaword corpus (Graff et al., 2003) and pre-process it identically to Rush et al. (2015) resulting in 3.8M training examples and 190K for validation. We evaluate on the DUC-2004 test data comprising 500 article-title pairs (Over et al., 2007) and report three variants of recall-based ROUGE (Lin, 2004), namely, ROUGE-1 (unigrams), ROUGE-2 (bigrams), and ROUGE-L (longest-common substring). We also evaluate on a Gigaword test set of 2000 pairs which is identical to the one used by Rush et al. (2015) and we report F1 ROUGE similar to prior work. Similar to Shen et al. (2016) we use a source and target vocabulary of 30K words and require outputs to be at least 14 words long.

4.2. Model Parameters and Optimization

We use 512 hidden units for both encoders and decoders, unless otherwise stated. All embeddings, including the output produced by the decoder before the final linear layer, have dimensionality 512; we use the same dimensionalities for linear layers mapping between the hidden and embedding sizes (§3.2).

We train our convolutional models with Nesterov’s accelerated gradient method (Sutskever et al., 2013) using a momentum value of 0.99 and renormalize gradients if their norm exceeds 0.1 (Pascanu et al., 2013). We use a learning rate of 0.25 and once the validation perplexity stops improving, we reduce the learning rate by an order of magnitude after each epoch until it falls below 10^{-4} .

Unless otherwise stated, we use mini-batches of 64 sentences. We restrict the maximum number of words in a mini-batch to make sure that batches with long sentences

backtranslations/en-ro.

³<http://nlp.stanford.edu/projects/nmt>

still fit in GPU memory. If the threshold is exceeded, we simply split the batch until the threshold is met and process the parts separately. Gradients are normalized by the number of non-padding tokens per mini-batch. We also use weight normalization for all layers except for lookup tables (Salimans & Kingma, 2016).

Besides dropout on the embeddings and the decoder output, we also apply dropout to the input of the convolutional blocks (Srivastava et al., 2014). All models are implemented in Torch (Collobert et al., 2011) and trained on a single Nvidia M40 GPU except for WMT’14 English-French for which we use a multi-GPU setup on a single machine. We train on up to eight GPUs synchronously by maintaining copies of the model on each card and split the batch so that each worker computes 1/8-th of the gradients; at the end we sum the gradients via Nvidia NCCL.

4.3. Evaluation

We report average results over three runs of each model, where each differs only in the initial random seed. Translations are generated by a beam search and we normalize log-likelihood scores by sentence length. We use a beam of width 5. We divide the log-likelihoods of the final hypothesis in beam search by their length $|y|$. For WMT’14 English-German we tune a length normalization constant on a separate development set (newstest2015) and we normalize log-likelihoods by $|y|^\alpha$ (Wu et al., 2016). On other datasets we did not find any benefit with length normalization.

For word-based models, we perform unknown word replacement based on attention scores after generation (Jean et al., 2015). Unknown words are replaced by looking up the source word with the maximum attention score in a pre-computed dictionary. If the dictionary contains no translation, then we simply copy the source word. Dictionaries were extracted from the word aligned training data that we obtained with fast_align (Dyer et al., 2013). Each source word is mapped to the target word it is most frequently aligned to. In our multi-step attention (§3.3) we simply average the attention scores over all layers. Finally, we compute case-sensitive tokenized BLEU, except for WMT’16 English-Romanian where we use detokenized BLEU to be comparable with Sennrich et al. (2016b).⁴

⁴<https://github.com/moses-smt/mosesdecoder/blob/617e8c8/scripts/generic/{multi-bleu.perl,mteval-v13a.pl}>

5. Results

5.1. Recurrent vs. Convolutional Models

We first evaluate our convolutional model on three translation tasks. On WMT’16 English-Romanian translation we compare to [Sennrich et al. \(2016b\)](#) which is the winning entry on this language pair at WMT’16 ([Bojar et al., 2016](#)). Their model implements the attention-based sequence to sequence architecture of [Bahdanau et al. \(2014\)](#) and uses GRU cells both in the encoder and decoder. We test both word-based and BPE vocabularies (§4).

Table 1 shows that our fully convolutional sequence to sequence model (ConvS2S) outperforms the WMT’16 winning entry for English-Romanian by 1.9 BLEU with a BPE encoding and by 1.3 BLEU with a word factored vocabulary. This instance of our architecture has 20 layers in the encoder and 20 layers in the decoder, both using kernels of width 3 and hidden size 512 throughout. Training took between 6 and 7.5 days on a single GPU.

On WMT’14 English to German translation we compare to the following prior work: [Luong et al. \(2015\)](#) is based on a four layer LSTM attention model, ByteNet ([Kalchbrenner et al., 2016](#)) propose a convolutional model based on characters without attention, with 30 layers in the encoder and 30 layers in the decoder, GNMT ([Wu et al., 2016](#)) represents the state of the art on this dataset and they use eight encoder LSTMs as well as eight decoder LSTMs, we quote their result for a word-based model, such as ours, as well as a word-piece model ([Schuster & Nakajima, 2012](#)).⁵

The results (Table 1) show that our convolutional model outperforms GNMT by 0.5 BLEU. Our encoder has 15 layers and the decoder has 15 layers, both with 512 hidden units in the first ten layers and 768 units in the subsequent three layers, all using kernel width 3. The final two layers have 2048 units which are just linear mappings with a single input. We trained this model on a single GPU over a period of 18.5 days with a batch size of 48. LSTM sparse mixtures have shown strong accuracy at 26.03 BLEU for a single run ([Shazeer et al., 2016](#)) which compares to 25.39 BLEU for our best run. This mixture sums the output of four experts, not unlike an ensemble which sums the output of multiple networks. ConvS2S also benefits from ensembling (§5.2), therefore mixtures are a promising direction.

Finally, we train on the much larger WMT’14 English-French task where we compare to the state of the art result of GNMT ([Wu et al., 2016](#)). Our model is trained with a simple token-level likelihood objective and we improve over GNMT in the same setting by 1.6 BLEU on average. We also outperform their reinforcement (RL) models by 0.5

⁵We did not use the exact same vocabulary size because word pieces and BPE estimate the vocabulary differently.

WMT’16 English-Romanian	BLEU
Sennrich et al. (2016b) GRU (BPE 90K)	28.1
ConvS2S (Word 80K)	29.45
ConvS2S (BPE 40K)	30.02
WMT’14 English-German	BLEU
Luong et al. (2015) LSTM (Word 50K)	20.9
Kalchbrenner et al. (2016) ByteNet (Char)	23.75
Wu et al. (2016) GNMT (Word 80K)	23.12
Wu et al. (2016) GNMT (Word pieces)	24.61
ConvS2S (BPE 40K)	25.16
WMT’14 English-French	BLEU
Wu et al. (2016) GNMT (Word 80K)	37.90
Wu et al. (2016) GNMT (Word pieces)	38.95
Wu et al. (2016) GNMT (Word pieces) + RL	39.92
ConvS2S (BPE 40K)	40.51

Table 1. Accuracy on WMT tasks compared to previous work. ConvS2S and GNMT results are averaged over several runs.

BLEU. Reinforcement learning is equally applicable to our architecture and we believe that it would further improve our results.

The ConvS2S model for this experiment uses 15 layers in the encoder and 15 layers in the decoder, both with 512 hidden units in the first five layers, 768 units in the subsequent four layers, 1024 units in the next 3 layers, all using kernel width 3; the final two layers have 2048 units and 4096 units each but they are linear mappings with kernel width 1. This model has an effective context size of only 25 words, beyond which it cannot access any information on the target size. Our results are based on training with 8 GPUs for about 37 days and batch size 32 on each worker.⁶ The same configuration as for WMT’14 English-German achieves 39.41 BLEU in two weeks on this dataset in an eight GPU setup.

[Zhou et al. \(2016\)](#) report a non-averaged result of 39.2 BLEU. More recently, [Ha et al. \(2016\)](#) showed that one can generate weights with one LSTM for another LSTM. This approach achieves 40.03 BLEU but the result is not averaged. [Shazeer et al. \(2016\)](#) compares at 40.56 BLEU to our best single run of 40.70 BLEU.

⁶This is half of the GPU time consumed by a basic model of [Wu et al. \(2016\)](#) who use 96 GPUs for 6 days. We expect the time to train our model to decrease substantially in a multi-machine setup.

WMT'14 English-German	BLEU
Wu et al. (2016) GNMT	26.20
Wu et al. (2016) GNMT + RL	26.30
ConvS2S	26.43

WMT'14 English-French	BLEU
Zhou et al. (2016)	40.4
Wu et al. (2016) GNMT	40.35
Wu et al. (2016) GNMT + RL	41.16
ConvS2S	41.44
ConvS2S (10 models)	41.62

Table 2. Accuracy of ensembles with eight models. We show both likelihood and Reinforce (RL) results for GNMT; Zhou et al. (2016) and ConvS2S use simple likelihood training.

The translations produced by our models often match the length of the references, particularly for the large WMT'14 English-French task, or are very close for small to medium data sets such as WMT'14 English-German or WMT'16 English-Romanian.

5.2. Ensemble Results

Next, we ensemble eight likelihood-trained models for both WMT'14 English-German and WMT'14 English-French and compare to previous work which also reported ensemble results. For the former, we also show the result when ensembling 10 models. Table 2 shows that we outperform the best current ensembles on both datasets.

5.3. Generation Speed

Next, we evaluate the inference speed of our architecture on the development set of the WMT'14 English-French task which is the concatenation of newstest2012 and newstest2013; it comprises 6003 sentences. We measure generation speed both on GPU and CPU hardware. Specifically, we measure GPU speed on three generations of Nvidia cards: a GTX-1080ti, an M40 as well as an older K40 card. CPU timings are measured on one host with 48 hyper-threaded cores (Intel Xeon E5-2680 @ 2.50GHz) with 40 workers. In all settings, we batch up to 128 sentences, composing batches with sentences of equal length. Note that the majority of batches is smaller because of the small size of the development set. We experiment with beams of size 5 as well as greedy search, i.e beam of size 1. To make generation fast, we do not recompute convolution states that have not changed compared to the previous time step but rather copy (shift) these activations.

We compare to results reported in Wu et al. (2016) who

	BLEU	Time (s)
GNMT GPU (K80)	31.20	3,028
GNMT CPU 88 cores	31.20	1,322
GNMT TPU	31.21	384
ConvS2S GPU (K40) $b = 1$	33.45	327
ConvS2S GPU (M40) $b = 1$	33.45	221
ConvS2S GPU (GTX-1080ti) $b = 1$	33.45	142
ConvS2S CPU 48 cores $b = 1$	33.45	142
ConvS2S GPU (K40) $b = 5$	34.10	587
ConvS2S CPU 48 cores $b = 5$	34.10	482
ConvS2S GPU (M40) $b = 5$	34.10	406
ConvS2S GPU (GTX-1080ti) $b = 5$	34.10	256

Table 3. CPU and GPU generation speed in seconds on the development set of WMT'14 English-French. We show results for different beam sizes b . GNMT figures are taken from Wu et al. (2016). CPU speeds are not directly comparable because Wu et al. (2016) use a 88 core machine versus our 48 core setup.

use Nvidia K80 GPUs which are essentially two K40s. We did not have such a GPU available and therefore run experiments on an older K40 card which is inferior to a K80, in addition to the newer M40 and GTX-1080ti cards. The results (Table 3) show that our model can generate translations on a K40 GPU at 9.3 times the speed and 2.25 higher BLEU; on an M40 the speed-up is up to 13.7 times and on a GTX-1080ti card the speed is 21.3 times faster. A larger beam of size 5 decreases speed but gives better BLEU.

On CPU, our model is up to 9.3 times faster, however, the GNMT CPU results were obtained with an 88 core machine whereas our results were obtained with just over half the number of cores. On a per CPU core basis, our model is 17 times faster at a better BLEU. Finally, our CPU speed is 2.7 times higher than GNMT on a custom TPU chip which shows that high speed can be achieved on commodity hardware. We do not report TPU figures as we do not have access to this hardware.

5.4. Position Embeddings

In the following sections, we analyze the design choices in our architecture. The remaining results in this paper are based on the WMT'14 English-German task with 13 encoder layers at kernel size 3 and 5 decoder layers at kernel size 5. We use a target vocabulary of 160K words as well as vocabulary selection (Mi et al., 2016; L'Hostis et al., 2016) to decrease the size of the output layer which speeds up training and testing. The average vocabulary size for each training batch is about 20K target words. All figures are averaged over three runs (§4) and BLEU is reported on newstest2014 before unknown word replacement.

We start with an experiment that removes the position em-

	PPL	BLEU
ConvS2S	6.64	21.7
-source position	6.69	21.3
-target position	6.63	21.5
-source & target position	6.68	21.2

Table 4. Effect of removing position embeddings from our model in terms of validation perplexity (valid PPL) and BLEU.

beddings from the encoder and decoder (§3.1). These embeddings allow our model to identify which portion of the source and target sequence it is dealing with but also impose a restriction on the maximum sentence length. Table 4 shows that position embeddings are helpful but that our model still performs well without them. Removing the source position embeddings results in a larger accuracy decrease than target position embeddings. However, removing both source and target positions decreases accuracy only by 0.5 BLEU. We had assumed that the model would not be able to calibrate the length of the output sequences very well without explicit position information, however, the output lengths of models without position embeddings closely matches models with position information. This indicates that the models can learn relative position information within the contexts visible to the encoder and decoder networks which can observe up to 27 and 25 words respectively.

Recurrent models typically do not use explicit position embeddings since they can learn where they are in the sequence through the recurrent hidden state computation. In our setting, the use of position embeddings requires only a simple addition to the input word embeddings which is a negligible overhead.

5.5. Multi-step Attention

The multiple attention mechanism (§3.3) computes a separate source context vector for each decoder layer. The computation also takes into account contexts computed for preceding decoder layers of the current time step as well as previous time steps that are within the receptive field of the decoder. How does multiple attention compare to attention in fewer layers or even only in a single layer as is usual? Table 5 shows that attention in all decoder layers achieves the best validation perplexity (PPL). Furthermore, removing more and more attention layers decreases accuracy, both in terms of BLEU as well as PPL.

The computational overhead for attention is very small compared to the rest of the network. Training with attention in all five decoder layers processes 3624 target words per second on average on a single GPU, compared to 3772 words per second for attention in a single layer. This is only

Attn Layers	PPL	BLEU
1,2,3,4,5	6.65	21.63
1,2,3,4	6.70	21.54
1,2,3	6.95	21.36
1,2	6.92	21.47
1,3,5	6.97	21.10
1	7.15	21.26
2	7.09	21.30
3	7.11	21.19
4	7.19	21.31
5	7.66	20.24

Table 5. Multi-step attention in all five decoder layers or fewer layers in terms of validation perplexity (PPL) and test BLEU.

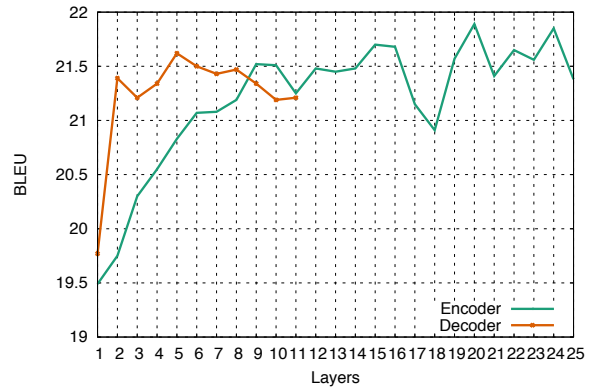


Figure 2. Encoder and decoder with different number of layers.

a 4% slow down when adding 4 attention modules. Most neural machine translation systems only use a single module. This demonstrates that attention is not the bottleneck in neural machine translation, even though it is quadratic in the sequence length (cf. Kalchbrenner et al., 2016). Part of the reason for the low impact on speed is that we batch the computation of an attention module over all target words, similar to Kalchbrenner et al. (2016). However, for RNNs batching of the attention may be less effective because of the dependence on the previous time step.

5.6. Kernel size and Depth

Figure 2 shows accuracy when we change the number of layers in the encoder or decoder. The kernel width for layers in the encoder is 3 and for the decoder it is 5. Deeper architectures are particularly beneficial for the encoder but less so for the decoder. Decoder setups with two layers already perform well whereas for the encoder accuracy keeps increasing steadily with more layers until up to 9 layers when accuracy starts to plateau.

	DUC-2004			Gigaword		
	RG-1 (R)	RG-2 (R)	RG-L (R)	RG-1 (F)	RG-2 (F)	RG-L (F)
RNN MLE (Shen et al., 2016)	24.92	8.60	22.25	32.67	15.23	30.56
RNN MRT (Shen et al., 2016)	30.41	10.87	26.79	36.54	16.59	33.44
WFE (Suzuki & Nagata, 2017)	32.28	10.54	27.80	36.30	17.31	33.88
ConvS2S	30.44	10.84	26.90	35.88	17.48	33.29

Table 6. Accuracy on two summarization tasks in terms of Rouge-1 (RG-1), Rouge-2 (RG-2), and Rouge-L (RG-L).

Kernel width	Encoder layers		
	5	9	13
3	20.61	21.17	21.63
5	20.80	21.02	21.42
7	20.81	21.30	21.09

Table 7. Encoder with different kernel width in terms of BLEU.

Kernel width	Decoder layers		
	3	5	7
3	21.10	21.71	21.62
5	21.09	21.63	21.24
7	21.40	21.31	21.33

Table 8. Decoder with different kernel width in terms of BLEU.

Aside from increasing the depth of the networks, we can also change the kernel width. Table 7 shows that encoders with narrow kernels and many layers perform better than wider kernels. These networks can also be faster since the amount of work to compute a kernel operating over 3 input elements is less than half compared to kernels over 7 elements. We see a similar picture for decoder networks with large kernel sizes (Table 8). Dauphin et al. (2016) shows that context sizes of 20 words are often sufficient to achieve very good accuracy on language modeling for English.

5.7. Summarization

Finally, we evaluate our model on abstractive sentence summarization which takes a long sentence as input and outputs a shortened version. The current best models on this task are recurrent neural networks which either optimize the evaluation metric (Shen et al., 2016) or address specific problems of summarization such as avoiding repeated generations (Suzuki & Nagata, 2017). We use standard likelihood training for our model and a simple model with six layers in the encoder and decoder each, hidden size 256, batch size 128, and we trained on a single GPU in one night. Table 6 shows that our likelihood trained model outperforms the likelihood trained model (RNN MLE) of Shen et al. (2016) and is not far behind the best models on this task which benefit from task-specific optimization and

model structure. We expect our model to benefit from these improvements as well.

6. Conclusion and Future Work

We introduce the first fully convolutional model for sequence to sequence learning that outperforms strong recurrent models on very large benchmark datasets at an order of magnitude faster speed. Compared to recurrent networks, our convolutional approach allows to discover compositional structure in the sequences more easily since representations are built hierarchically. Our model relies on gating and performs multiple attention steps.

We achieve a new state of the art on several public translation benchmark data sets. On the WMT’16 English-Romanian task we outperform the previous best result by 1.9 BLEU, on WMT’14 English-French translation we improve over the LSTM model of Wu et al. (2016) by 1.6 BLEU in a comparable setting, and on WMT’14 English-German translation we outperform the same model by 0.5 BLEU. In future work, we would like to apply convolutional architectures to other sequence to sequence learning problems which may benefit from learning hierarchical representations as well.

Acknowledgements

We thank Benjamin Graham for providing a fast 1-D convolution, and Ronan Collobert as well as Yann LeCun for helpful discussions related to this work.

References

- Ba, Jimmy Lei, Kiros, Jamie Ryan, and Hinton, Geoffrey E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- Bojar, Ondrej, Chatterjee, Rajen, Federmann, Christian, Graham, Yvette, Haddow, Barry, Huck, Matthias,

A. Weight Initialization

We derive a weight initialization scheme tailored to the GLU activation function similar to [Glorot & Bengio \(2010\)](#); [He et al. \(2015b\)](#) by focusing on the variance of activations within the network for both forward and backward passes. We also detail how we modify the weight initialization for dropout.

A.1. Forward Pass

Assuming that the inputs $\mathbf{x}_l$ of a convolutional layer l and its weights W_l are independent and identically distributed (i.i.d.), the variance of its output, computed as $\mathbf{y}_l = W_l \mathbf{x}_l + \mathbf{b}_l$, is

$$Var[\mathbf{y}_l] = n_l Var[w_l x_l] \quad (3)$$

where n_l is the number inputs to the layer. For one-dimensional convolutional layers with kernel width k and input dimension c , this is kc . We adopt the notation in [\(He et al., 2015b\)](#), i.e. y_l , w_l and x_l represent the random variables in $\mathbf{y}_l$, W_l and $\mathbf{x}_l$. With w_l and x_l independent from each other and normally distributed with zero mean, this amounts to

$$Var[y_l] = n_l Var[w_l] Var[x_l]. \quad (4)$$

$\mathbf{x}_l$ is the result of the GLU activation function $\mathbf{y}_{l-1}^a \sigma(\mathbf{y}_{l-1}^b)$ with $\mathbf{y}_{l-1} = (\mathbf{y}_{l-1}^a, \mathbf{y}_{l-1}^b)$, and $\mathbf{y}_{l-1}^a, \mathbf{y}_{l-1}^b$ i.i.d. Next, we formulate upper and lower bounds in order to approximate $Var[x_l]$. If $\mathbf{y}_{l-1}$ follows a symmetric distribution with mean 0, then

$$Var[x_l] = Var[y_{l-1}^a \sigma(y_{l-1}^b)] \quad (5)$$

$$= E[(y_{l-1}^a \sigma(y_{l-1}^b))^2] - E^2[y_{l-1}^a \sigma(y_{l-1}^b)] \quad (6)$$

$$= Var[y_{l-1}^a] E[\sigma(y_{l-1}^b)^2]. \quad (7)$$

A lower bound is given by $(1/4)Var[y_{l-1}^a]$ when expanding (6) with $E^2[\sigma(y_{l-1}^b)] = 1/4$:

$$Var[x_l] = Var[y_{l-1}^a \sigma(y_{l-1}^b)] \quad (8)$$

$$= Var[y_{l-1}^a] E^2[\sigma(y_{l-1}^b)] + \quad (9)$$

$$Var[y_{l-1}^a] Var[\sigma(y_{l-1}^b)] \\ = \frac{1}{4} Var[y_{l-1}^a] + Var[y_{l-1}^a] Var[\sigma(y_{l-1}^b)] \quad (10)$$

and $Var[y_{l-1}^a] Var[\sigma(y_{l-1}^b)] > 0$. We utilize the relation $\sigma(x)^2 \leq (1/16)x^2 - 1/4 + \sigma(x)$ ([Appendix B](#)) to provide an upper bound on $E[\sigma(x)^2]$:

$$E[\sigma(x)^2] \leq E\left[\frac{1}{16}x^2 - \frac{1}{4} + \sigma(x)\right] \quad (11)$$

$$= \frac{1}{16} E[x^2] - \frac{1}{4} + E[\sigma(x)] \quad (12)$$

With $x \sim \mathcal{N}(0, std(x))$, this yields

$$E[\sigma(x)^2] \leq \frac{1}{16} E[x^2] - \frac{1}{4} + \frac{1}{2} \quad (13)$$

$$= \frac{1}{16} Var[x] + \frac{1}{4}. \quad (14)$$

With (7) and $Var[y_{l-1}^a] = Var[y_{l-1}^b] = Var[y_{l-1}]$, this results in

$$Var[x_l] \leq \frac{1}{16} Var[y_{l-1}]^2 + \frac{1}{4} Var[y_{l-1}]. \quad (15)$$

We initialize the embedding matrices in our network with small variances (around 0.01), which allows us to dismiss the quadratic term and approximate the GLU output variance with

$$Var[x_l] \approx \frac{1}{4} Var[y_{l-1}]. \quad (16)$$

If L network layers of equal size and with GLU activations are combined, the variance of the final output $\mathbf{y}_L$ is given by

$$Var[y_L] \approx Var[y_1] \left(\prod_{l=2}^L \frac{1}{4} n_l Var[w_l] \right). \quad (17)$$

Following [\(He et al., 2015b\)](#), we aim to satisfy the condition

$$\frac{1}{4} n_l Var[w_l] = 1, \forall l \quad (18)$$

so that the activations in a network are neither exponentially magnified nor reduced. This is achieved by initializing W_l from $\mathcal{N}(0, \sqrt{4/n_l})$.

A.2. Backward Pass

The gradient of a convolutional layer is computed via back-propagation as $\Delta \mathbf{x}_l = \hat{W}_l \mathbf{y}_l$. Considering separate gradients $\Delta \mathbf{y}_l^a$ and $\Delta \mathbf{y}_l^b$ for GLU, the gradient of $\mathbf{x}$ is given by

$$\Delta \mathbf{x}_l = \hat{W}_l^a \Delta \mathbf{y}_l^a + \hat{W}_l^b \Delta \mathbf{y}_l^b. \quad (19)$$

$\hat{W}$ corresponds to W with re-arranged weights to enable back-propagation. Analogously to the forward pass, Δx_l , $\hat{w}_l$ and Δy_l represent the random variables for the values in $\Delta \mathbf{x}_l$, $\hat{W}_l$ and $\Delta \mathbf{y}_l$, respectively. Note that W and $\hat{W}$ contain the same values, i.e. $\hat{w} = w$. Similar to (3), the variance of Δx_l is

$$Var[\Delta x_l] = \hat{n}_l \left(Var[w_l^a] Var[\Delta y_l^a] + Var[w_l^b] Var[\Delta y_l^b] \right). \quad (20)$$

Here, $\hat{n}_l$ is the number of inputs to layer $l+1$. The gradients for the GLU inputs are:

$$\Delta \mathbf{y}_l^a = \Delta \mathbf{x}_{l+1} \sigma(\mathbf{y}_l^b) \quad \text{and} \quad (21)$$

$$\Delta \mathbf{y}_l^b = \Delta \mathbf{x}_{l+1} \mathbf{y}_l^a \sigma'(\mathbf{y}_l^b). \quad (22)$$

The approximation for the forward pass can be used for $Var[\Delta y_l^a]$, and for estimating $Var[\Delta y_l^b]$ we assume an upper bound on $E[\sigma'(y_l^b)^2]$ of $1/16$ since $\sigma'(y_l^b) \in [0, \frac{1}{4}]$. Hence,

$$Var[\Delta y_l^a] - \frac{1}{4}Var[\Delta x_{l+1}] \leq \frac{1}{16}Var[\Delta x_{l+1}]Var[y_l^b] \quad (23)$$

$$Var[\Delta y_l^b] \leq \frac{1}{16}\Delta Var[\Delta x_{l+1}]Var[y_l^a] \quad (24)$$

We observe relatively small gradients in our network, typically around 0.001 at the start of training. Therefore, we approximate by discarding the quadratic terms above, i.e.

$$Var[\Delta y_l^a] \approx \frac{1}{4}Var[\Delta x_{l+1}] \quad (25)$$

$$Var[\Delta y_l^b] \approx 0 \quad (26)$$

$$Var[\Delta x_l] \approx \frac{1}{4}\hat{n}_l Var[w_l^a]Var[\Delta x_{l+1}] \quad (27)$$

As for the forward pass, the above result can be generalized to backpropagation through many successive layers, resulting in

$$Var[\Delta x_2] \approx Var[\Delta x_{L+1}] \left(\prod_{l=2}^L \frac{1}{4}\hat{n}_l Var[w_l^a] \right) \quad (28)$$

and a similar condition, i.e. $(1/4)\hat{n}_l Var[w_l^a] = 1$. In the networks we consider, successions of convolutional layers usually operate on the same number of inputs so that most cases $n_l = \hat{n}_l$. Note that W_l^b is discarded in the approximation; however, for the sake of consistency we use the same initialization for W_l^a and W_l^b .

For arbitrarily large variances of network inputs and activations, our approximations are invalid; in that case, the initial values for W_l^a and W_l^b would have to be balanced for the input distribution to be retained. Alternatively, methods that explicitly control the variance in the network, e.g. batch normalization (Ioffe & Szegedy, 2015) or layer normalization (Ba et al., 2016) could be employed.

A.3. Dropout

Dropout retains activations in a neural network with a probability p and sets them to zero otherwise (Srivastava et al., 2014). It is common practice to scale the retained activations by $1/p$ during training so that the weights of the network do not have to be modified at test time when p is set to 1. In this case, dropout amounts to multiplying activations x by a Bernoulli random variable r where $\Pr[r = 1/p] = p$ and $\Pr[r = 0] = 1 - p$ (Srivastava et al., 2014). It holds that $E[r] = 1$ and $Var[r] = (1 - p)/p$. If x is independent

of r and $E[x] = 0$, the variance after dropout is

$$Var[xr] = E[r]^2 Var[x] + Var[r]Var[x] \quad (29)$$

$$= \left(1 + \frac{1-p}{p}\right) Var[x] \quad (30)$$

$$= \frac{1}{p} Var[x] \quad (31)$$

Assuming that the *input* of a convolutional layer has been subject to dropout with a retain probability p , the variations of the forward and backward activations from §A.1 and §A.2 can now be approximated with

$$Var[x_{l+1}] \approx \frac{1}{4p}n_l Var[w_l]Var[x_l] \quad \text{and} \quad (32)$$

$$Var[\Delta x_l] \approx \frac{1}{4p}n_l Var[w_l^a]Var[\Delta x_{l+1}]. \quad (33)$$

This amounts to a modified initialization of W_l from a normal distribution with zero mean and a standard deviation of $\sqrt{4p/n}$. For layers without a succeeding GLU activation function, we initialize weights from $\mathcal{N}(0, \sqrt{p/n})$ to calibrate for any immediately preceding dropout application.

B. Upper Bound on Squared Sigmoid

The sigmoid function $\sigma(x)$ can be expressed as a hyperbolic tangent by using the identity $\tanh(x) = 2\sigma(2x) - 1$. The derivative of $\tanh$ is $\tanh'(x) = 1 - \tanh^2(x)$, and with $\tanh(x) \in [0, 1]$, $x \geq 0$ it holds that

$$\tanh'(x) \leq 1, x \geq 0 \quad (34)$$

$$\int_0^x \tanh'(x) dx \leq \int_0^x 1 dx \quad (35)$$

$$\tanh(x) \leq x, x \geq 0 \quad (36)$$

We can express this relation with $\sigma(x)$ as follows:

$$2\sigma(x) - 1 \leq \frac{1}{2}x, x \geq 0 \quad (37)$$

Both terms of this inequality have rotational symmetry w.r.t 0, and thus

$$(2\sigma(x) - 1)^2 \leq \left(\frac{1}{2}x\right)^2 \quad \forall x \quad (38)$$

$$\Leftrightarrow \sigma(x)^2 \leq \frac{1}{16}x^2 - \frac{1}{4} + \sigma(x). \quad (39)$$

C. Attention Visualization

Figure 3 shows attention scores for a generated sentence from the WMT'14 English-German task. The model used for this plot has 8 decoder layers and a 80K BPE vocabulary. The attention passes in different decoder layers capture different portions of the source sentence. Layer 1, 3

and 6 exhibit a linear alignment. The first layer shows the clearest alignment, although it is slightly off and frequently attends to the corresponding source word of the previously generated target word. Layer 2 and 8 lack a clear structure and are presumably collecting information about the whole source sentence. The fourth layer shows high alignment scores on nouns such as “festival”, “way” and “work” for both the generated target nouns as well as their preceding words. Note that in German, those preceding words depend on gender and object relationship of the respective noun. Finally, the attention scores in layer 5 and 7 focus on “built”, which is reordered in the German translation and is moved from the beginning to the very end of the sentence. One interpretation for this is that as generation progresses, the model repeatedly tries to perform the re-ordering. “aufgebaut” can be generated after a noun or pronoun only, which is reflected in the higher scores at positions 2, 5, 8, 11 and 13.

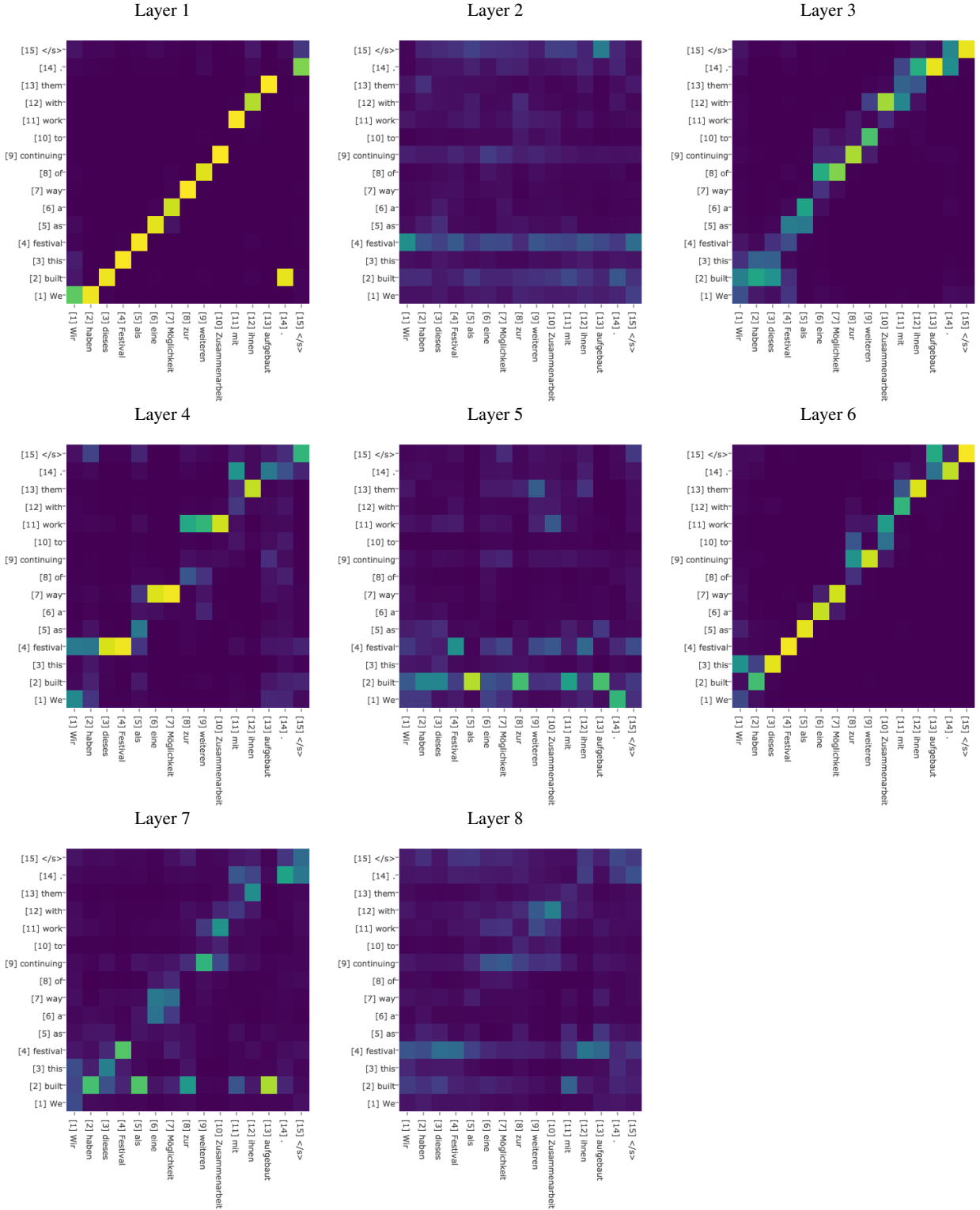


Figure 3. Attention scores for different decoder layers for a sentence translated from English (y-axis) to German (x-axis). This model uses 8 decoder layers and a 80k BPE vocabulary.

WAVENET: A GENERATIVE MODEL FOR RAW AUDIO

Aäron van den Oord

Sander Dieleman

Heiga Zen[†]

Karen Simonyan

Oriol Vinyals

Alex Graves

Nal Kalchbrenner

Andrew Senior

Koray Kavukcuoglu

{avdnoord, sedielem, heigazen, simonyan, vinyals, graves, nalk, andrewsenior, korayk}@google.com
Google DeepMind, London, UK

[†] Google, London, UK

ABSTRACT

This paper introduces WaveNet, a deep neural network for generating raw audio waveforms. The model is fully probabilistic and autoregressive, with the predictive distribution for each audio sample conditioned on all previous ones; nonetheless we show that it can be efficiently trained on data with tens of thousands of samples per second of audio. When applied to text-to-speech, it yields state-of-the-art performance, with human listeners rating it as significantly more natural sounding than the best parametric and concatenative systems for both English and Mandarin. A single WaveNet can capture the characteristics of many different speakers with equal fidelity, and can switch between them by conditioning on the speaker identity. When trained to model music, we find that it generates novel and often highly realistic musical fragments. We also show that it can be employed as a discriminative model, returning promising results for phoneme recognition.

1 INTRODUCTION

This work explores raw audio generation techniques, inspired by recent advances in neural autoregressive generative models that model complex distributions such as images (van den Oord et al., 2016a,b) and text (Jóźefowicz et al., 2016). Modeling joint probabilities over pixels or words using neural architectures as products of conditional distributions yields state-of-the-art generation

Remarkably, these architectures are able to model distributions over thousands of random variables (e.g. 64×64 pixels as in PixelRNN (van den Oord et al., 2016a)). The question this paper addresses is whether similar approaches can succeed in generating wideband raw audio waveforms, which are signals with very high temporal resolution, at least 16,000 samples per second (see Fig. 1).



Figure 1: A second of generated speech.

This paper introduces *WaveNet*, an audio generative model based on the PixelCNN (van den Oord et al., 2016a,b) architecture. The main contributions of this work are as follows:

- We show that WaveNets can generate raw speech signals with subjective naturalness never before reported in the field of text-to-speech (TTS), as assessed by human raters.

- In order to deal with long-range temporal dependencies needed for raw audio generation, we develop new architectures based on dilated causal convolutions, which exhibit very large receptive fields.
- We show that when conditioned on a speaker identity, a single model can be used to generate different voices.
- The same architecture shows strong results when tested on a small speech recognition dataset, and is promising when used to generate other audio modalities such as music.

We believe that WaveNets provide a generic and flexible framework for tackling many applications that rely on audio generation (e.g. TTS, music, speech enhancement, voice conversion, source separation).

2 WAVE.NET

In this paper we introduce a new generative model operating directly on the raw audio waveform. The joint probability of a waveform $\mathbf{x} = \{x_1, \dots, x_T\}$ is factorised as a product of conditional probabilities as follows:

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \quad (1)$$

Each audio sample x_t is therefore conditioned on the samples at all previous timesteps.

Similarly to PixelCNNs (van den Oord et al., 2016a,b), the conditional probability distribution is modelled by a stack of convolutional layers. There are no pooling layers in the network, and the output of the model has the same time dimensionality as the input. The model outputs a categorical distribution over the next value x_t with a softmax layer and it is optimized to maximize the log-likelihood of the data w.r.t. the parameters. Because log-likelihoods are tractable, we tune hyper-parameters on a validation set and can easily measure if the model is overfitting or underfitting.

2.1 DILATED CAUSAL CONVOLUTIONS

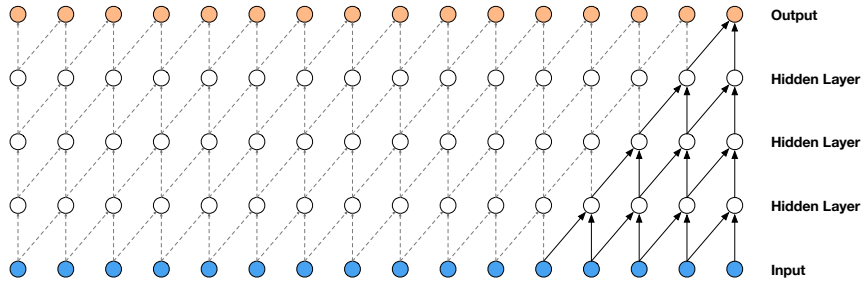


Figure 2: Visualization of a stack of causal convolutional layers.

The main ingredient of WaveNet are causal convolutions. By using causal convolutions, we make sure the model cannot violate the ordering in which we model the data: the prediction $p(x_{t+1} | x_1, \dots, x_t)$ emitted by the model at timestep t cannot depend on any of the future timesteps $x_{t+1}, x_{t+2}, \dots, x_T$ as shown in Fig. 2. For images, the equivalent of a causal convolution is a masked convolution (van den Oord et al., 2016a) which can be implemented by constructing a mask tensor and doing an elementwise multiplication of this mask with the convolution kernel before applying it. For 1-D data such as audio one can more easily implement this by shifting the output of a normal convolution by a few timesteps.

At training time, the conditional predictions for all timesteps can be made in parallel because all timesteps of ground truth $\mathbf{x}$ are known. When generating with the model, the predictions are sequential: after each sample is predicted, it is fed back into the network to predict the next sample.

Because models with causal convolutions do not have recurrent connections, they are typically faster to train than RNNs, especially when applied to very long sequences. One of the problems of causal convolutions is that they require many layers, or large filters to increase the receptive field. For example, in Fig. 2 the receptive field is only 5 ($= \text{\#layers} + \text{filter length} - 1$). In this paper we use dilated convolutions to increase the receptive field by orders of magnitude, without greatly increasing computational cost.

A dilated convolution (also called *à trous*, or convolution with holes) is a convolution where the filter is applied over an area larger than its length by skipping input values with a certain step. It is equivalent to a convolution with a larger filter derived from the original filter by dilating it with zeros, but is significantly more efficient. A dilated convolution effectively allows the network to operate on a coarser scale than with a normal convolution. This is similar to pooling or strided convolutions, but here the output has the same size as the input. As a special case, dilated convolution with dilation 1 yields the standard convolution. Fig. 3 depicts dilated causal convolutions for dilations 1, 2, 4, and 8. Dilated convolutions have previously been used in various contexts, e.g. signal processing (Holschneider et al., 1989; Dutilleul, 1989), and image segmentation (Chen et al., 2015; Yu & Koltun, 2016).

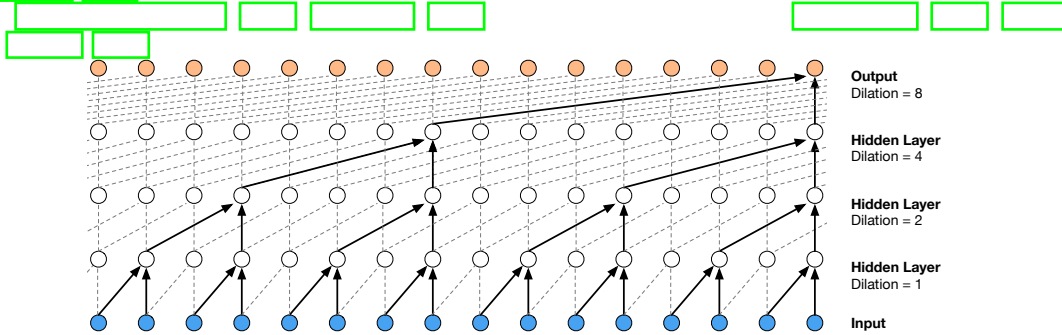


Figure 3: Visualization of a stack of *dilated* causal convolutional layers.

Stacked dilated convolutions enable networks to have very large receptive fields with just a few layers, while preserving the input resolution throughout the network as well as computational efficiency. In this paper, the dilation is doubled for every layer up to a limit and then repeated: e.g.

$$1, 2, 4, \dots, 512, 1, 2, 4, \dots, 512, 1, 2, 4, \dots, 512.$$

The intuition behind this configuration is two-fold. First, exponentially increasing the dilation factor results in exponential receptive field growth with depth (Yu & Koltun, 2016). For example each $1, 2, 4, \dots, 512$ block has receptive field of size 1024, and can be seen as a more efficient and discriminative (non-linear) counterpart of a 1×1024 convolution. Second, stacking these blocks further increases the model capacity and the receptive field size.

2.2 SOFTMAX DISTRIBUTIONS

One approach to modeling the conditional distributions $p(x_t | x_1, \dots, x_{t-1})$ over the individual audio samples would be to use a mixture model such as a mixture density network (Bishop, 1994) or mixture of conditional Gaussian scale mixtures (MCGSM) (Theis & Bethge, 2015). However, van den Oord et al. (2016a) showed that a softmax distribution tends to work better, even when the data is implicitly continuous (as is the case for image pixel intensities or audio sample values). One of the reasons is that a categorical distribution is more flexible and can more easily model arbitrary distributions because it makes no assumptions about their shape.

Because raw audio is typically stored as a sequence of 16-bit integer values (one per timestep), a softmax layer would need to output 65,536 probabilities per timestep to model all possible values. To make this more tractable, we first apply a μ -law companding transformation (ITU-T, 1988) to the data, and then quantize it to 256 possible values:

$$f(x_t) = \text{sign}(x_t) \frac{\ln(1 + \mu |x_t|)}{\ln(1 + \mu)},$$

where $-1 < x_t < 1$ and $\mu = 255$. This non-linear quantization produces a significantly better reconstruction than a simple linear quantization scheme. Especially for speech, we found that the reconstructed signal after quantization sounded very similar to the original.

2.3 GATED ACTIVATION UNITS

We use the same gated activation unit as used in the gated PixelCNN (van den Oord et al., 2016b):

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x}) \odot \sigma(W_{g,k} * \mathbf{x}), \quad (2)$$

where $*$ denotes a convolution operator, $\odot$ denotes an element-wise multiplication operator, $\sigma(\cdot)$ is a sigmoid function, k is the layer index, f and g denote filter and gate, respectively, and W is a learnable convolution filter. In our initial experiments, we observed that this non-linearity worked significantly better than the rectified linear activation function (Nair & Hinton, 2010) for modeling audio signals.

2.4 RESIDUAL AND SKIP CONNECTIONS

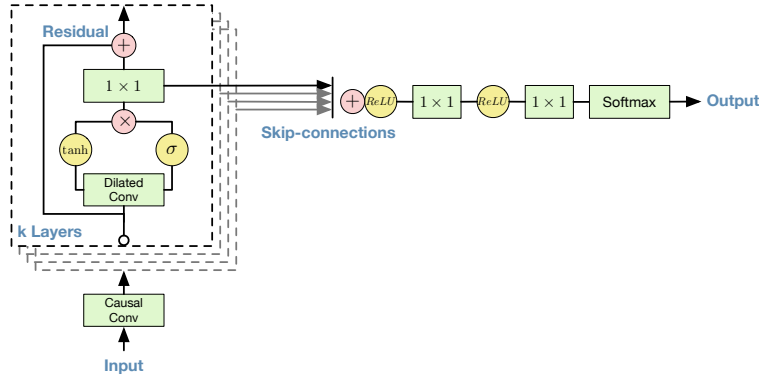


Figure 4: Overview of the residual block and the entire architecture.

Both residual (He et al., 2015) and parameterised skip connections are used throughout the network, to speed up convergence and enable training of much deeper models. In Fig. 4 we show a residual block of our model, which is stacked many times in the network.

2.5 CONDITIONAL WAVENETS

Given an additional input $\mathbf{h}$, WaveNets can model the conditional distribution $p(\mathbf{x} | \mathbf{h})$ of the audio given this input. Eq. (1) now becomes

$$p(\mathbf{x} | \mathbf{h}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}, \mathbf{h}). \quad (3)$$

By conditioning the model on other input variables, we can guide WaveNet’s generation to produce audio with the required characteristics. For example, in a multi-speaker setting we can choose the speaker by feeding the speaker identity to the model as an extra input. Similarly, for TTS we need to feed information about the text as an extra input.

We condition the model on other inputs in two different ways: global conditioning and local conditioning. Global conditioning is characterised by a single latent representation $\mathbf{h}$ that influences the output distribution across all timesteps, e.g. a speaker embedding in a TTS model. The activation function from Eq. (2) now becomes:

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x} + V_{f,k}^T \mathbf{h}) \odot \sigma(W_{g,k} * \mathbf{x} + V_{g,k}^T \mathbf{h}).$$

where $V_{*,k}$ is a learnable linear projection, and the vector $V_{*,k}^T \mathbf{h}$ is broadcast over the time dimension.

For local conditioning we have a second timeseries h_t , possibly with a lower sampling frequency than the audio signal, e.g. linguistic features in a TTS model. We first transform this time series using a transposed convolutional network (learned upsampling) that maps it to a new time series $\mathbf{y} = f(\mathbf{h})$ with the same resolution as the audio signal, which is then used in the activation unit as follows:

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x} + V_{f,k} * \mathbf{y}) \odot \sigma(W_{g,k} * \mathbf{x} + V_{g,k} * \mathbf{y}),$$

where $V_{f,k} * \mathbf{y}$ is now a 1×1 convolution. As an alternative to the transposed convolutional network, it is also possible to use $V_{f,k} * \mathbf{h}$ and repeat these values across time. We saw that this worked slightly worse in our experiments.

2.6 CONTEXT STACKS

We have already mentioned several different ways to increase the receptive field size of a WaveNet: increasing the number of dilation stages, using more layers, larger filters, greater dilation factors, or a combination thereof. A complementary approach is to use a separate, smaller *context* stack that processes a long part of the audio signal and locally conditions a larger WaveNet that processes only a smaller part of the audio signal (cropped at the end). One can use multiple context stacks with varying lengths and numbers of hidden units. Stacks with larger receptive fields have fewer units per layer. Context stacks can also have pooling layers to run at a lower frequency. This keeps the computational requirements at a reasonable level and is consistent with the intuition that less capacity is required to model temporal correlations at longer timescales.

3 EXPERIMENTS

To measure WaveNet’s audio modelling performance, we evaluate it on three different tasks: multi-speaker speech generation (not conditioned on text), TTS, and music audio modelling. We provide samples drawn from WaveNet for these experiments on the accompanying webpage:

<https://www.deepmind.com/blog/wavenet-generative-model-raw-audio/>.

3.1 MULTI-SPEAKER SPEECH GENERATION

For the first experiment we looked at free-form speech generation (not conditioned on text). We used the English multi-speaker corpus from CSTR voice cloning toolkit (VCTK) (Yamagishi, 2012) and conditioned WaveNet only on the speaker. The conditioning was applied by feeding the speaker ID to the model in the form of a one-hot vector. The dataset consisted of 44 hours of data from 109 different speakers.

Because the model is not conditioned on text, it generates non-existent but human language-like words in a smooth way with realistic sounding intonations. This is similar to generative models of language or images, where samples look realistic at first glance, but are clearly unnatural upon closer inspection. The lack of long range coherence is partly due to the limited size of the model’s receptive field (about 300 milliseconds), which means it can only remember the last 2–3 phonemes it produced.

A single WaveNet was able to model speech from any of the speakers by conditioning it on a one-hot encoding of a speaker. This confirms that it is powerful enough to capture the characteristics of all 109 speakers from the dataset in a single model. We observed that adding speakers resulted in better validation set performance compared to training solely on a single speaker. This suggests that WaveNet’s internal representation was shared among multiple speakers.

Finally, we observed that the model also picked up on other characteristics in the audio apart from the voice itself. For instance, it also mimicked the acoustics and recording quality, as well as the breathing and mouth movements of the speakers.

3.2 TEXT-TO-SPEECH

For the second experiment we looked at TTS. We used the same single-speaker speech databases from which Google’s North American English and Mandarin Chinese TTS systems are built. The North American English dataset contains 24.6 hours of speech data, and the Mandarin Chinese dataset contains 34.8 hours; both were spoken by professional female speakers.

WaveNets for the TTS task were locally conditioned on *linguistic features* which were derived from input texts. We also trained WaveNets conditioned on the logarithmic fundamental frequency ($\log F_0$) values in addition to the linguistic features. External models predicting $\log F_0$ values and phone durations from linguistic features were also trained for each language. The receptive field size of the WaveNets was 240 milliseconds. As example-based and model-based speech synthesis baselines, hidden Markov model (HMM)-driven unit selection concatenative (Gonzalvo et al., 2016) and long short-term memory recurrent neural network (LSTM-RNN)-based statistical parametric (Zen et al., 2016) speech synthesizers were built. Since the same datasets and linguistic features were used to train both the baselines and WaveNets, these speech synthesizers could be fairly compared.

To evaluate the performance of WaveNets for the TTS task, subjective paired comparison tests and mean opinion score (MOS) tests were conducted. In the paired comparison tests, after listening to each pair of samples, the subjects were asked to choose which they preferred, though they could choose “neutral” if they did not have any preference. In the MOS tests, after listening to each stimulus, the subjects were asked to rate the naturalness of the stimulus in a five-point Likert scale score (1: Bad, 2: Poor, 3: Fair, 4: Good, 5: Excellent). Please refer to Appendix B for details.

Fig. 5 shows a selection of the subjective paired comparison test results (see Appendix B for the complete table). It can be seen from the results that WaveNet outperformed the baseline statistical parametric and concatenative speech synthesizers in both languages. We found that WaveNet conditioned on linguistic features could synthesize speech samples with natural segmental quality but sometimes it had unnatural prosody by stressing wrong words in a sentence. This could be due to the long-term dependency of F_0 contours: the size of the receptive field of the WaveNet, 240 milliseconds, was not long enough to capture such long-term dependency. WaveNet conditioned on both linguistic features and F_0 values did not have this problem: the external F_0 prediction model runs at a lower frequency (200 Hz) so it can learn long-range dependencies that exist in F_0 contours.

Table 1 show the MOS test results. It can be seen from the table that WaveNets achieved 5-scale MOSs in naturalness above 4.0, which were significantly better than those from the baseline systems. They were the highest ever reported MOS values with these training datasets and test sentences. The gap in the MOSs from the best synthetic speech to the natural ones decreased from 0.69 to 0.34 (51%) in US English and 0.42 to 0.13 (69%) in Mandarin Chinese.

Speech samples	Subjective 5-scale MOS in naturalness	
	North American English	Mandarin Chinese
LSTM-RNN parametric	3.67 ± 0.098	3.79 ± 0.084
HMM-driven concatenative	3.86 ± 0.137	3.47 ± 0.108
WaveNet (L+F)	4.21 ± 0.081	4.08 ± 0.085
Natural (8-bit μ -law)	4.46 ± 0.067	4.25 ± 0.082
Natural (16-bit linear PCM)	4.55 ± 0.075	4.21 ± 0.071

Table 1: Subjective 5-scale mean opinion scores of speech samples from LSTM-RNN-based statistical parametric, HMM-driven unit selection concatenative, and proposed WaveNet-based speech synthesizers, 8-bit μ -law encoded natural speech, and 16-bit linear pulse-code modulation (PCM) natural speech. WaveNet improved the previous state of the art significantly, reducing the gap between natural speech and best previous model by more than 50%.

3.3 MUSIC

For our third set of experiments we trained WaveNets to model two music datasets:

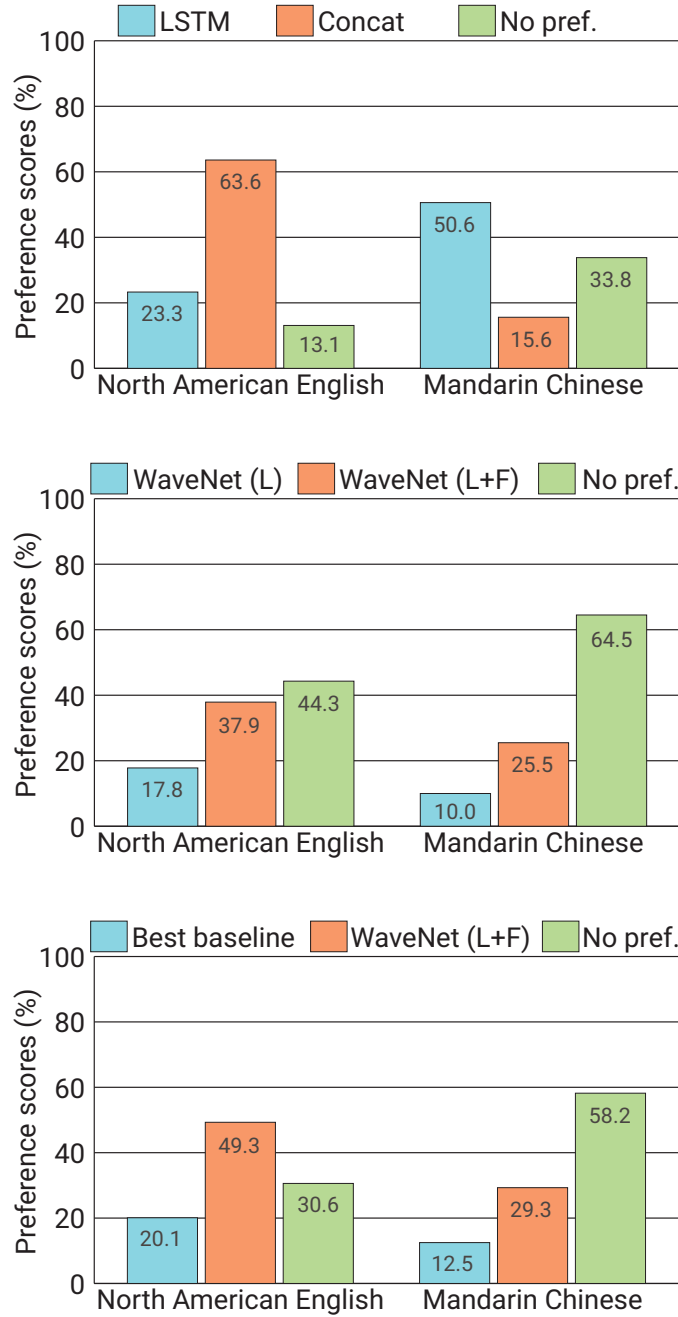


Figure 5: Subjective preference scores (%) of speech samples between (top) two baselines, (middle) two WaveNets, and (bottom) the best baseline and WaveNet. Note that LSTM and Concat correspond to LSTM-RNN-based statistical parametric and HMM-driven unit selection concatenative baseline synthesizers, and WaveNet (L) and WaveNet (L+F) correspond to the WaveNet conditioned on linguistic features only and that conditioned on both linguistic features and $\log F_0$ values.

- the MagnaTagATune dataset (Law & Von Ahn, 2009), which consists of about 200 hours of music audio. Each 29-second clip is annotated with tags from a set of 188, which describe the genre, instrumentation, tempo, volume and mood of the music.
- the YouTube piano dataset, which consists of about 60 hours of solo piano music obtained from YouTube videos. Because it is constrained to a single instrument, it is considerably easier to model.

Although it is difficult to quantitatively evaluate these models, a subjective evaluation is possible by listening to the samples they produce. We found that enlarging the receptive field was crucial to obtain samples that sounded musical. Even with a receptive field of several seconds, the models did not enforce long-range consistency which resulted in second-to-second variations in genre, instrumentation, volume and sound quality. Nevertheless, the samples were often harmonic and aesthetically pleasing, even when produced by unconditional models.

Of particular interest are conditional music models, which can generate music given a set of tags specifying e.g. genre or instruments. Similarly to conditional speech models, we insert biases that depend on a binary vector representation of the tags associated with each training clip. This makes it possible to control various aspects of the output of the model when sampling, by feeding in a binary vector that encodes the desired properties of the samples. We have trained such models on the MagnaTagATune dataset; although the tag data bundled with the dataset was relatively noisy and had many omissions, after cleaning it up by merging similar tags and removing those with too few associated clips, we found this approach to work reasonably well.

3.4 SPEECH RECOGNITION

Although WaveNet was designed as a generative model, it can straightforwardly be adapted to discriminative audio tasks such as speech recognition.

Traditionally, speech recognition research has largely focused on using log mel-filterbank energies or mel-frequency cepstral coefficients (MFCCs), but has been moving to raw audio recently (Palaz et al., 2013; Tüske et al., 2014; Hoshen et al., 2015; Sainath et al., 2015). Recurrent neural networks such as LSTM-RNNs (Hochreiter & Schmidhuber, 1997) have been a key component in these new speech classification pipelines, because they allow for building models with long range contexts. With WaveNets we have shown that layers of dilated convolutions allow the receptive field to grow longer in a much cheaper way than using LSTM units.

As a last experiment we looked at speech recognition with WaveNets on the TIMIT (Garofolo et al., 1993) dataset. For this task we added a mean-pooling layer after the dilated convolutions that aggregated the activations to coarser frames spanning 10 milliseconds ($160\times$ downsampling). The pooling layer was followed by a few non-causal convolutions. We trained WaveNet with two loss terms, one to predict the next sample and one to classify the frame, the model generalized better than with a single loss and achieved 18.8 PER on the test set, which is to our knowledge the best score obtained from a model trained directly on raw audio on TIMIT.

4 CONCLUSION

This paper has presented WaveNet, a deep generative model of audio data that operates directly at the waveform level. WaveNets are autoregressive and combine causal filters with dilated convolutions to allow their receptive fields to grow exponentially with depth, which is important to model the long-range temporal dependencies in audio signals. We have shown how WaveNets can be conditioned on other inputs in a global (e.g. speaker identity) or local way (e.g. linguistic features). When applied to TTS, WaveNets produced samples that outperform the current best TTS systems in subjective naturalness. Finally, WaveNets showed very promising results when applied to music audio modeling and speech recognition.

ACKNOWLEDGEMENTS

The authors would like to thank Lasse Espeholt, Jeffrey De Fauw and Grzegorz Swirszcz for their inputs, Adam Cain, Max Cant and Adrian Bolton for their help with artwork, Helen King, Steven

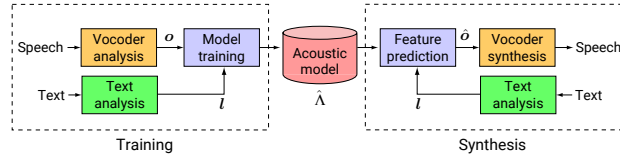


Figure 6: Outline of statistical parametric speech synthesis.

Zen, Heiga, Tokuda, Keiichi, and Kitamura, Tadashi. Reformulating the HMM as a trajectory model by imposing explicit relationships between static and dynamic features. *Comput. Speech Lang.*, 21(1):153–173, 2007.

Zen, Heiga, Tokuda, Keiichi, and Black, Alan W. Statistical parametric speech synthesis. *Speech Commn.*, 51(11):1039–1064, 2009.

Zen, Heiga, Senior, Andrew, and Schuster, Mike. Statistical parametric speech synthesis using deep neural networks. In *Proc. ICASSP*, pp. 7962–7966, 2013.

Zen, Heiga, Agiomyrghiannakis, Yannis, Egberts, Niels, Henderson, Fergus, and Szczepaniak, Przemysław. Fast, compact, and high quality LSTM-RNN based statistical parametric speech synthesizers for mobile devices. In *Interspeech*, 2016. URL <https://arxiv.org/abs/1606.06061>.

A TEXT-TO-SPEECH BACKGROUND

The goal of TTS synthesis is to render naturally sounding speech signals given a text to be synthesized. Human speech production process first translates a text (or concept) into movements of muscles associated with articulators and speech production-related organs. Then using air-flow from lung, vocal source excitation signals, which contain both periodic (by vocal cord vibration) and aperiodic (by turbulent noise) components, are generated. By filtering the vocal source excitation signals by time-varying vocal tract transfer functions controlled by the articulators, their frequency characteristics are modulated. Finally, the generated speech signals are emitted. The aim of TTS is to mimic this process by computers in some way.

TTS can be viewed as a sequence-to-sequence mapping problem; from a sequence of discrete symbols (text) to a real-valued time series (speech signals). A typical TTS pipeline has two parts; 1) text analysis and 2) speech synthesis. The text analysis part typically includes a number of natural language processing (NLP) steps, such as sentence segmentation, word segmentation, text normalization, part-of-speech (POS) tagging, and grapheme-to-phoneme (G2P) conversion. It takes a word sequence as input and outputs a phoneme sequence with a variety of linguistic contexts. The speech synthesis part takes the context-dependent phoneme sequence as its input and outputs a synthesized speech waveform. This part typically includes prosody prediction and speech waveform generation.

There are two main approaches to realize the speech synthesis part; non-parametric, example-based approach known as concatenative speech synthesis (Moulines & Charpentier, 1990; Sagisaka et al., 1992; Hunt & Black, 1996), and parametric, model-based approach known as statistical parametric speech synthesis (Yoshimura, 2002; Zen et al., 2009). The concatenative approach builds up the utterance from units of recorded speech, whereas the statistical parametric approach uses a generative model to synthesize the speech. The statistical parametric approach first extracts a sequence of vocoder parameters (Dudley, 1939) $\mathbf{o} = \{\mathbf{o}_1, \dots, \mathbf{o}_N\}$ from speech signals $\mathbf{x} = \{x_1, \dots, x_T\}$ and linguistic features $\mathbf{l}$ from the text W , where N and T correspond to the numbers of vocoder parameter vectors and speech signals. Typically a vocoder parameter vector $\mathbf{o}_n$ is extracted at every 5 milliseconds. It often includes cepstra (Imai & Furuichi, 1988) or line spectral pairs (Itakura, 1975), which represent vocal tract transfer function, and fundamental frequency (F_0) and aperiodicity (Kawahara et al., 2001), which represent characteristics of vocal source excitation signals. Then a set of generative models, such as hidden Markov models (HMMs) (Yoshimura, 2002), feed-forward neural networks (Zen et al., 2013), and recurrent neural networks (Tuerk & Robinson, 1993; Karaali et al., 1997; Fan et al., 2014), is trained from the extracted vocoder parameters and linguistic features

as

$$\hat{\Lambda} = \arg \max_{\Lambda} p(\mathbf{o} | \mathbf{l}, \Lambda), \quad (4)$$

where Λ denotes the set of parameters of the generative model. At the synthesis stage, the most probable vocoder parameters are generated given linguistic features extracted from a text to be synthesized as

$$\hat{\mathbf{o}} = \arg \max_{\mathbf{o}} p(\mathbf{o} | \mathbf{l}, \hat{\Lambda}). \quad (5)$$

Then a speech waveform is reconstructed from $\hat{\mathbf{o}}$ using a vocoder. The statistical parametric approach offers various advantages over the concatenative one such as small footprint and flexibility to change its voice characteristics. However, its subjective naturalness is often significantly worse than that of the concatenative approach; synthesized speech often sounds muffled and has artifacts. Zen et al. (2009) reported three major factors that can degrade the subjective naturalness; quality of vocoders, accuracy of generative models, and effect of oversmoothing. The first factor causes the artifacts and the second and third factors lead to the muffleness in the synthesized speech. There have been a number of attempts to address these issues individually, such as developing high-quality vocoders (Kawahara et al., 1999; Agiomyrgiannakis, 2015; Morise et al., 2016), improving the accuracy of generative models (Zen et al., 2007, 2013; Fan et al., 2014; Uria et al., 2015), and compensating the oversmoothing effect (Toda & Tokuda, 2007; Takamichi et al., 2016). Zen et al. (2016) showed that state-of-the-art statistical parametric speech synthesizers matched state-of-the-art concatenative ones in some languages. However, its vocoded sound quality is still a major issue.

Extracting vocoder parameters can be viewed as estimation of a generative model parameters given speech signals (Itakura & Saito, 1970; Imai & Furuichi, 1988). For example, linear predictive analysis (Itakura & Saito, 1970), which has been used in speech coding, assumes that the generative model of speech signals is a linear auto-regressive (AR) zero-mean Gaussian process;

$$x_t = \sum_{p=1}^P a_p x_{t-p} + \epsilon_t \quad (6)$$

$$\epsilon_t \sim \mathcal{N}(0, G^2) \quad (7)$$

where a_p is a p -th order linear predictive coefficient (LPC) and G^2 is a variance of modeling error. These parameters are estimated based on the maximum likelihood (ML) criterion. In this sense, the training part of the statistical parametric approach can be viewed as a two-step optimization and sub-optimal: extract vocoder parameters by fitting a generative model of speech signals then model trajectories of the extracted vocoder parameters by a separate generative model for time series (Tokuda, 2011). There have been attempts to integrate these two steps into a single one (Toda & Tokuda, 2008; Wu & Tokuda, 2008; Maia et al., 2010; Nakamura et al., 2014; Muthukumar & Black, 2014; Tokuda & Zen, 2015, 2016; Takaki & Yamagishi, 2016). For example, Tokuda & Zen (2016) integrated non-stationary, nonzero-mean Gaussian process generative model of speech signals and LSTM-RNN-based sequence generative model to a single one and jointly optimized them by back-propagation. Although they showed that this model could approximate natural speech signals, its segmental naturalness was significantly worse than the non-integrated model due to over-generalization and over-estimation of noise components in speech signals.

The conventional generative models of raw audio signals have a number of assumptions which are inspired from the speech production, such as

- Use of fixed-length analysis window; They are typically based on a stationary stochastic process (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Kameoka et al., 2010). To model time-varying speech signals by a stationary stochastic process, parameters of these generative models are estimated within a fixed-length, overlapping and shifting analysis window (typically its length is 20 to 30 milliseconds, and shift is 5 to 10 milliseconds). However, some phones such as stops are time-limited by less than 20 milliseconds (Rabiner & Juang, 1993). Therefore, using such fixed-size analysis window has limitations.
- Linear filter; These generative models are typically realized as a linear time-invariant filter (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Kameoka et al., 2010) within a windowed frame. However, the relationship between successive audio samples can be highly non-linear.

- Gaussian process assumption; The conventional generative models are based on Gaussian process (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Kameoka et al., 2010; Tokuda & Zen, 2015, 2016). From the source-filter model of speech production (Chiba & Kajiyama, 1942; Fant, 1970) point of view, this is equivalent to assuming that a vocal source excitation signal is a sample from a Gaussian distribution (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Tokuda & Zen, 2015; Kameoka et al., 2010; Tokuda & Zen, 2016). Together with the linear assumption above, it results in assuming that speech signals are normally distributed. However, distributions of real speech signals can be significantly different from Gaussian.

Although these assumptions are convenient, samples from these generative models tend to be noisy and lose important details to make these audio signals sounding natural.

WaveNet, which was described in Section 2, has none of the above-mentioned assumptions. It incorporates almost no prior knowledge about audio signals, except the choice of the receptive field and μ -law encoding of the signal. It can also be viewed as a non-linear causal filter for quantized signals. Although such non-linear filter can represent complicated signals while preserving the details, designing such filters is usually difficult (Peltonen et al., 2001). WaveNets give a way to train them from data.

B DETAILS OF TTS EXPERIMENT

The HMM-driven unit selection and WaveNet TTS systems were built from speech at 16 kHz sampling. Although LSTM-RNNs were trained from speech at 22.05 kHz sampling, speech at 16 kHz sampling was synthesized at runtime using a resampling functionality in the Vocaine vocoder (Agiomyriannakis, 2015). Both the LSTM-RNN-based statistical parametric and HMM-driven unit selection speech synthesizers were built from the speech datasets in the 16-bit linear PCM, whereas the WaveNet-based ones were trained from the same speech datasets in the 8-bit μ -law encoding.

The linguistic features include phone, syllable, word, phrase, and utterance-level features (Zen, 2006) (e.g. phone identities, syllable stress, the number of syllables in a word, and position of the current syllable in a phrase) with additional frame position and phone duration features (Zen et al., 2013). These features were derived and associated with speech every 5 milliseconds by phone-level forced alignment at the training stage. We used LSTM-RNN-based phone duration and autoregressive GNN-based $\log F_0$ prediction models. They were trained so as to minimize the mean squared errors (MSE). It is important to note that no post-processing was applied to the audio signals generated from the WaveNets.

The subjective listening tests were blind and crowdsourced. 100 sentences not included in the training data were used for evaluation. Each subject could evaluate up to 8 and 63 stimuli for North American English and Mandarin Chinese, respectively. Test stimuli were randomly chosen and presented for each subject. In the paired comparison test, each pair of speech samples was the same text synthesized by the different models. In the MOS test, each stimulus was presented to subjects in isolation. Each pair was evaluated by eight subjects in the paired comparison test, and each stimulus was evaluated by eight subjects in the MOS test. The subjects were paid and native speakers performing the task. Those ratings (about 40%) where headphones were not used were excluded when computing the preference and mean opinion scores. Table 2 shows the full details of the paired comparison test shown in Fig. 5.

Language	Subjective preference (%) in naturalness					p value
	LSTM	Concat	WaveNet (L)	WaveNet (L+F)	No preference	
North American English	23.3	63.6			13.1	$\ll 10^{-9}$
	18.7		69.3		12.0	$\ll 10^{-9}$
	7.6			82.0	10.4	$\ll 10^{-9}$
		32.4	41.2		26.4	0.003
		20.1		49.3	30.6	$\ll 10^{-9}$
			17.8	37.9	44.3	$\ll 10^{-9}$
Mandarin Chinese	50.6	15.6			33.8	$\ll 10^{-9}$
	25.0		23.3		51.8	0.476
	12.5			29.3	58.2	$\ll 10^{-9}$
		17.6	43.1		39.3	$\ll 10^{-9}$
		7.6		55.9	36.5	$\ll 10^{-9}$
			10.0	25.5	64.5	$\ll 10^{-9}$

Table 2: Subjective preference scores of speech samples between LSTM-RNN-based statistical parametric (**LSTM**), HMM-driven unit selection concatenative (**Concat**), and proposed WaveNet-based speech synthesizers. Each row of the table denotes scores of a paired comparison test between two synthesizers. Scores of the synthesizers which were significantly better than their competing ones at $p < 0.01$ level were shown in the bold type. Note that **WaveNet (L)** and **WaveNet (L+F)** correspond to WaveNet conditioned on linguistic features only and that conditioned on both linguistic features and F_0 values.

RotLSTM: rotating memories in Recurrent Neural Networks

Vlad Velici

School of Electronics and Computer Science
University of Southampton
Southampton, UK

Adam Prügel-Bennett

School of Electronics and Computer Science
University of Southampton
Southampton, UK

Abstract

Long Short-Term Memory (LSTM) units have the ability to memorise and use long-term dependencies between inputs to generate predictions on time series data. We introduce the concept of modifying the cell state (memory) of LSTMs using rotation matrices parametrised by a new set of trainable weights. This addition shows significant increases of performance on some of the tasks from the bAbI dataset.

1 Introduction

In the recent years, Recurrent Neural Networks (RNNs) have been successfully used to tackle problems with data that can be represented in the shape of time series. Application domains include Natural Language Processing (NLP) (translation (Rosca & Breuel, 2016), summarisation (Nallapati et al., 2016), question answering and more), speech recognition (Hannun et al., 2014; Graves et al., 2013), text to speech systems (Arik et al., 2017), computer vision tasks (Stewart et al., 2016; Wu et al., 2017), and differentiable programming language interpreters (Riedel et al., 2016; Rocktäschel & Riedel, 2017).

An intuitive explanation for the success of RNNs in fields such as natural language understanding is that they allow words at the beginning of a sentence or paragraph to be memorised. This can be crucial to understanding the semantic content. Thus in the phrase "*The cat ate the fish*" it is important to memorise the subject (cat). However, often later words can change the meaning of a sentence in subtle ways. For example, "*The cat ate the fish, **didn't it***" changes a simple statement into a question. In this paper, we study a mechanism to enhance a standard RNN to enable it to modify its memory, with the hope that this will allow it to capture in the memory cells sequence information using a shorter and more robust representation.

One of the most used RNN units is the Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuber, 1997). The core of the LSTM is that each unit has a *cell state* that is modified in a gated fashion at every time step. At a high level, the cell state has the role of providing the neural network with *memory* to hold long-term relationships between inputs. There are many small variations of LSTM units in the literature and most of them yield similar performance (Greff et al., 2017).

The memory (cell state) is expected to encode information necessary to make the next prediction. Currently the ability of the LSTMs to rotate and swap memory positions is limited to what can be achieved using the available gates. In this work we introduce a new operation on the memory that explicitly enables rotations and swaps of pairwise memory elements. Our preliminary tests show performance improvements on some of the bAbI tasks (Weston et al., 2015) compared with LSTM based architectures.

2 Related work

Adding rotations to RNN cells has been previously studied in [Henaff et al. \(2016\)](#). They do compare their results to LSTM models but do not directly add a new rotation gates to the LSTM like in our work. Their results suggest that rotations do not generalise well for multiple tasks.

Using unitary constraints for weight matrices in RNN units is shown in [Arjovsky et al. \(2015\)](#), where they use the weights in the complex domain (they show a way to only use real numbers in implementation). The EURNN [\(Jing et al., 2017b\)](#) presents a more efficient method of parametrising a weights matrix such that it is unitary and uses this parametrisation to add unitary constraints to weights of RNN cells. A similar work is GORU [\(Jing et al., 2017a\)](#) which adds the same unitary constraints on one of the gate weights in the GRU cell.

A new RNN cell, the Rotational Unit of Memory (RUM) [\(Dangovski et al., 2017\)](#), adds a rotation operation in a gated RNN cell. The rotation parametrisation is passed through to the next cell, similar to how the memory state of an LSTM is. They show increased performance on the bAbI tasks when compared to LSTM and GORU based models.

3 The rotation gate

In this section we introduce the idea of adding a new set of parameters for the RNN cell that enable rotation of the cell state. The following subsection shows how this is implemented in the LSTM unit.

One of the key innovations of LSTMs was the introduction of gated modified states so that if the gate neuron i is saturated then the memory $c_i(t-1)$ would be unaltered. That is, $c_i(t-1) \approx c_i(t)$ with high accuracy. The fact that the amplification factor is very close to 1 prevents the memory vanishing or exploding over many epochs.

To modify the memory, but retain an amplification factor of 1 we take the output after applying the forget and add gates (we call it $\mathbf{d}_t$), and apply a rotation matrix $\mathbf{U}$ to obtain a modified memory $\mathbf{c}_t = \mathbf{U}\mathbf{d}_t$. Note that, for a rotation matrix $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ so that $\|\mathbf{d}_t\| = \|\mathbf{c}_t\|$.

We parametrise the rotation by a vector of angles

$$\mathbf{u} = 2\pi\sigma(\mathbf{W}_{\text{rot}}\mathbf{x} + \mathbf{b}_{\text{rot}}), \quad (1)$$

where $\mathbf{W}_{\text{rot}}$ is a weight matrix and $\mathbf{b}_{\text{rot}}$ is a bias vector which we learn along with the other parameters. $\mathbf{x}$ is the vector of our concatenated inputs (in LSTMs given by concatenating the input for the current timestep with the output from the previous time step).

A full rotation matrix is parametrisable by $n(n-1)/2$ parameters (angles). Using all of these would introduce a huge number of weights, which is likely to over-fit. Instead, we have limited ourselves to considering rotations between pairs of inputs $d_i(t)$ and $d_{i+1}(t)$.

Our rotation matrix is a block-diagonal matrix of 2D rotations

$$\mathbf{U}(\mathbf{u}) = \begin{bmatrix} \cos u_1 & -\sin u_1 & & & \\ \sin u_1 & \cos u_1 & & & \\ & & \ddots & & \\ & & & \cos u_{n/2} & -\sin u_{n/2} \\ & & & \sin u_{n/2} & \cos u_{n/2} \end{bmatrix}, \quad (2)$$

where the cell state is of size n . Our choice of rotations only needs $n/2$ angles.

3.1 RotLSTM

In this section we show how to add memory rotation to the LSTM unit. The rotation is applied after the forget and add gates and before using the current cell state to produce an output.

The RotLSTM equations are as follows:

$$\mathbf{x} = [\mathbf{h}_{t-1}, \mathbf{x}_t], \quad (3)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x} + \mathbf{b}_f), \quad (4)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x} + \mathbf{b}_i), \quad (5)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x} + \mathbf{b}_o), \quad (6)$$

$$\mathbf{u}_t = 2\pi\sigma(\mathbf{W}_{\text{rot}} \mathbf{x} + \mathbf{b}_{\text{rot}}), \quad (7)$$

$$\mathbf{d}_t = \mathbf{f}_t \circ \mathbf{c}_{t-1} + \mathbf{i}_t \circ \tanh(\mathbf{W}_c \mathbf{x} + \mathbf{b}_c), \quad (8)$$

$$\mathbf{c}_t = \mathbf{U}(\mathbf{u}_t) \mathbf{d}_t, \quad (9)$$

$$\mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{c}_t), \quad (10)$$

where $\mathbf{W}_{\{f,i,o,\text{rot},c\}}$ are weight matrices, $\mathbf{b}_{\{f,i,o,\text{rot},c\}}$ are biases ($\mathbf{W}$ s and $\mathbf{b}$ s learned during training), $\mathbf{h}_{t-1}$ is the previous cell output, $\mathbf{h}_t$ is the output the cell produces for the current timestep, similarly $\mathbf{c}_{t-1}$ and $\mathbf{c}_t$ are the cell states for the previous and current timestep, $\circ$ is element-wise multiplication and $[\cdot, \cdot]$ is concatenation. $\mathbf{U}$ as defined in Equation 2 parametrised by $\mathbf{u}_t$. Figure 1 shows a RotLSTM unit in detail.

Assuming cell state size n , input size m , the RotLSTM has $n(n+m)/2$ extra parameters, a 12.5% increase (ignoring biases). Our expectation is that we can decrease n without harming performance and the rotations will enforce a better representation for the cell state.

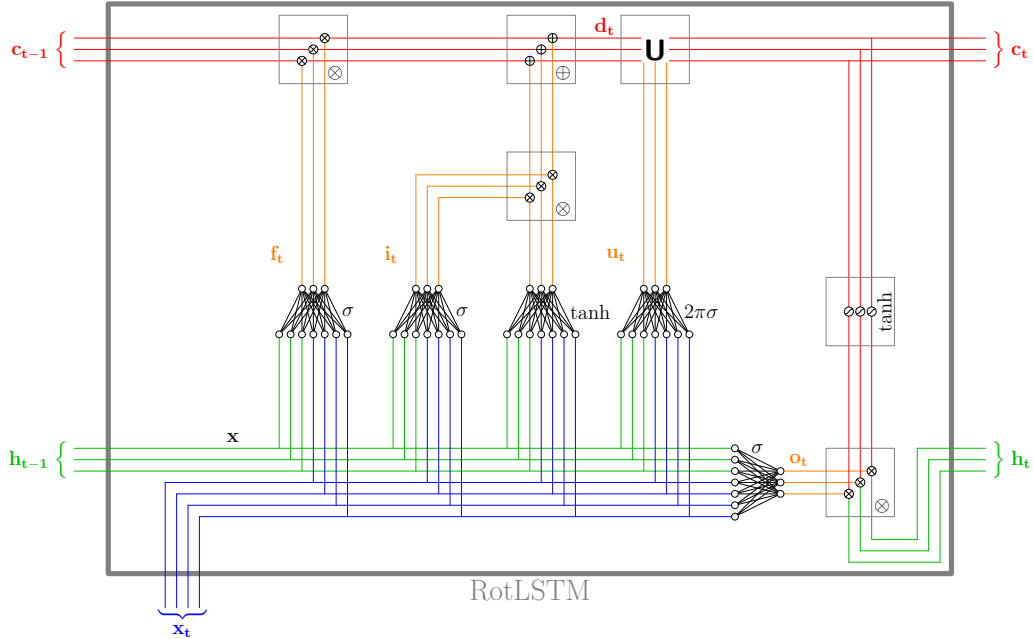


Figure 1: RotLSTM diagram. $\mathbf{x}$ is the concatenation of $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ in the diagram (green and blue lines). Note that this differs from a regular LSTM by the introduction of the network producing angles $\mathbf{u}_t$ and the rotation module marked $\mathbf{U}$. In the diagram input size is 4 and cell state size is 3.

4 Experiments and results

To empirically evaluate the performance of adding the rotation gate to LSTMs we use the toy NLP dataset bAbI with 1000 samples per task. The bAbI dataset is composed of 20 different tasks of various difficulties, starting from easy questions based on a single supporting fact (for example: *John is in the kitchen. Where is John? A: Kitchen*) and going to more difficult tasks of reasoning about size (example: *The football fits in the suitcase. The box is smaller than the football. Will the box fit in the suitcase? A: yes*) and path finding (example: *The bathroom is south of the office. The bathroom is north of the hallway. How do you go from the hallway to the office? A: north, north*). A summary of

Table 1: bAbI dataset tasks.

#	Description	#	Description
1	Factoid QA with one supporting fact	11	Basic coreference
2	Factoid QA with two supporting facts	12	Conjunction
3	Factoid QA with three supporting facts	13	Compound coreference
4	Two argument relations: subject vs. object	14	Time manipulation
5	Three argument relations	15	Basic deduction
6	Yes/No questions	16	Basic induction
7	Counting	17	Positional reasoning
8	Lists/Sets	18	Reasoning about size
9	Simple Negation	19	Path finding
10	Indefinite Knowledge	20	Reasoning about agent’s motivation

all tasks is available in Table 1. We are interested in evaluating the behaviour and performance of rotations on RNN units rather than beating state of the art.

We compare a model based on RotLSTM with the same model based on the traditional LSTM. All models are trained with the same hyperparameters and we do not perform any hyperparameter tuning apart from using the sensible defaults provided in the Keras library and example code (Chollet et al., 2015).

For the first experiment we train a LSTM and RotLSTM based model 10 times using a fixed cell state size of 50. In the second experiment we train the same models but vary the cell state size from 6 to 50 to assess whether the rotations help our models achieve good performance with smaller state sizes. We only choose even numbers for the cell state size to make all units go through rotations.

4.1 The model architecture

The model architecture, illustrated in Figure 2, is based on the Keras example implementation¹. This model architecture, empirically, shows better performance than the LSTM baseline published in Weston et al. (2015). The input question and sentences are passed through a word embedding layer (not shared, embeddings are different for questions and sentences). The question is fed into an RNN which produces a representation of the question. This representation is concatenated to every word vector from the story, which is then used as input to the second RNN. Intuitively, this helps the second RNN (Query) to *focus* on the important words to answer the question. The output of the second RNN is passed to a fully connected layer with a softmax activation of the size of the dictionary. The answer is the word with the highest activation.

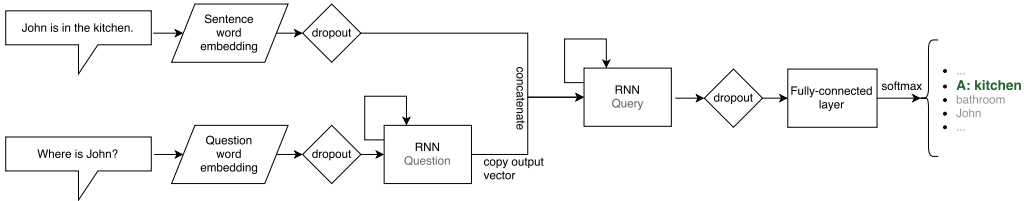


Figure 2: Model architecture for the bAbI dataset. The RNN is either LSTM or RotLSTM.

The categorical cross-entropy loss function was used for training. All dropout layers are dropping 30% of the nodes. The train-validation dataset split used was 95%-5%. The optimizer used was Adam with learning rate 0.001, no decay, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. The training set was randomly shuffled before every epoch. All models were trained for 40 epochs. After every epoch the model performance was evaluated on the validation and training sets, and every 10 epochs on the test set. We set the random seeds to the same number for reproducibility and ran

¹ Available at <https://goo.gl/9wfzr5>.

Table 2: Performance comparison on the bAbI dataset. Values are % average accuracy on test set $\pm$ standard deviation taken from training each model 10 times. For each of the trained models the test accuracy is taken for the epoch with the best validation accuracy (picked from epoch numbers 1, 11, 21, 31, 40 since we only evaluated on the test set for those).

Task:	1	2	3	4	5	6	7	8	9	10
LSTM	49.8 $\pm$ 3.3	26.5 $\pm$ 3.8	21.1 $\pm$ 1.0	63.7 $\pm$ 9.4	33.6 $\pm$ 6.2	49.2 $\pm$ 0.8	62.7 $\pm$ 15.5	68.8 $\pm$ 8.3	63.9 $\pm$ 0.2	45.2 $\pm$ 2.0
RotLSTM	52.3 $\pm$ 1.1	27.1 $\pm$ 1.3	22.4 $\pm$ 1.3	65.8 $\pm$ 3.6	55.7 $\pm$ 5.5	50.1 $\pm$ 1.9	76.7 $\pm$ 3.0	66.1 $\pm$ 6.2	61.5 $\pm$ 1.9	48.1 $\pm$ 1.1
Task:	11	12	13	14	15	16	17	18	19	20
LSTM	73.6 $\pm$ 2.2	74.3 $\pm$ 1.7	94.4 $\pm$ 0.2	20.7 $\pm$ 3.3	21.4 $\pm$ 0.5	48.0 $\pm$ 2.2	48.0 $\pm$ 0.0	70.3 $\pm$ 20.8	8.5 $\pm$ 0.6	87.7 $\pm$ 4.2
RotLSTM	73.9 $\pm$ 1.2	76.5 $\pm$ 1.1	94.4 $\pm$ 0.0	19.9 $\pm$ 2.0	28.8 $\pm$ 8.8	46.7 $\pm$ 2.0	54.2 $\pm$ 3.1	90.5 $\pm$ 0.9	8.9 $\pm$ 1.1	89.9 $\pm$ 2.4

the experiments 10 times with 10 different random seeds. The source code is available at <https://github.com/vladvelici/swaplstm>.

4.2 Results

In this subsection we compare the the performance of models based on the LSTM and RotLSTM units on the bAbI dataset.

Applying rotations on the unit memory of the LSTM cell gives a slight improvement in performance overall, and significant improvements on specific tasks. Results are shown in Table 2. The most significant improvements are faster convergence, as shown in Figure 3, and requiring smaller state sizes, illustrated in Figure 4.

On tasks 1 (basic factoid), 11 (basic coreference), 12 (conjunction) and 13 (compound coreference) the RotLSTM model reaches top performance a couple of epochs before the LSTM model consistently. The RotLSTM model also needs a smaller cell state size, reaching top performance at state size 10 to 20 where the LSTM needs 20 to 30. The top performance is, however, similar for both models, with RotLSTM improving the accuracy with up to 2.5%.

The effect is observed on task 18 (reasoning about size) at a greater magnitude where the RotLSTM reaches top performance before epoch 20, after which it plateaus, while the LSTM model takes 40 epochs to fit the data. The training is more stable for RotLSTM and the final accuracy is improved by 20%. The RotLSTM reaches top performance using cell state 10 and the LSTM needs size 40. Similar performance increase for the RotLSTM (22.1%) is observed in task 5 (three argument relations), reaching top performance around epoch 25 and using a cell state of 50. Task 7 (counting) shows a similar behaviour with an accuracy increase of 14% for RotLSTM.

Tasks 4 (two argument relations) and 20 (agent motivation) show quicker learning (better performance in the early epochs) for the RotLSTM model but both models reach their top performance after the same amount of training. On task 20 the RotLSTM performance reaches top accuracy using state size 10 while the LSTM incrementally improves until using state size 40 to 50.

Signs of overfitting for the RotLSTM model can be observed more prominently than for the LSTM model on tasks 15 (basic deduction) and 17 (positional reasoning).

Our models, both LSTM and RotLSTM, perform poorly on tasks 2 and 3 (factoid questions with 2 and 3 supporting facts, respectively) and 14 (time manipulation). These problem classes are solved very well using models that look over the input data many times and use an attention mechanism that allows the model to focus on the relevant input sentences to answer a question (Sukhbaatar et al., 2015; Kumar et al., 2016). Our models only look at the input data once and we do not filter out irrelevant information.

5 RotGRU

A popular and successful Recurrent Neural Network (RNN) cell is the Gated Recurrent Unit (GRU) unit (Cho et al., 2014). Unlike the LSTM, the GRU unit only has a cell state that can be interpreted as output when needed. Intuitively we want the transformation matrix to enable the unit to learn more compact, richer representations. The rotation is inserted at the *remember* (r) gate to avoid applying a

Table 3: Performance comparison on the bAbI dataset. Values are % average accuracy on test set $\pm$ standard deviation taken from training each model 10 times. For each of the trained models the test accuracy is taken for the epoch with the best validation accuracy (picked from epoch numbers 1, 11, 21, 31, 40 since we only evaluated on the test set for those).

Task:	1	2	3	4	5	6	7	8	9	10
GRU	51.3 $\pm$ 0.9	34.6 $\pm$ 3.8	21.5 $\pm$ 1.4	69.8 $\pm$ 1.9	67.1 $\pm$ 5.7	49.9 $\pm$ 1.0	79.2 $\pm$ 0.5	76.3 $\pm$ 1.2	62.6 $\pm$ 0.7	48.2 $\pm$ 1.1
RotGRU	50.2 $\pm$ 1.3	35.7 $\pm$ 1.9	21.3 $\pm$ 1.1	70.8 $\pm$ 1.2	73.4 $\pm$ 2.6	50.1 $\pm$ 1.1	78.4 $\pm$ 2.6	75.9 $\pm$ 2.0	62.8 $\pm$ 0.4	48.4 $\pm$ 1.0
Task:	11	12	13	14	15	16	17	18	19	20
GRU	77.7 $\pm$ 3.3	76.8 $\pm$ 0.8	94.4 $\pm$ 0.0	31.4 $\pm$ 5.6	27.8 $\pm$ 6.7	48.0 $\pm$ 1.3	48.9 $\pm$ 1.7	91.1 $\pm$ 0.5	8.9 $\pm$ 0.8	92.3 $\pm$ 1.9
RotGRU	79.2 $\pm$ 2.6	76.7 $\pm$ 0.7	94.3 $\pm$ 0.1	29.9 $\pm$ 4.7	23.3 $\pm$ 4.1	47.1 $\pm$ 3.7	48.6 $\pm$ 1.6	90.9 $\pm$ 0.8	8.9 $\pm$ 1.0	93.6 $\pm$ 2.9

transformation directly on the outputs. The RotGRU equations are the following:

$$\mathbf{x} = [\mathbf{h}_{t-1}, \mathbf{x}_t], \quad (11)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x} + \mathbf{b}_z), \quad (12)$$

$$\mathbf{d}_t = \mathbf{h}_{t-1} \circ \sigma(\mathbf{W}_r \mathbf{x} + \mathbf{b}_r), \quad (13)$$

$$\mathbf{U} = 2\pi \sigma(\mathbf{W}_{\text{rot}} \mathbf{x} + \mathbf{b}_{\text{rot}}), \quad (14)$$

$$\mathbf{r}_t = \mathbf{U} \mathbf{d}_t \quad (15)$$

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W}_h [\mathbf{r}_t, \mathbf{x}_t] + \mathbf{b}_h), \quad (16)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \circ \mathbf{h}_{t-1} + \mathbf{z}_t \circ \hat{\mathbf{h}}_t. \quad (17)$$

We show the results of training a RotGRU and a GRU model on the bAbI tasks in Table 3 and Figure 5. Notice that models based on the GRU cell perform better than the LSTM and RotLSTM cells. From this experiment we also observe that RotGRU performs the same or slightly worse than the initial GRU cell. The RotGRU also requires more computation and introduces more parameters, as a result it takes longer (wall clock) time to train and run for inference.

6 Discussion

A limitation of the models in our experiments is only applying pairwise 2D rotations. Representations of past input can be larger groups of the cell state vector, thus 2D rotations might not fully exploit the benefits of transformations. Rotating groups of elements and multi-dimensional rotations was not investigated, but could potentially also force the models to learn a more structured representation of the world, similar to how forcing a model to learn specific representations of scenes, as presented in Higgins et al. (2017), yields semantic representations of the scene.

In this work we presented preliminary tests for adding rotations to simple models but we only used a toy dataset. The bAbI dataset has certain advantages such as being small thus easy to train many models on a single machine, not having noise as it is generated from a simulation, and having a wide range of tasks of various difficulties. However it is a toy dataset that has a very limited vocabulary and lacks the complexity of real world datasets (noise, inconsistencies, larger vocabularies, more complex language constructs, and so on).

A brief exploration of the angles produced by $\mathbf{u}$ and the weight matrix $\mathbf{W}_{\text{rot}}$ show that $\mathbf{u}$ does not saturate, thus rotations are in fact applied to our cell states and do not converge to 0 (or 360 degrees). A more in-depth qualitative analysis of the rotation gate might give a better insight into what the rotations are actually achieving. Peeking into the activations of our rotation gates could help understand the behaviour of rotations and to what extent they help (or do not) better represent long-term memory.

A very successful and popular mutation of the LSTM is the GRU unit (Cho et al., 2014). The GRU only has an output as opposed to both a cell state and an output and uses fewer gates. We showed that adding similar rotations to GRU cells does not show the same performance improvements, and that the baseline GRU model outperforms both LSTM and RotLSTM models in most cases.

7 Conclusion

We have introduced a novel gating mechanism for RNN units that enables applying a parametrised transformation matrix to the cell state. We picked pairwise 2D rotations as the transformation and shown how this can be added to the popular LSTM units to create what we call RotLSTM.

We trained a simple model using RotLSTM units and compared them with the same model based on LSTM units. We show that for the LSTM-based architectures adding rotations has a positive impact on most bAbI tasks, making the training require fewer epochs to achieve similar or higher accuracy. On some tasks the RotLSTM model can use a lower dimensional cell state vector and maintain its performance.

Significant accuracy improvements of approximately 20% for the RotLSTM model over the LSTM model are visible on bAbI tasks 5 (three argument relations) and 18 (reasoning about size).

The same improvements could not be replicated into the GRU cell when adding our rotation gate, and models based on the GRU cell outperform both the LSTM and RotLSTM on most tasks.

References

- Sercan O Arik, Mike Chrzanowski, Adam Coates, Gregory Diamos, Andrew Gibiansky, Yongguo Kang, Xian Li, John Miller, Jonathan Raiman, Shubho Sengupta, et al. Deep voice: Real-time neural text-to-speech. *arXiv preprint arXiv:1702.07825*, 2017.
- Martín Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. *CoRR*, abs/1511.06464, 2015. URL <http://arxiv.org/abs/1511.06464>.
- Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- François Chollet et al. Keras. <https://github.com/fchollet/keras>, 2015.
- Rumen Dangovski, Li Jing, and Marin Soljagic. Rotational unit of memory. *CoRR*, abs/1710.09537, 2017. URL <http://arxiv.org/abs/1710.09537>.
- Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *Acoustics, speech and signal processing (icassp), 2013 ieee international conference on*, pp. 6645–6649. IEEE, 2013.
- Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. Lstm: A search space odyssey. *IEEE transactions on neural networks and learning systems*, 2017.
- Awni Y. Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Sathesh, Shubho Sengupta, Adam Coates, and Andrew Y. Ng. Deep speech: Scaling up end-to-end speech recognition. *CoRR*, abs/1412.5567, 2014. URL <http://arxiv.org/abs/1412.5567>.
- Mikael Henaff, Arthur Szlam, and Yann LeCun. Orthogonal rnns and long-memory tasks. *CoRR*, abs/1602.06662, 2016. URL <http://arxiv.org/abs/1602.06662>.
- Irina Higgins, Nicolas Sonnerat, Loic Matthey, Arka Pal, Christopher P Burgess, Matthew Botvinick, Demis Hassabis, and Alexander Lerchner. Scan: Learning abstract hierarchical compositional visual concepts. *arXiv preprint arXiv:1707.03389*, 2017.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.
- Li Jing, Çağlar Gülçehre, John Peurifoy, Yichen Shen, Max Tegmark, Marin Soljagic, and Yoshua Bengio. Gated orthogonal recurrent units: On learning to forget. *CoRR*, abs/1706.02761, 2017a. URL <http://arxiv.org/abs/1706.02761>.

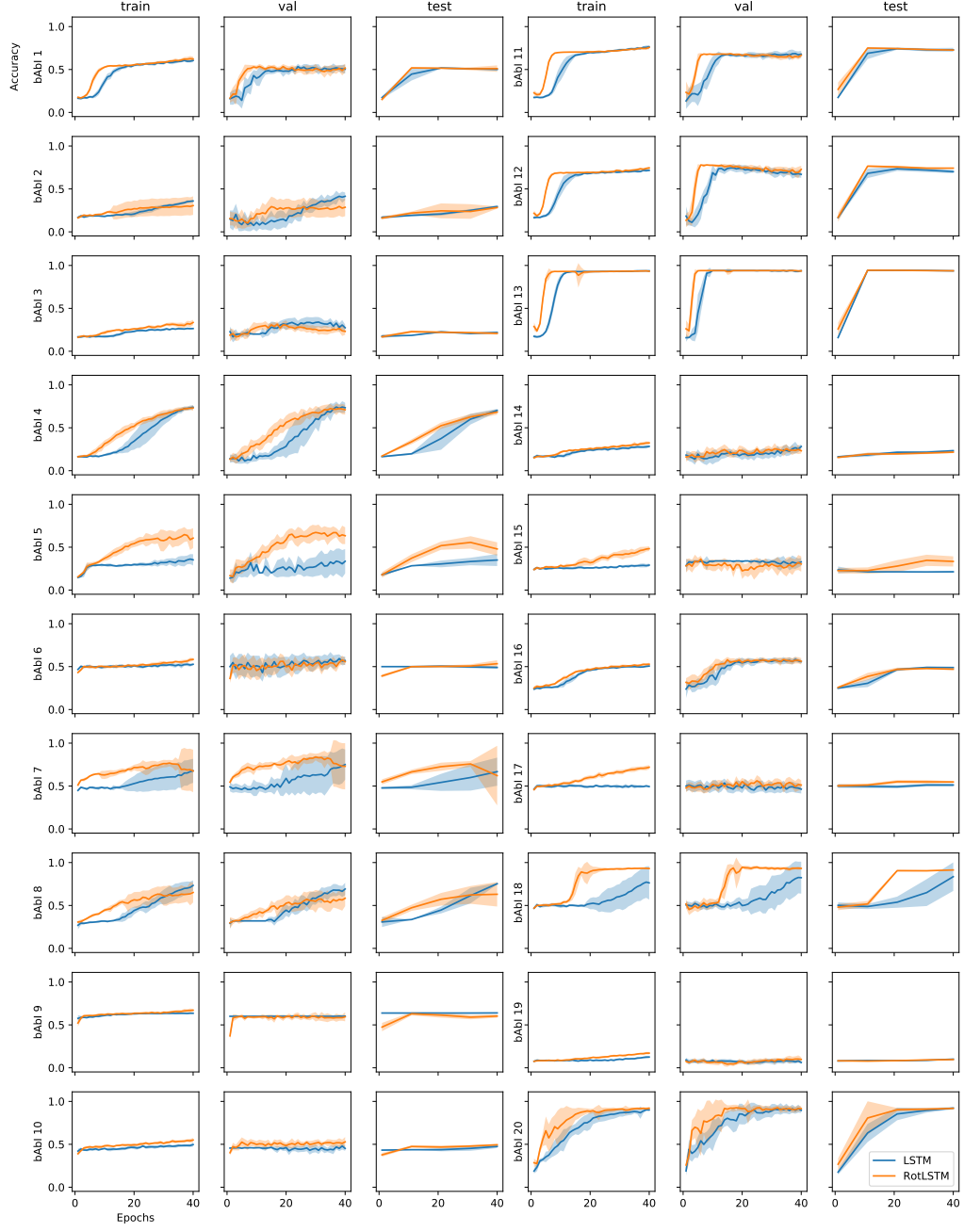


Figure 3: Accuracy comparison on training, validation (val) and test sets over 40 epochs for LSTM and RotLSTM models. The models were trained 10 times and shown is the average accuracy and in faded colour is the standard deviation. Test set accuracy was computed every 10 epochs.

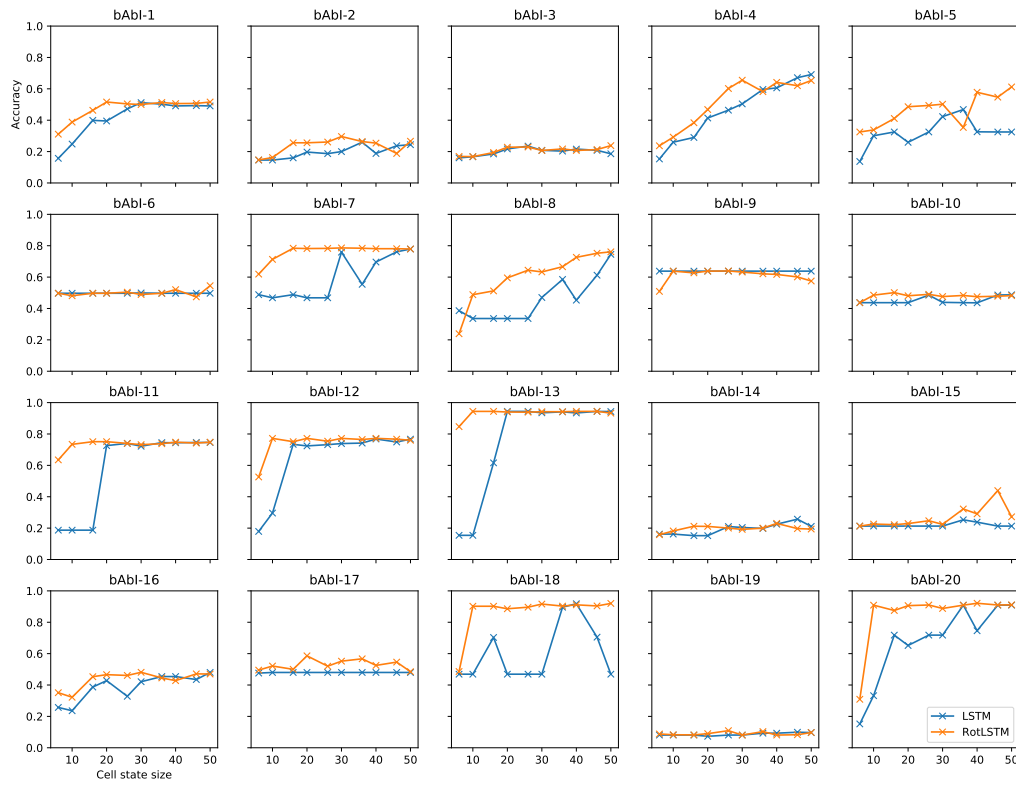


Figure 4: Accuracy on the test set for the LSTM and RotLSTM while varying the cell state size from 6 to 50. The shown numbers are for the epochs with best validation set accuracy.

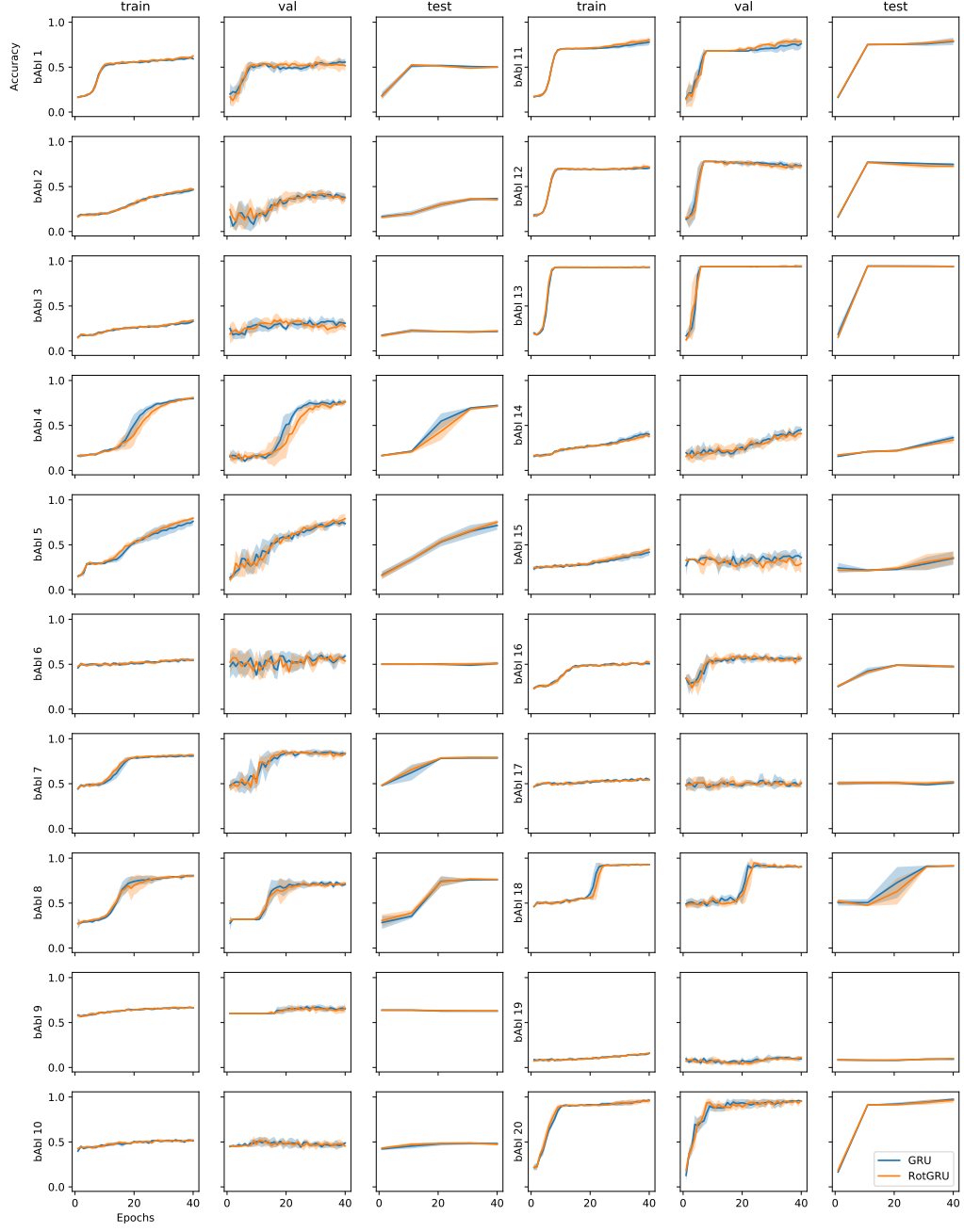


Figure 5: Accuracy comparison on training, validation (val) and test sets over 40 epochs for GRU and RotGRU models. The models were trained 5 times and shown is the average accuracy and in faded colour is the standard deviation. Test set accuracy was computed every 10 epochs.

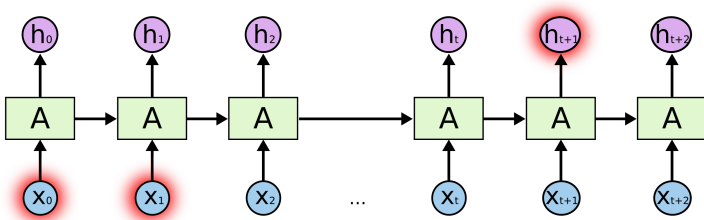
Tunable Efficient Unitary Neural Networks (EUNN) and their application to RNNs

Li Jing*, Yichen Shen*, Tena Dubček, John Peurifoy, Scott Skirlo,
Yann LeCun, Max Tegmark, Marin Soljačić

** equal contribution*

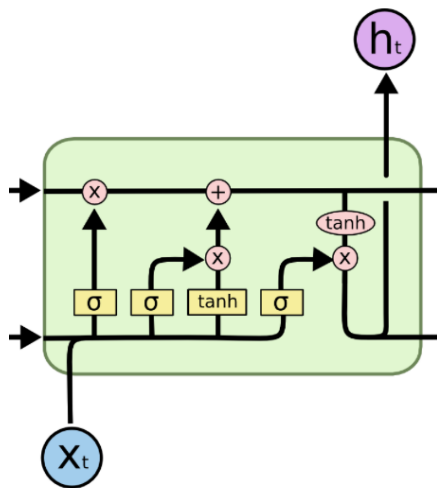


Gradient Vanishing/Explosion Problem



- During backpropagation through time, hidden to hidden Jacobin matrix is multiplied multiple times.
- Gradient vanishing/explosion makes RNN hard to train

Conventional Solution: LSTM



- Practically, gradient clipping is required
- slow to learn long term dependency

New Solution: Unitary/Orthogonal RNN

Unitary/Orthogonal matrices keep the norm of vectors: $\|U\mathbf{x}\| = \|\mathbf{x}\|$

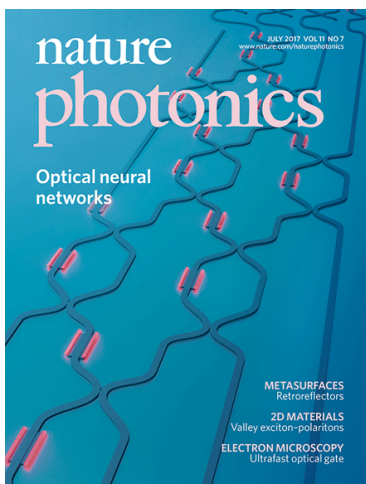
By enforcing hidden to hidden transition matrix to be unitary/orthogonal, no matter how many time steps are propagated, the norm of the gradient will stay the same

$$\left\| \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}^{(k+1)}}{\partial \mathbf{h}^{(k)}} \right\| \sim 1$$

Related Works

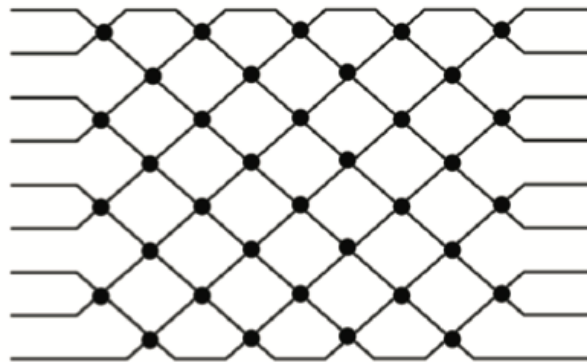
- Restricted-capacity Unitary Matrix Parametrization (Arjovsky, ICML 2016)
- Full-capacity Unitary Matrix by projection (Wisdom, NIPS 2016)
- Orthogonal Matrix Parametrization by reflection (Mhammedi, ICML 2017)
- Orthogonal Matrix by regularization (Vorontsov, ICML 2017)

Efficient Unitary Matrix Parametrization

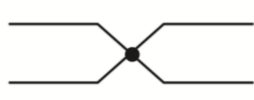


(Y. Shen, et al, Nature Photonics 2017)

Physical system inspired:



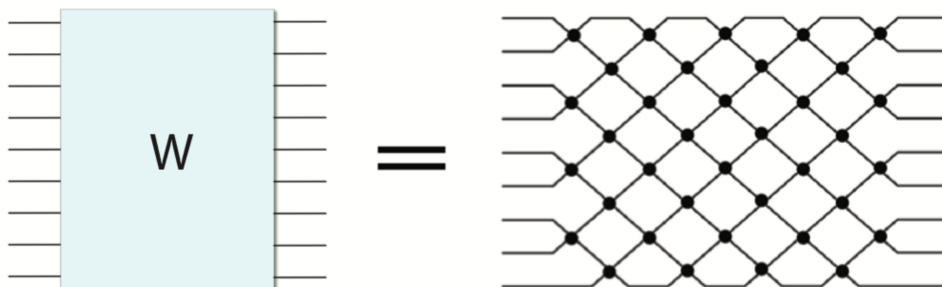
Efficient Unitary Matrix Parametrization



$$= \begin{pmatrix} e^{i\phi} \cos \theta & -e^{i\phi} \sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

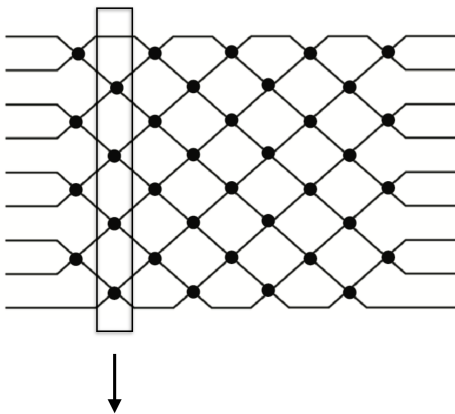
2-by-2 element

- SU(2) element: rotation matrix + relevant phase



- strict, complete, unique
- Full unitary space

Efficient Unitary Matrix Implementation



sparse block diagonal matrix
element-wise functions

Algorithm 1 Efficient implementation for $\mathbf{F}$ with parameter θ_i and ϕ_i .

Input: input $\mathbf{x}$, size N ; parameters θ and ϕ , size $N/2$; constant permutation index list $\mathbf{ind}_1$ and $\mathbf{ind}_2$.

Output: output $\mathbf{y}$, size N .

$\mathbf{v}_1 \leftarrow \text{concatenate}(\cos \theta, \cos \theta * \exp(i\phi))$

$\mathbf{v}_2 \leftarrow \text{concatenate}(\sin \theta, -\sin \theta * \exp(i\phi))$

$\mathbf{v}_1 \leftarrow \text{permute}(\mathbf{v}_1, \mathbf{ind}_1)$

$\mathbf{v}_2 \leftarrow \text{permute}(\mathbf{v}_2, \mathbf{ind}_1)$

$\mathbf{y} \leftarrow \mathbf{v}_1 * \mathbf{x} + \mathbf{v}_2 * \text{permute}(\mathbf{x}, \mathbf{ind}_2)$

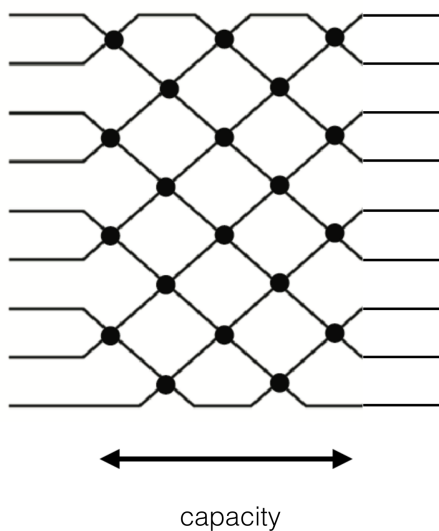
- Parallel, Sparse
- $O(1)$ per parameter
- No need to customize gradient

Tensorflow: <https://github.com/jingli9111/EUNN-tensorflow>

PyTorch: <https://github.com/jingli9111/URNN-PyTorch>

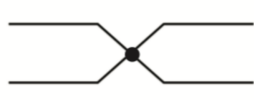
Theano: <https://github.com/iguanaus/EUNN-theano>

Tunable Parametrization



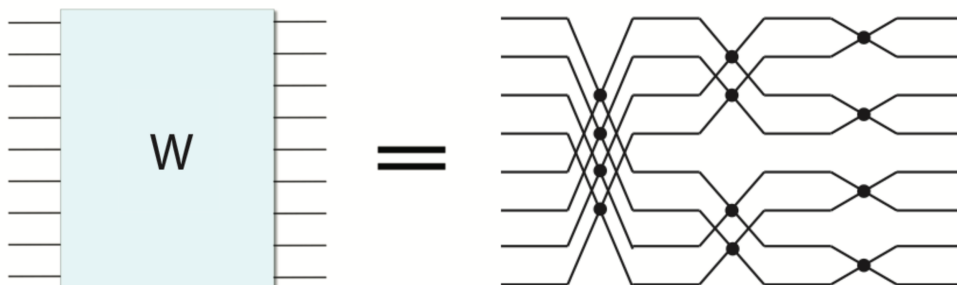
- reduce capacity allows larger hidden state
- reduce capacity increases training speed

FFT-style Parametrization



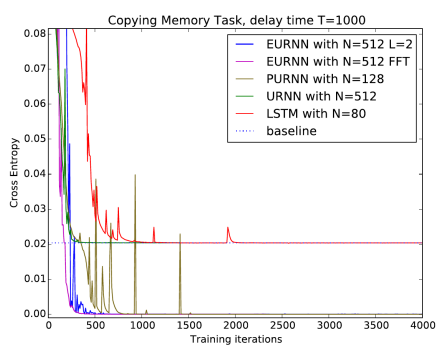
$$= \begin{pmatrix} e^{i\phi} \cos \theta & -e^{i\phi} \sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

- $n \log(n)$ parameters
- minimum number of parameters for symmetric unitary space
- almost same performance as Tunable style with large capacity



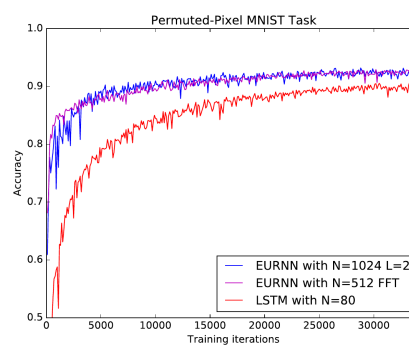
Experiment results

- Copying Memory



EURNN outperforms all other models for long delay time case.

- Pixel Permuted MNIST



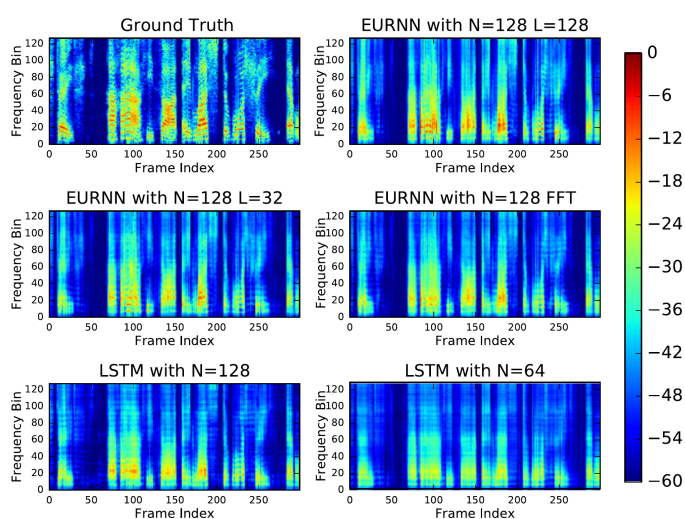
EURNN outperforms LSTM in both final performance and training speed.

EURNN tunable-style achieves highest accuracy with least number of parameters.

Model	hidden size (capacity)	number of parameters	validation accuracy	test accuracy
LSTM	80	16k	0.908	0.902
URNN	512	16k	0.942	0.933
PURNN	116	16k	0.922	0.921
EURNN (tunable style)	1024 (2)	13.3k	0.940	0.937
EURNN (FFT style)	512 (FFT)	9.0k	0.928	0.925

Experiment results

- Speech spectrum prediction



TIMIT spectrum sampled in 8 GHz. RNN model is required to predict next frame based on previous frames.

EURNN outperforms LSTM with less number of parameters

Model	hidden size (capacity)	number of parameters	MSE (validation)	MSE (test)
LSTM	64	33k	71.4	66.0
LSTM	128	98k	55.3	54.5
EURNN (tunable style)	128 (2)	33k	63.3	63.3
EURNN (tunable style)	128 (32)	35k	52.3	52.7
EURNN (tunable style)	128 (128)	41k	51.8	51.9
EURNN (FFT style)	128 (FFT)	34k	52.3	52.4

Conclusion

Efficient Unitary NN (EUNN)

- **Efficient:** $O(1)$ operation per parameter
- **Strict:** strictly unitary parametrization
- **Tunable:** from small subspace to full unitary space, trade off capacity to hidden size
- **Easy implementation:** element-wise functions, no need to implement backpropagation
- **FFT approximation:** provides further speed-up

Tunable Efficient Unitary Neural Networks (EUNN) and their application to RNNs

Li Jing^{*1} Yichen Shen^{*1} Tena Dubcek¹ John Peurifoy¹ Scott Skirlo¹ Yann LeCun² Max Tegmark¹
Marin Soljačić¹

Abstract

Using unitary (instead of general) matrices in artificial neural networks (ANNs) is a promising way to solve the gradient explosion/vanishing problem, as well as to enable ANNs to learn long-term correlations in the data. This approach appears particularly promising for Recurrent Neural Networks (RNNs). In this work, we present a new architecture for implementing an Efficient Unitary Neural Network (EUNNs); its main advantages can be summarized as follows. Firstly, the representation capacity of the unitary space in an EUNN is fully tunable, ranging from a subspace of $SU(N)$ to the entire unitary space. Secondly, the computational complexity for training an EUNN is merely $\mathcal{O}(1)$ per parameter. Finally, we test the performance of EUNNs on the standard copying task, the pixel-permuted MNIST digit recognition benchmark as well as the Speech Prediction Test (TIMIT). We find that our architecture significantly outperforms both other state-of-the-art unitary RNNs and the LSTM architecture, in terms of the final performance and/or the wall-clock training speed. EUNNs are thus promising alternatives to RNNs and LSTMs for a wide variety of applications.

1. Introduction

Deep Neural Networks (LeCun et al., 2015) have been successful on numerous difficult machine learning tasks, including image recognition (Krizhevsky et al., 2012; Donahue et al., 2015), speech recognition (Hinton et al., 2012) and natural language processing (Collobert et al., 2011; Bahdanau et al., 2014; Sutskever et al., 2014). However, deep neural networks can suffer from vanishing and ex-

ploding gradient problems (Hochreiter, 1991; Bengio et al., 1994), which are known to be caused by matrix eigenvalues far from unity being raised to large powers. Because the severity of these problems grows with the depth of a neural network, they are particularly grave for Recurrent Neural Networks (RNNs), whose recurrence can be equivalent to thousands or millions of equivalent hidden layers.

Several solutions have been proposed to solve these problems for RNNs. Long Short Term Memory (LSTM) networks (Hochreiter & Schmidhuber, 1997), which help RNNs contain information inside hidden layers with gates, remains one of the most popular RNN implementations. Other recently proposed methods such as GRUs (Cho et al., 2014) and Bidirectional RNNs (Berglund et al., 2015) also perform well in numerous applications. However, none of these approaches has fundamentally solved the vanishing and exploding gradient problems, and gradient clipping is often required to keep gradients in a reasonable range.

A recently proposed solution strategy is using orthogonal hidden weight matrices or their complex generalization (unitary matrices) (Saxe et al., 2013; Le et al., 2015; Arjovsky et al., 2015; Henaff et al., 2016), because all their eigenvalues will then have absolute values of unity, and can safely be raised to large powers. This has been shown to help both when weight matrices are initialized to be unitary (Saxe et al., 2013; Le et al., 2015) and when they are kept unitary during training, either by restricting them to a more tractable matrix subspace (Arjovsky et al., 2015) or by alternating gradient-descent steps with projections onto the unitary subspace (Wisdom et al., 2016).

In this paper, we will first present an Efficient Unitary Neural Network (EUNN) architecture that parametrizes the entire space of unitary matrices in a complete and computationally efficient way, thereby eliminating the need for time-consuming unitary subspace-projections. Our architecture has a wide range of capacity-tunability to represent subspace unitary models by fixing some of our parameters; the above-mentioned unitary subspace models correspond to special cases of our architecture. We also implemented an EUNN with an earlier introduced FFT-like architecture which efficiently approximates the unitary space with min-

^{*}Equal contribution ¹Massachusetts Institute of Technology
²New York University, Facebook AI Research. Correspondence to: Li Jing <ljing@mit.edu>, Yichen Shen <ycshen@mit.edu>.

imum number of required parameters(Mathieu & LeCun, 2014b).

We then benchmark EUNN’s performance on both simulated and real tasks: the standard copying task, the pixel-permuted MNIST task, and speech prediction with the TIMIT dataset (Garofolo et al., 1993). We show that our EUNN algorithm with an $\mathcal{O}(N)$ hidden layer size can compute up to the entire $N \times N$ gradient matrix using $\mathcal{O}(1)$ computational steps and memory access per parameter. This is superior to the $\mathcal{O}(N)$ computational complexity of the existing training method for a full-space unitary network (Wisdom et al., 2016) and $\mathcal{O}(\log N)$ more efficient than the subspace Unitary RNN(Arjovsky et al., 2015).

2. Background

2.1. Basic Recurrent Neural Networks

A recurrent neural network takes an input sequence and uses the current hidden state to generate a new hidden state during each step, memorizing past information in the hidden layer. We first review the basic RNN architecture.

Consider an RNN updated at regular time intervals $t = 1, 2, \dots$ whose input is the sequence of vectors $\mathbf{x}^{(t)}$ whose hidden layer $\mathbf{h}^{(t)}$ is updated according to the following rule:

$$\mathbf{h}^{(t)} = \sigma(\mathbf{U}\mathbf{x}^{(t)} + \mathbf{W}\mathbf{h}^{(t-1)}), \quad (1)$$

where σ is the nonlinear activation function. The output is generated by

$$\mathbf{y}^{(t)} = \mathbf{W}\mathbf{h}^{(t)} + \mathbf{b}, \quad (2)$$

where $\mathbf{b}$ is the bias vector for the hidden-to-output layer. For $t = 0$, the hidden layer $\mathbf{h}^{(0)}$ can be initialized to some special vector or set as a trainable variable. For convenience of notation, we define $\mathbf{z}^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + \mathbf{W}\mathbf{h}^{(t-1)}$ so that $\mathbf{h}^{(t)} = \sigma(\mathbf{z}^{(t)})$.

2.2. The Vanishing and Exploding Gradient Problems

When training the neural network to minimize a cost function C that depends on a parameter vector $\mathbf{a}$, the gradient descent method updates this vector to $\mathbf{a} - \lambda \frac{\partial C}{\partial \mathbf{a}}$, where λ is a fixed learning rate and $\frac{\partial C}{\partial \mathbf{a}} \equiv \nabla C$. For an RNN, the vanishing or exploding gradient problem is most significant during back propagation from hidden to hidden layers, so we will only focus on the gradient for hidden layers. Training the input-to-hidden and hidden-to-output matrices is relatively trivial once the hidden-to-hidden matrix has been successfully optimized.

In order to evaluate $\frac{\partial C}{\partial W_{ij}}$, one first computes the derivative

$\frac{\partial C}{\partial \mathbf{h}^{(t)}}$ using the chain rule:

$$\frac{\partial C}{\partial \mathbf{h}^{(t)}} = \frac{\partial C}{\partial \mathbf{h}^{(T)}} \frac{\partial \mathbf{h}^{(T)}}{\partial \mathbf{h}^{(t)}} \quad (3)$$

$$= \frac{\partial C}{\partial \mathbf{h}^{(T)}} \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}^{(k+1)}}{\partial \mathbf{h}^{(k)}} \quad (4)$$

$$= \frac{\partial C}{\partial \mathbf{h}^{(T)}} \prod_{k=t}^{T-1} \mathbf{D}^{(k)} \mathbf{W}, \quad (5)$$

where $\mathbf{D}^{(k)} = \text{diag}\{\sigma'(\mathbf{U}\mathbf{x}^{(k)} + \mathbf{W}\mathbf{h}^{(k-1)})\}$ is the Jacobian matrix of the pointwise nonlinearity. For large times T , the term $\prod \mathbf{W}$ plays a significant role. As long as the eigenvalues of $\mathbf{D}^{(k)}$ are of order unity, then if $\mathbf{W}$ has eigenvalues $\lambda_i \gg 1$, they will cause gradient explosion $|\frac{\partial C}{\partial \mathbf{h}^{(T)}}| \rightarrow \infty$, while if $\mathbf{W}$ has eigenvalues $\lambda_i \ll 1$, they can cause gradient vanishing, $|\frac{\partial C}{\partial \mathbf{h}^{(T)}}| \rightarrow 0$. Either situation prevents the RNN from working efficiently.

3. Unitary RNNs

3.1. Partial Space Unitary RNNs

In a breakthrough paper, Arjovsky, Shah & Bengio (Arjovsky et al., 2015) showed that unitary RNNs can overcome the exploding and vanishing gradient problems and perform well on long term memory tasks if the hidden-to-hidden matrix is parametrized in the following unitary form:

$$\mathbf{W} = \mathbf{D}_3 \mathbf{T}_2 \mathcal{F}^{-1} \mathbf{D}_2 \mathbf{\Pi} \mathbf{T}_1 \mathcal{F} \mathbf{D}_1. \quad (6)$$

Here $\mathbf{D}_{1,2,3}$ are diagonal matrices with each element $e^{i\omega_j}$, $j = 1, 2, \dots, n$. $\mathbf{T}_{1,2}$ are reflection matrices, and $\mathbf{T} = \mathbf{I} - 2 \frac{\hat{\mathbf{v}} \hat{\mathbf{v}}^\dagger}{\|\hat{\mathbf{v}}\|^2}$, where $\hat{\mathbf{v}}$ is a vector with each of its entries as a parameter to be trained. $\mathbf{\Pi}$ is a fixed permutation matrix. $\mathcal{F}$ and $\mathcal{F}^{-1}$ are Fourier and inverse Fourier transform matrices respectively. Since each factor matrix here is unitary, the product $\mathbf{W}$ is also a unitary matrix.

This model uses $\mathcal{O}(N)$ parameters, which spans merely a part of the whole $\mathcal{O}(N^2)$ -dimensional space of unitary $N \times N$ matrices to enable computational efficiency. Several subsequent papers have tried to expand the space to $\mathcal{O}(N^2)$ in order to achieve better performance, as summarized below.

3.2. Full Space Unitary RNNs

In order to maximize the power of Unitary RNNs, it is preferable to have the option to optimize the weight matrix $\mathbf{W}$ over the full space of unitary matrices rather than a subspace as above. A straightforward method for implementing this is by simply updating $\mathbf{W}$ with standard back-propagation and then projecting the resulting matrix (which will typically no longer be unitary) back onto to the space

of unitary matrices. Defining $\mathbf{G}_{ij} \equiv \frac{\partial C}{\partial W_{ij}}$ as the gradient with respect to $\mathbf{W}$, this can be implemented by the procedure defined by (Wisdom et al., 2016):

$$\begin{aligned} \mathbf{A}^{(t)} &\equiv \mathbf{G}^{(t)\dagger} \mathbf{W}^{(t)} - \mathbf{W}^{(t)\dagger} \mathbf{G}^{(k)}, \\ \mathbf{W}^{(t+1)} &\equiv \left(\mathbf{I} + \frac{\lambda}{2} \mathbf{A}^{(t)} \right)^{-1} \left(\mathbf{I} - \frac{\lambda}{2} \mathbf{A}^{(t)} \right) \mathbf{W}^{(t)} \end{aligned} \quad (7)$$

This method shows that full space unitary networks are superior on many RNN tasks (Wisdom et al., 2016). A key limitation is that the back-propagation in this method cannot avoid N -dimensional matrix multiplication, incurring $\mathcal{O}(N^3)$ computational cost.

4. Efficient Unitary Neural Network (EUNN) Architectures

In the following, we first describe a general parametrization method able to represent arbitrary unitary matrices with up to N^2 degrees of freedom. We then present an efficient algorithm for this parametrization scheme, requiring only $\mathcal{O}(1)$ computational and memory access steps to obtain the gradient for each parameter. Finally, we show that our scheme performs significantly better than the above mentioned methods on a few well-known benchmarks.

4.1. Unitary Matrix Parametrization

Any $N \times N$ unitary matrix $\mathbf{W}_N$ can be represented as a product of rotation matrices $\{\mathbf{R}_{ij}\}$ and a diagonal matrix $\mathbf{D}$, such that $\mathbf{W}_N = \mathbf{D} \prod_{i=2}^N \prod_{j=1}^{i-1} \mathbf{R}_{ij}$, where $\mathbf{R}_{ij}$ is defined as the N -dimensional identity matrix with the elements R_{ii} , R_{ij} , R_{ji} and R_{jj} replaced as follows (Reck et al., 1994; Clements et al., 2016):

$$\begin{pmatrix} R_{ii} & R_{ij} \\ R_{ji} & R_{jj} \end{pmatrix} = \begin{pmatrix} e^{i\phi_{ij}} \cos \theta_{ij} & -e^{i\phi_{ij}} \sin \theta_{ij} \\ \sin \theta_{ij} & \cos \theta_{ij} \end{pmatrix}. \quad (9)$$

where θ_{ij} and ϕ_{ij} are unique parameters corresponding to $\mathbf{R}_{ij}$. Each of these matrices performs a $U(2)$ unitary transformation on a two-dimensional subspace of the N -dimensional Hilbert space, leaving an $(N-2)$ -dimensional subspace unchanged. In other words, a series of $U(2)$ rotations can be used to successively make all off-diagonal elements of the given $N \times N$ unitary matrix zero. This generalizes the familiar factorization of a 3D rotation matrix into 2D rotations parametrized by the three Euler angles. To provide intuition for how this works, let us briefly describe a simple way of doing this that is similar to Gaussian elimination by finishing one column at a time. There are infinitely many alternative decomposition schemes as well; Fig. 1 shows two that are particularly convenient to implement in software (and even in neuromorphic hardware (Shen et al., 2016)). The unitary matrix $\mathbf{W}_N$ is multiplied from the right by a succession of unitary matrices

$\mathbf{R}_{Nj}$ for $j = N-1, \dots, 1$. Once all elements of the last row except the one on the diagonal are zero, this row will not be affected by later transformations. Since all transformations are unitary, the last column will then also contain only zeros except on the diagonal:

$$\mathbf{W}_N \mathbf{R}_{N,N-1} \mathbf{R}_{N,N-2} \cdots \mathbf{R}_{N,1} = \begin{pmatrix} \mathbf{W}_{N-1} & 0 \\ 0 & e^{iw_N} \end{pmatrix} \quad (10)$$

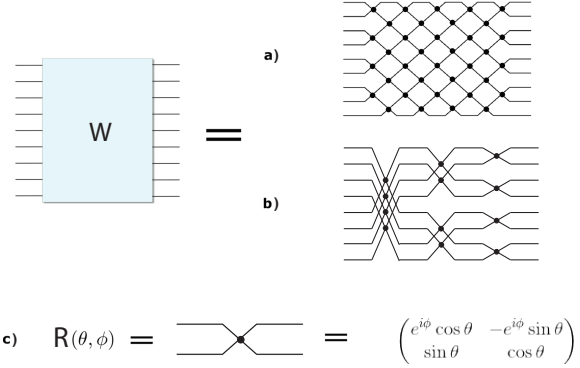


Figure 1. Unitary matrix decomposition: An arbitrary unitary matrix $\mathbf{W}$ can be decomposed (a) with the square decomposition method of Clements *et al.* (Clements et al., 2016) discussed in section 4.2; or approximated (b) by the Fast Fourier Transformation (FFT) style decomposition method (Mathieu & LeCun, 2014b) discussed in section 4.3. Each junction in the a) and b) graphs above represent the $U(2)$ matrix as shown in c).

The effective dimensionality of the the matrix $\mathbf{W}$ is thus reduced to $N-1$. The same procedure can then be repeated $N-1$ times until the effective dimension of $\mathbf{W}$ is reduced to 1, leaving us with a diagonal matrix:¹

$$\mathbf{W}_N \mathbf{R}_{N,N-1} \mathbf{R}_{N,N-2} \cdots \mathbf{R}_{i,j} \mathbf{R}_{i,j-1} \cdots \mathbf{R}_{3,1} \mathbf{R}_{2,1} = \mathbf{D}, \quad (11)$$

where $\mathbf{D}$ is a diagonal matrix whose diagonal elements are e^{iw_j} , from which we can write the direct representation of $\mathbf{W}_N$ as

$$\begin{aligned} \mathbf{W}_N &= \mathbf{D} \mathbf{R}_{2,1}^{-1} \mathbf{R}_{3,1}^{-1} \cdots \mathbf{R}_{N,N-2}^{-1} \mathbf{R}_{N,N-1}^{-1} \\ &= \mathbf{D} \mathbf{R}'_{2,1} \mathbf{R}'_{3,1} \cdots \mathbf{R}'_{N,N-2} \mathbf{R}'_{N,N-1}. \end{aligned} \quad (12)$$

where

$$\mathbf{R}'_{ij} = \mathbf{R}(-\theta_{ij}, -\phi_{ij}) = \mathbf{R}(\theta_{ij}, \phi_{ij})^{-1} = \mathbf{R}_{ij}^{-1} \quad (13)$$

¹Note that Gaussian Elimination would make merely the upper triangle of a matrix vanish, requiring a subsequent series of rotations (complete Gauss-Jordan Elimination) to zero the lower triangle. We need no such subsequent series because since $\mathbf{W}$ is unitary: it is easy to show that if a unitary matrix is triangular, it must be diagonal.

This parametrization thus involves $N(N-1)/2$ different θ_{ij} -values, $N(N-1)/2$ different ϕ_{ij} -values and N different w_i -values, combining to N^2 parameters in total and spans the entire unitary space. Note we can always fix a portion of our parameters, to span only a subset of unitary space – indeed, our benchmark test below will show that for certain tasks, full unitary space parametrization is not necessary.²

4.2. Tunable space implementation

The representation in Eq. 12 can be made more compact by reordering and grouping specific rotational matrices, as was shown in the optical community (Reck et al., 1994; Clements et al., 2016) in the context of universal multiport interferometers. For example (Clements et al., 2016), a unitary matrix can be decomposed as

$$\begin{aligned} \mathbf{W}_N &= \mathbf{D} \left(\mathbf{R}_{1,2}^{(1)} \mathbf{R}_{3,4}^{(1)} \dots \mathbf{R}_{N/2-1,N/2}^{(1)} \right) \\ &\quad \times \left(\mathbf{R}_{2,3}^{(2)} \mathbf{R}_{4,5}^{(2)} \dots \mathbf{R}_{N/2-2,N/2-1}^{(2)} \right) \\ &\quad \times \dots \\ &= \mathbf{D} \mathbf{F}_A^{(1)} \mathbf{F}_B^{(2)} \dots \mathbf{F}_B^{(L)}, \end{aligned} \quad (14)$$

where every

$$\mathbf{F}_A^{(l)} = \mathbf{R}_{1,2}^{(l)} \mathbf{R}_{3,4}^{(l)} \dots \mathbf{R}_{N/2-1,N/2}^{(l)}$$

is a block diagonal matrix, with N angle parameters in total, and

$$\mathbf{F}_B^{(l)} = \mathbf{R}_{2,3}^{(l)} \mathbf{R}_{4,5}^{(l)} \dots \mathbf{R}_{N/2-2,N/2-1}^{(l)}$$

with $N-1$ parameters, as is schematically shown in Fig. 1a. By choosing different values for L , $\mathbf{W}_N$ will span a different subspace of the unitary space. Specifically, when $L = N$, $\mathbf{W}_N$ will span the entire unitary space.

Following this physics-inspired scheme, we decompose our unitary hidden-to-hidden layer matrix $\mathbf{W}$ as

$$\mathbf{W} = \mathbf{D} \mathbf{F}_A^{(1)} \mathbf{F}_B^{(2)} \mathbf{F}_A^{(3)} \mathbf{F}_B^{(4)} \dots \mathbf{F}_B^{(L)}. \quad (15)$$

4.3. FFT-style approximation

Inspired by (Mathieu & LeCun, 2014a), an alternative way to organize the rotation matrices is implementing an FFT-style architecture. Instead of using adjacent rotation matrices, each $\mathbf{F}$ here performs a certain distance pairwise rotations as shown in Fig. 1b:

$$\mathbf{W} = \mathbf{D} \mathbf{F}_1 \mathbf{F}_2 \mathbf{F}_3 \mathbf{F}_4 \dots \mathbf{F}_{\log(N)}. \quad (16)$$

The rotation matrices in $\mathbf{F}_i$ are performed between pairs of coordinates

$$(2pk + j, p(2k + 1) + j) \quad (17)$$

²Our preliminary experimental tests even suggest that a full-capacity unitary RNN is even undesirable for some tasks.

where $p = \frac{N}{2^i}$, $k \in \{0, \dots, 2^{i-1}\}$ and $j \in \{1, \dots, p\}$. This requires only $\log(N)$ matrices, so there are a total of $N \log(N)/2$ rotational pairs. This is also the minimal number of rotations that can have all input coordinates interacting with each other, providing an approximation of arbitrary unitary matrices.

4.4. Efficient implementation of rotation matrices

To implement this decomposition efficiently in an RNN, we apply vector element-wise multiplications and permutations: we evaluate the product $\mathbf{F}\mathbf{x}$ as

$$\mathbf{F}\mathbf{x} = \mathbf{v}_1 * \mathbf{x} + \mathbf{v}_2 * \text{permute}(\mathbf{x}) \quad (18)$$

where $*$ represents element-wise multiplication, $\mathbf{F}$ refers to general rotational matrices such as $\mathbf{F}_{A/B}$ in Eq. 14 and $\mathbf{F}_i$ in Eq. 16. For the case of the tunable-space implementation, if we want to implement $\mathbf{F}_A^{(l)}$ in Eq. 14, we define $\mathbf{v}$ and the permutation as follows:

$$\begin{aligned} \mathbf{v}_1 &= (e^{i\phi_1^{(l)}} \cos \theta_1^{(l)}, \cos \theta_1^{(l)}, e^{i\phi_2^{(l)}} \cos \theta_2^{(l)}, \cos \theta_2^{(l)}, \dots) \\ \mathbf{v}_2 &= (-e^{i\phi_1^{(l)}} \sin \theta_1^{(l)}, \sin \theta_1^{(l)}, -e^{i\phi_2^{(l)}} \sin \theta_2^{(l)}, \sin \theta_2^{(l)}, \dots) \\ \text{permute}(\mathbf{x}) &= (x_2, x_1, x_4, x_3, x_6, x_5, \dots). \end{aligned}$$

For the FFT-style approach, if we want to implement $\mathbf{F}_1$ in Eq. 16, we define $\mathbf{v}$ and the permutation as follows:

$$\begin{aligned} \mathbf{v}_1 &= (e^{i\phi_1^{(l)}} \cos \theta_1^{(l)}, e^{i\phi_2^{(l)}} \cos \theta_2^{(l)}, \dots, \cos \theta_1^{(l)}, \cos \theta_2^{(l)}, \dots) \\ \mathbf{v}_2 &= (-e^{i\phi_1^{(l)}} \sin \theta_1^{(l)}, -e^{i\phi_2^{(l)}} \sin \theta_2^{(l)}, \dots, \sin \theta_1^{(l)}, \sin \theta_2^{(l)}, \dots) \\ \text{permute}(\mathbf{x}) &= (x_{\frac{n}{2}+1}, x_{\frac{n}{2}+2}, \dots, x_n, x_1, x_2, \dots). \end{aligned}$$

In general, the pseudocode for implementing operation $\mathbf{F}$ is as follows:

Algorithm 1 Efficient implementation for $\mathbf{F}$ with parameter θ_i and ϕ_i .

Input: input $\mathbf{x}$, size N ; parameters θ and ϕ , size $N/2$; constant permutation index list ind_1 and ind_2 .

Output: output $\mathbf{y}$, size N .

$\mathbf{v}_1 \leftarrow \text{concatenate}(\cos \theta, \cos \theta * \exp(i\phi))$

$\mathbf{v}_2 \leftarrow \text{concatenate}(\sin \theta, -\sin \theta * \exp(i\phi))$

$\mathbf{v}_1 \leftarrow \text{permute}(\mathbf{v}_1, \text{ind}_1)$

$\mathbf{v}_2 \leftarrow \text{permute}(\mathbf{v}_2, \text{ind}_1)$

$\mathbf{y} \leftarrow \mathbf{v}_1 * \mathbf{x} + \mathbf{v}_2 * \text{permute}(\mathbf{x}, \text{ind}_2)$

Note that ind_1 and ind_2 are different for different $\mathbf{F}$.

From a computational complexity viewpoint, since the operations $*$ and permute take $\mathcal{O}(N)$ computational steps, evaluating $\mathbf{F}\mathbf{x}$ only requires $\mathcal{O}(N)$ steps. The product $\mathbf{D}\mathbf{x}$ is trivial, consisting of an element-wise vector multiplication. Therefore, the product $\mathbf{W}\mathbf{x}$ with the total unitary

matrix $\mathbf{W}$ can be computed in only $\mathcal{O}(NL)$ steps, and only requires $\mathcal{O}(NL)$ memory access (for full-space implementation $L = N$, for FFT-style approximation gives $L = \log N$). A detailed comparison on computational complexity of the existing unitary RNN architectures is given in Table 1.

4.5. Nonlinearity

We use the same nonlinearity as (Arjovsky et al., 2015):

$$(\text{modReLU}(\mathbf{z}, \mathbf{b}))_i = \frac{z_i}{|z_i|} * \text{ReLU}(|z_i| + b_i) \quad (19)$$

where the bias vector $\mathbf{b}$ is a shared trainable parameter, and $|z_i|$ is the norm of the complex number z_i .

For real number input, modReLU can be simplified to:

$$(\text{modReLU}(\mathbf{z}, \mathbf{b}))_i = \text{sign}(z_i) * \text{ReLU}(|z_i| + b_i) \quad (20)$$

where $|z_i|$ is the absolute value of the real number z_i .

We empirically find that this nonlinearity function performs the best. We believe that this function possibly also serves as a forgetting filter that removes the noise using the bias threshold.

5. Experimental tests of our method

In this section, we compare the performance of our Efficient Unitary Recurrent Neural Network (EURNN) with

1. an LSTM RNN (Hochreiter & Schmidhuber, 1997),
2. a Partial Space URNN (Arjovsky et al., 2015), and
3. a Projective full-space URNN (Wisdom et al., 2016).

All models are implemented in both Tensorflow and Theano, available from <https://github.com/jingli9111/EUNN-tensorflow> and <https://github.com/iguanaus/EUNN-theano>.

5.1. Copying Memory Task

We compare these networks by applying them all to the well defined *Copying Memory Task* (Hochreiter & Schmidhuber, 1997; Arjovsky et al., 2015; Henaff et al., 2016). The copying task is a synthetic task that is commonly used to test the network’s ability to remember information seen T time steps earlier.

Specifically, the task is defined as follows (Hochreiter & Schmidhuber, 1997; Arjovsky et al., 2015; Henaff et al., 2016). An alphabet consists of symbols $\{a_i\}$, the first n of which represent data, and the remaining two representing “blank” and “start recall”, respectively; as illustrated by the following example where $T = 20$ and $M = 5$:

Input: BACCA-----:----
Output: -----BACCA

In the above example, $n = 3$ and $\{a_i\} = \{A, B, C, -, :\}$. The input consists of M random data symbols ($M = 5$ above) followed by $T - 1$ blanks, the “start recall” symbol and M more blanks. The desired output consists of $M + T$ blanks followed by the data sequence. The cost function C is defined as the cross entropy of the input and output sequences, which vanishes for perfect performance.

We use $n = 8$ and input length $M = 10$. The symbol for each input is represented by an n -dimensional one-hot vector. We trained all five RNNs for $T = 1000$ with the same batch size 128 using RMSProp optimization with a learning rate of 0.001. The decay rate is set to 0.5 for EURNN, and 0.9 for all other models respectively. (Fig. 2). This results show that the EURNN architectures introduced in both Sec.4.2 (EURNN with $N=512$, selecting $L=2$) and Sec.4.3 (FFT-style EURNN with $N=512$) outperform the LSTM model (which suffers from long term memory problems and only performs well on the copy task for small time delays T) and all other unitary RNN models, both in-terms of learnability and in-terms of convergence rate. Note that the only other unitary RNN model that is able to beat the baseline for $T = 1000$ (Wisdom et al., 2016) is significantly slower than our method.

Moreover, we find that by either choosing smaller L or by using the FFT-style method (so that $\mathbf{W}$ spans a smaller unitary subspace), the EURNN converges toward optimal performance significantly more efficiently (and also faster in wall clock time) than the partial (Arjovsky et al., 2015) and projective (Wisdom et al., 2016) unitary methods. The EURNN also performed more robustly. This means that a full-capacity unitary matrix is not necessary for this particular task.

5.2. Pixel-Permuted MNIST Task

The MNIST handwriting recognition problem is one of the classic benchmarks for quantifying the learning ability of neural networks. MNIST images are formed by a 28×28 grayscale image with a target label between 0 and 9.

To test different RNN models, we feed all pixels of the MNIST images into the RNN models in 28×28 time steps, where one pixel at a time is fed in as a floating-point number. A fixed random permutation is applied to the order of input pixels. The output is the probability distribution quantifying the digit prediction. We used RMSProp with a learning rate of 0.0001 and a decay rate of 0.9, and set the batch size to 128.

As shown in Fig. 3, EURNN significantly outperforms LSTM with the same number of parameters. It learns faster, in fewer iteration steps, and converges to a higher classifi-

Table 1. Performance comparison of four Recurrent Neural Network algorithms: URNN (Arjovsky et al., 2015), PURNN (Wisdom et al., 2016), and EUNN (our algorithm). T denotes the RNN length and N denotes the hidden state size. For the tunable-style EUNN, L is an integer between 1 and N parametrizing the unitary matrix capacity.

Model	Time complexity of one online gradient step	number of parameters in the hidden matrix	Transition matrix search space
URNN	$\mathcal{O}(TN \log N)$	$\mathcal{O}(N)$	subspace of $\mathbf{U}(N)$
PURNN	$\mathcal{O}(TN^2 + N^3)$	$\mathcal{O}(N^2)$	full space of $\mathbf{U}(N)$
EUNN (tunable style)	$\mathcal{O}(TNL)$	$\mathcal{O}(NL)$	tunable space of $\mathbf{U}(N)$
EUNN (FFT style)	$\mathcal{O}(TN \log N)$	$\mathcal{O}(N \log N)$	subspace of $\mathbf{U}(N)$

Table 2. MNIST Task result. EUNN corresponds to our algorithm, PURNN corresponds to algorithm presented in (Wisdom et al., 2016), URNN corresponds to the algorithm presented in (Arjovsky et al., 2015).

Model	hidden size (capacity)	number of parameters	validation accuracy	test accuracy
LSTM	80	16k	0.908	0.902
URNN	512	16k	0.942	0.933
PURNN	116	16k	0.922	0.921
EUNN (tunable style)	1024 (2)	13.3k	0.940	0.937
EUNN (FFT style)	512 (FFT)	9.0k	0.928	0.925

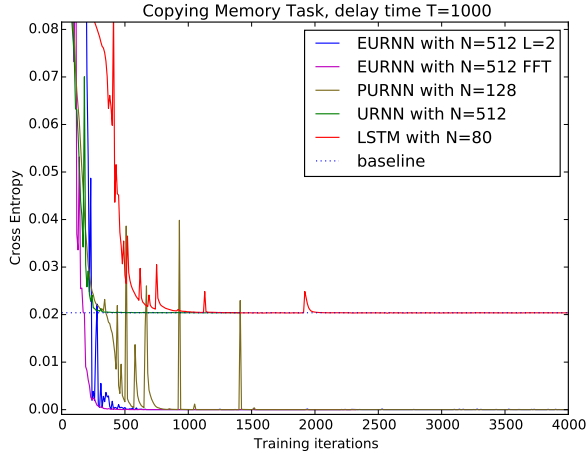


Figure 2. Copying Task for $T = 1000$. EUNN corresponds to our algorithm, projective URNN corresponds to algorithm presented in (Wisdom et al., 2016), URNN corresponds to the algorithm presented in (Arjovsky et al., 2015). A useful baseline performance is that of the memoryless strategy, which outputs $M+T$ blanks followed by M random data symbols and produces a cross entropy $C = (M \log n)/(T + 2 * M)$. [Note that each iteration for PURNN takes about 32 times longer than for EUNN models, for this particular simulation, so the speed advantage is much greater than apparent in this plot.]

cation accuracy. In addition, the EUNN reaches a similar accuracy with fewer parameters. In Table. 2, we compare the performance of different RNN models on this task.

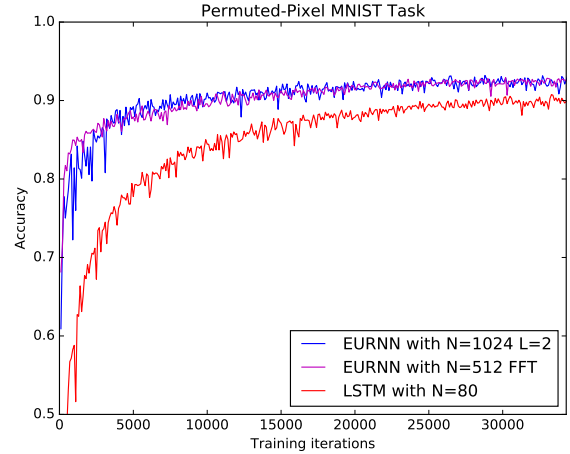


Figure 3. Pixel-permuted MNIST performance on the validation dataset.

5.3. Speech Prediction on TIMIT dataset

We also apply our EUNN to real-world speech prediction task and compare its performance to LSTM. The main task we consider is predicting the log-magnitude of future frames of a short-time Fourier transform (STFT) (Wisdom et al., 2016; Sejdi et al., 2009). We use the TIMIT dataset (Garofolo et al., 1993) sampled at 8 kHz. The audio .wav file is initially diced into different time frames (all frames have the same duration referring to the Hann analysis window below). The audio amplitude in each frame is then

Table 3. Speech Prediction Task result. EURNN corresponds to our algorithm, projective URNN corresponds to algorithm presented in (Wisdom et al., 2016), URNN corresponds to the algorithm presented in (Arjovsky et al., 2015).

Model	hidden size (capacity)	number of parameters	MSE (validation)	MSE (test)
LSTM	64	33k	71.4	66.0
LSTM	128	98k	55.3	54.5
EURNN (tunable style)	128 (2)	33k	63.3	63.3
EURNN (tunable style)	128 (32)	35k	52.3	52.7
EURNN (tunable style)	128 (128)	41k	51.8	51.9
EURNN (FFT style)	128 (FFT)	34k	52.3	52.4

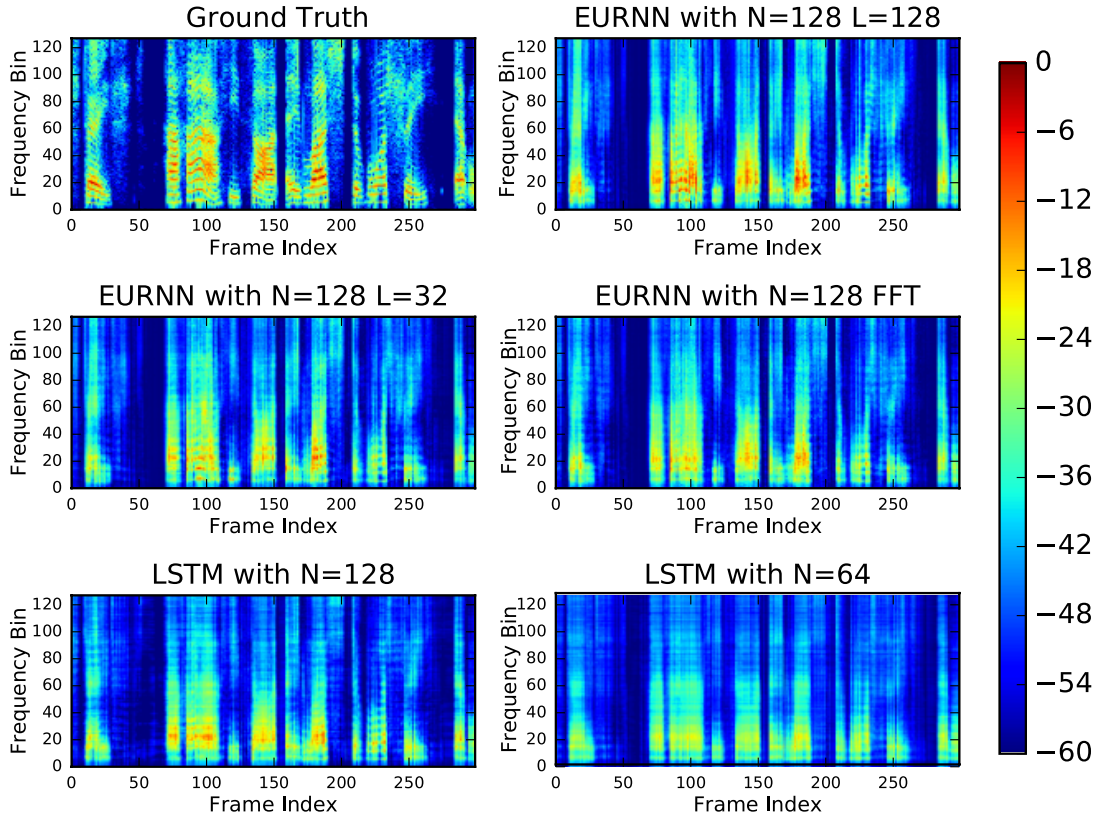


Figure 4. Example spectrograms of ground truth and RNN prediction results from evaluation sets.

Fourier transformed into the frequency domain. The log-magnitude of the Fourier amplitude is normalized and used as the data for training/testing each model. In our STFT operation we use a Hann analysis window of 256 samples (32 milliseconds) and a window hop of 128 samples (16 milliseconds). The frame prediction task is as follows: given all the log-magnitudes of STFT frames up to time t , predict the log-magnitude of the STFT frame at time $t + 1$ that has the minimum mean square error (MSE). We use

a training set with 2400 utterances, a validation set of 600 utterances and an evaluation set of 1000 utterances. The training, validation, and evaluation sets have distinct speakers. We trained all RNNs for with the same batch size 32 using RMSProp optimization with a learning rate of 0.001, a momentum of 0.9 and a decay rate of 0.1.

The results are given in Table 3, in terms of the mean-squared error (MSE) loss function. Figure 4 shows prediction examples from the three types of networks, illustrat-

ing how EURNNs generally perform better than LSTMs. Furthermore, in this particular task, full-capacity EURNNs outperform small capacity EURNNs and FFT-style EURNNs.

6. Conclusion

We have presented a method for implementing an Efficient Unitary Neural Network (EUNN) whose computational cost is merely $\mathcal{O}(1)$ per parameter, which is $\mathcal{O}(\log N)$ more efficient than the other methods discussed above. It significantly outperforms existing RNN architectures on the standard Copying Task, and the pixel-permuted MNIST Task using a comparable parameter count, hence demonstrating the highest recorded ability to memorize sequential information over long time periods.

It also performs well on real tasks such as speech prediction, outperforming an LSTM on TIMIT data speech prediction.

We want to emphasize the generality and tunability of our method. The ordering of the rotation matrices we presented in Fig. 1 are merely two of many possibilities; we used it simply as a concrete example. Other ordering options that can result in spanning the full unitary matrix space can be used for our algorithm as well, with identical speed and memory performance. This tunability of the span of the unitary space and, correspondingly, the total number of parameters makes it possible to use different capacities for different tasks, thus opening the way to an optimal performance of the EUNN. For example, as we have shown, a small subspace of the full unitary space is preferable for the copying task, whereas the MNIST task and TIMIT task are better performed by EUNN covering a considerably larger unitary space. Finally, we note that our method remains applicable even if the unitary matrix is decomposed into a different product of matrices (Eq. 12).

This powerful and robust unitary RNN architecture also might be promising for natural language processing because of its ability to efficiently handle tasks with long-term correlation and very high dimensionality.

Acknowledgment

We thank Hugo Larochelle and Yoshua Bengio for helpful discussions and comments.

This work was partially supported by the Army Research Office through the Institute for Soldier Nanotechnologies under contract W911NF-13-D0001, the National Science Foundation under Grant No. CCF-1640012 and the Rothberg Family Fund for Cognitive Science.

References

- Arjovsky, Martin, Shah, Amar, and Bengio, Yoshua. Unitary evolution recurrent neural networks. *arXiv preprint arXiv:1511.06464*, 2015.
- Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- Bengio, Yoshua, Simard, Patrice, and Frasconi, Paolo. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2): 157–166, 1994.
- Berglund, Mathias, Raiko, Tapani, Honkala, Mikko, Kärkkäinen, Leo, Vetek, Akos, and Karhunen, Juha T. Bidirectional recurrent neural networks as generative models. In *Advances in Neural Information Processing Systems*, pp. 856–864, 2015.
- Cho, Kyunghyun, Van Merriënboer, Bart, Bahdanau, Dzmitry, and Bengio, Yoshua. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.
- Clements, William R., Humphreys, Peter C., Metcalf, Benjamin J., Kolthammer, W. Steven, and Walmsley, Ian A. An optimal design for universal multiport interferometers, 2016. arXiv:1603.08788.
- Collobert, Ronan, Weston, Jason, Bottou, Léon, Karlen, Michael, Kavukcuoglu, Koray, and Kuksa, Pavel. Natural language processing (almost) from scratch. *Journal of Machine Learning Research*, 12(Aug):2493–2537, 2011.
- Donahue, Jeffrey, Anne Hendricks, Lisa, Guadarrama, Sergio, Rohrbach, Marcus, Venugopalan, Subhashini, Saenko, Kate, and Darrell, Trevor. Long-term recurrent convolutional networks for visual recognition and description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2625–2634, 2015.
- Garofolo, John S, Lamel, Lori F, Fisher, William M, Fiscus, Jonathon G, and Pallett, David S. Darpa timit acoustic-phonetic continuous speech corpus cd-rom. nist speech disc 1-1.1. *NASA STI/Recon technical report n*, 93, 1993.
- Henaff, Mikael, Szlam, Arthur, and LeCun, Yann. Orthogonal rnns and long-memory tasks. *arXiv preprint arXiv:1602.06662*, 2016.
- Hinton, Geoffrey, Deng, Li, Yu, Dong, Dahl, George E, Mohamed, Abdel-rahman, Jaitly, Navdeep, Senior, Andrew, Vanhoucke, Vincent, Nguyen, Patrick, Sainath,